

On-Device Training Library

Complete Code Reference — All Files, All Functions

[C Syntax](#) · [Architecture](#) · [Relationships](#) · [Detailed Function Guide](#)

This document is a comprehensive, teaching-style reference for the complete On-Device Training (ODT) C library. It covers: (1) a C language syntax reference chapter explaining every C concept used in the project, (2) the full project architecture with dependency maps, (3) detailed documentation for every source file including purpose, relationships, and all public functions with syntax, parameters, return values, worked examples, and design notes, and (4) a complete quick-reference index.

The library implements a neural-network training and inference engine for microcontrollers (STM32 Nucleo-F746ZG). It supports quantized tensors, element-wise arithmetic, convolutional layers, dense layers, loss functions, SGD optimiser, data loading from NumPy files, serialisation, and a clean user API — all written in portable C99.

Chapter 1 — Project Architecture & File Relationships

The library is organised into eight layers. Each layer builds on the one below. A file in layer N includes files from layer N or lower — never higher. This keeps the dependency graph a directed acyclic graph (DAG), which means you can always understand any file without needing to read the files above it.

Layer	Files	Role
0 — Foundation	Common.h DTypes.h / DTypes.c	Debug macros and raw byte ↔ typed-value conversion. No project dependencies.
1 — Data Types	Quantization.h / .c Rounding.h / .c	Enum of numeric schemes (INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL), config structs, init functions, and rounding modes.
2 — Tensor	Tensor.h / Tensor.c	Core tensor_t struct (data buffer + shape + quantization + sparsity). All size/index/shape calculations. BOOL bit-packing.
3 — Conversion	TensorConversion.h / .c	Convert between numeric types. 6×6 dispatch table (conversionMatrix). Dynamic and fixed-scale re-quantization.
4 — Arithmetic	Arithmetic.h / .c Add, Sub, Mul, Div, Matmul, MinMax, Log, Sum, Square, Rounding, Comparison, Kernel, SlidingWindow, Conv1dKernel, ...	Generic element-wise engine (function-pointer pattern). Concrete operation files wrap single math functions and pass them to the engine. Matmul has its own loop.
5 — Layers	Layer.h / .c Linear, Conv1d, Relu, Softmax, Dropout, Flatten, LayerNorm, AvgPool1d, MaxPool1d, ...	Neural-network layers. Each has a forward pass (inference) and backward pass (gradient computation). Built on Layer 4 arithmetic and Layer 3 conversion.
6 — Training	LossFunction, CrossEntropy, MSE, Optimizer, Sgd, DataLoader, NPYLoader, RNG, Bernoulli, Serialize, Deserialize	Loss computation, SGD weight update, dataset loading from .npy files, random-number generation for Dropout, and binary model serialisation.
7 — User API	src/userApi/* InferenceApi, TrainingLoopApi, StateDictApi, StorageApi, TensorApi, QuantizationApi, LayerWeightsApi, ModelValidationApi, + all layer/optimizer wrappers	High-level entry points the application calls to build, train, save, and run a model. Wraps everything below.

1.1 Dependency Map — which file includes which

Every arrow means 'includes'. Reading top-to-bottom: Foundation files are included by everyone; User API files include almost everything.

File	Directly includes (project headers only)
Common.h	— none (only standard library)
DTypes.h / DTypes.c	— none (only stddef.h, stdint.h)
Rounding.h / .c	— none
Quantization.h / .c	Rounding.h

Tensor.h / Tensor.c	Common.h, DTypes.h, Quantization.h
TensorConversion.h	Tensor.h
TensorConversion.c	Common.h, DTypes.h, Tensor.h, Quantization.h
Arithmetic.h / .c	Common.h, DTypes.h, Tensor.h, Matmul.h
Add.h / Add.c	Arithmetic.h, Tensor.h
Sub.h / Sub.c	Arithmetic.h, Tensor.h
Mul.h / Mul.c	Arithmetic.h, Tensor.h
Div.h / Div.c	Arithmetic.h, Tensor.h
Matmul.h / Matmul.c	DTypes.h, Tensor.h
MinMax.h / .c	DTypes.h (reads bytes directly)
Sum.h / Sum.c	— (operates on plain arrays)
Log.h / Log.c	— (math.h only)
Rounding.h / .c	stdint.h only
Layer.h / Layer.c	Tensor.h
Linear.h / Linear.c	Layer.h, Matmul.h, Add.h, TensorConversion.h
Conv1d.h / Conv1d.c	Layer.h, Kernel.h, Conv1dKernel.h, Add.h
Relu.h / Relu.c	Layer.h, Comparison.h, Arithmetic.h
Softmax.h / .c	Layer.h, Arithmetic.h, Sum.h, Log.h
Dropout.h / .c	Layer.h, Bernoulli.h
LayerNorm.h / .c	Layer.h, Arithmetic.h, Sum.h
LossFunction.h	Tensor.h
CrossEntropy.h / .c	LossFunction.h, Log.h, Sum.h, Arithmetic.h
MSE.h / MSE.c	LossFunction.h, Arithmetic.h, Sub.h
Optimizer.h	Layer.h, Quantization.h, Tensor.h
Sgd.h / Sgd.c	Optimizer.h, Arithmetic.h, Mul.h, Sub.h
DataLoader.h / .c	Dataset.h, Tensor.h
NPYLoader.h / .c	DataLoader.h, DTypes.h
Serialize.h / .c	Layer.h (includes Tensor.h transitively)
Deserialize.h / .c	Layer.h, DTypes.h
RNG.h / RNG.c	— none (stdint.h only)
Bernoulli.h / .c	RNG.h, Tensor.h
src/userApi/*	All layers above as needed

1.2 How a Forward Pass Works — Call Chain

When you call `forwardPass(model, input, output)` from the User API, here is the sequence of function calls that runs for a Linear layer:

```
InferenceApi → linearForward(layer, input, output) [Linear.c]
→ matmulSymInt32Tensors(weights, input, output) [Matmul.c]
→ readBytesAsInt32(&weights->data[byteIdx]) [DTypes.c]
→ writeInt32ToByteArray(result, &output->data[...]) [DTypes.c]
→ addSymInt32Tensors(output, bias, output) [Add.c]
→ int32PointWiseArithmetic(out, bias, addInt32s, out) [Arithmetic.c]
→ calcIndicesByRowIndex(...) [Arithmetic.c]
→ calcElementIndexByIndices(...) [Arithmetic.c]
→ readBytesAsInt32(...) [DTypes.c]
→ addInt32s(a, b) [Add.c]
→ writeInt32ToByteArray(...) [DTypes.c]
→ requantSymInt32Tensor(output, output) [TensorConversion.c]
→ findAbsMaxInt32(output->data, n) [MinMax.c]
→ roundByMode(value, HALF_AWAY) [Rounding.c]
```

Every layer ultimately comes down to DTypes.c functions reading and writing bytes. Arithmetic.c provides the loop and index logic. The specific math function (addInt32s, mulInt32s, etc.) is just a tiny leaf at the bottom of this chain.

Chapter 2 — C Language Syntax Reference

This chapter explains every C language concept used in the ODT library. If you see an unfamiliar piece of syntax in any of the source files, look it up here first before reading the file description.

2.1 Header Guards — `#ifndef` / `#define` / `#endif`

```
#ifndef ODT_TENSOR_H
#define ODT_TENSOR_H
/* ... all declarations ... */
#endif // ODT_TENSOR_H
```

Every .h file starts with a header guard. The three lines work together to prevent the same header from being included twice in the same translation unit. When the compiler first sees `#include "Tensor.h"`, it enters the file, finds `#ifndef ODT_TENSOR_H` (meaning 'if this macro is NOT yet defined'), the condition is true, so it defines `ODT_TENSOR_H` and processes everything inside. If any other file also includes `Tensor.h`, the compiler sees the file again but `#ifndef ODT_TENSOR_H` is now FALSE (the macro is already defined), so it skips everything to `#endif`. Without guards, the struct definitions would be duplicated and the compiler would report 'redefinition' errors.

2.2 typedef struct — Creating Named Structure Types

```
typedef struct shape {
    size_t numberOfDimensions;
    size_t *dimensions;
    size_t *orderOfDimensions;
} shape_t;
```

This does two things at once: it defines a struct (a group of variables bundled together under one name) AND creates a type alias for it. Without typedef you would have to write 'struct shape' everywhere. With typedef you can write just 'shape_t'. The convention in this project is to suffix every typedef'd type name with '_t' to make it immediately obvious it is a type and not a variable.

The fields inside are: `numberOfDimensions` (an integer telling you how many axes the tensor has), `dimensions` (a POINTER to an array of sizes — one per axis), and `orderOfDimensions` (a pointer to an array that says which physical slot corresponds to each logical axis — used for transpose).

2.3 Pointers — `uint8_t *`, `tensor_t *`, `size_t *`

```
uint8_t *data; // pointer to byte array
tensor_t *inputTensor; // pointer to a tensor_t struct
size_t *dims; // pointer to size_t array
// Dereference (read the value at the pointer):
uint8_t first_byte = data[0]; // same as *(data + 0)
uint8_t third_byte = data[2]; // same as *(data + 2)
// & means 'address of' (get a pointer to this variable):
readBytesAsInt32(&tensor->data[byteIndex]);
// &tensor->data[byteIndex] means: address of element byteIndex in data array
```

A pointer is a variable that stores a memory address. When you declare `'uint8_t *data'`, you are saying 'data holds the address of a `uint8_t`'. You can then use `data[i]` to read the byte at address (data + i). Pointers are used everywhere in this library because tensors hold large amounts of data — passing a pointer is cheap (8 bytes on 64-bit), but copying the entire `tensor_t` and all its data would be slow and wasteful.

The `->` operator accesses a struct field THROUGH a pointer. `'tensor->shape'` means: follow the pointer `'tensor'`, then get its `'shape'` field. It is exactly equivalent to `(*tensor).shape`.

2.4 typedef enum — Named Integer Constants

```
typedef enum qtype {
    INT32, // = 0
    FLOAT32, // = 1
    SYM_INT32, // = 2
    SYM, // = 3
    ASYM, // = 4
    BOOL // = 5
} qtype_t;
// Usage:
qtype_t myType = SYM_INT32;
if (myType == FLOAT32) { ... }
```

An enum (enumeration) gives friendly names to a set of integer values. The compiler assigns 0 to the first name, 1 to the second, and so on. You could write `'int myType = 2'` to mean `SYM_INT32`, but using the name `'SYM_INT32'` makes the code self-documenting. In this library the `qtype_t` enum is used as the row and column index of the 6×6 `conversionMatrix` — that is why the exact integer values matter.

2.5 Function Pointers — typedef int32_t (*fn_t)(int32_t, int32_t)

```
// Step 1 – declare the TYPE of function pointer:
typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);
// Step 2 – write a function that matches the type:
int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
int32_t mulInt32s(int32_t a, int32_t b) { return a * b; }
// Step 3 – store the function in a variable of that type:
int32ElementArithmeticFunc_t myOp = addInt32s; // no () – not calling it yet
// Step 4 – call it through the pointer:
int32_t result = myOp(3, 4); // calls addInt32s(3, 4) → 7
// Step 5 – pass it as a parameter to another function:
int32PointWiseArithmetic(aTensor, bTensor, addInt32s, outputTensor);
int32PointWiseArithmetic(aTensor, bTensor, mulInt32s, outputTensor);
```

A function pointer is a pointer that points to a function instead of data. The typedef creates a human-readable name for the pointer type. Breaking down the syntax: `'typedef'` says we are creating a new type name; `'int32_t'` is the return type; `'(*int32ElementArithmeticFunc_t)'` gives the new type its name (the `*` means it is a pointer); `'(int32_t a, int32_t b)'` says the function takes two `int32_t` arguments. This pattern is the heart of `Arithmetic.c` — by passing different functions as arguments, one generic loop handles addition, subtraction, multiplication, and any other element-wise operation without code duplication.

2.6 VLA — Variable Length Arrays (Stack-allocated runtime-sized arrays)

```
size_t numberOfDims = tensor->shape->numberOfDimensions; // known only at runtime
// VLA: array size is a variable, not a compile-time constant
size_t aOrderedDims[numberOfDims]; // allocated on the STACK
size_t aIndices[numberOfDims];
// Compare with regular fixed-size array (NOT a VLA):
size_t fixedArray[4]; // size 4 is a compile-time constant
```

A VLA (Variable Length Array) lets you declare a stack array whose size is determined at runtime rather than at compile time. This is a C99 feature. The array lives on the stack (not the heap), so no malloc/free is needed. It is automatically freed when the function returns. In this library VLAs are used in `doDimensionsMatch`, `int32PointWiseArithmetic`, and similar functions where the tensor rank (`numberOfDimensions`) is not known at compile time. Important: on microcontrollers with small stacks, VLAs must be kept small to avoid a stack overflow. In this library the arrays are at most `numberOfDimensions` elements, which is typically 1–4.

2.7 sizeof, memcpy, memmove, memset

```
#include <string.h> // for memcpy, memmove, memset
// sizeof – compile-time size of a type in bytes
size_t bytesPerElement = sizeof(int32_t); // = 4
size_t bytesPerFloat = sizeof(float); // = 4
size_t structSize = sizeof(tensor_t);
// memcpy – copy n bytes from src to dst (regions must NOT overlap)
int32_t value;
memcpy(&value, bytes, sizeof(int32_t)); // safe alias-free copy
// memmove – like memcpy but safe when regions OVERLAP
memmove(outputData, inputData, numberOfBytes);
// memset – fill n bytes with a constant value
memset(tensor->data, 0, numberOfBytes); // zero out a buffer
```

`sizeof` returns the number of bytes a type or variable occupies. It is evaluated at compile time (no runtime cost). `memcpy` is the correct way to copy between a `uint8_t*` and a typed value (like `int32_t`) without violating strict-aliasing rules. Direct casting `*(int32_t*)bytes` would be undefined behaviour in C. `memmove` is the safe alternative when the source and destination overlap (e.g. in `convertTensorsWithSameType` where input and output may be the same buffer). `memset` is used to zero-initialise buffers before filling them.

2.8 static Functions — File-Private Functions

```
// In TensorConversion.c – only visible within this .c file:
static void zeroTensorData(tensor_t *tensor) {
    memset(tensor->data, 0, calcBytesPerTensor(tensor));
}
static void convertTensorsWithSameType(tensor_t *in, tensor_t *out) {
    /* ... */
}
```

When a function is declared 'static' inside a .c file it is file-private — it cannot be called from any other .c file. This is the C equivalent of 'private' in object-oriented languages. The convention in this library is: functions declared in the .h file are public (any file can call them); functions declared only inside the .c file with 'static' are private helpers used only within that one file. This reduces the public API surface and prevents accidental misuse.

2.9 extern — Declaring a Variable Defined in Another File

```
// In TensorConversion.h (declaration – no storage allocated here):
extern conversionFunction_t conversionMatrix[6][6];
// In TensorConversion.c (definition – storage allocated here):
conversionFunction_t conversionMatrix[6][6] = {
    [INT32][FLOAT32] = convertInt32TensorToFloatTensor,
    [FLOAT32][SYM_INT32] = convertFloatTensorToSymInt32Tensor,
    /* ... */
};
```

extern tells the compiler: 'this variable EXISTS, but its memory is allocated in a different .c file'. Without extern, every .c file that includes the header would try to create its own copy of the array, causing linker errors. With extern, only TensorConversion.c allocates the array, and every other file that includes TensorConversion.h can use it through the extern declaration.

2.10 Type Casts — (float)x, (int32_t)x

```
int32_t mantissa = 127;
float scale = 0.05f;
// Without cast: integer arithmetic (result is int, fractional part lost)
float wrong = mantissa / 2; // → 63 (integer division!)
// With cast: promotes mantissa to float before dividing
float real = (float)mantissa * scale; // → 6.35
// Truncating float to int32:
int32_t quantized = (int32_t)roundf(real / scale); // → 127
// Used in requantSymInt32Tensor:
float realValue = (float)mantissa * inScale;
int32_t out = roundByMode(clamp(realValue / targetScale, qMin, qMax), mode);
```

A cast explicitly converts a value from one type to another. (float)mantissa converts the integer 127 to the floating-point value 127.0f so that the multiplication with scale happens in floating-point arithmetic. Without the cast, 'mantissa * scale' would be promoted to float automatically in some cases, but being explicit prevents subtle bugs. Casts in the other direction — (int32_t)someFloat — truncate toward zero, which is why roundByMode is always called first to get correct rounding.

2.11 Integer Types — size_t, uint8_t, int32_t, bool

```
#include <stddef.h> // size_t
#include <stdint.h> // uint8_t, int32_t, uint32_t, int16_t ...
#include <stdbool.h> // bool, true, false
size_t count = 100; // unsigned, pointer-wide: 32-bit on ARM, 64-bit on x64
uint8_t byte = 0xFF; // unsigned 8-bit: 0 to 255
int32_t val = -32000; // signed 32-bit: -2147483648 to 2147483647
bool flag = true; // 0 or 1
// Why uint8_t for tensor data?
// A tensor stores bytes. uint8_t = exactly 1 byte, unsigned.
// We reassemble bytes into int32_t or float using memcpy (DTypes.c).
```

The standard integer types from stdint.h guarantee exact bit widths. int is ambiguous — it could be 16 or 32 bits depending on the platform. int32_t is always exactly 32 bits signed. This matters on embedded targets where int might be 16-bit. size_t is the correct type for array indices and byte counts because it is guaranteed to be wide enough to hold any valid array index on the target platform. bool from stdbool.h is a convenience — in C, any non-zero integer is true, but using bool makes the intent clear.

2.12 Guard Checks — if (!condition) { PRINT_ERROR; exit(1); }


```

bool doDimensionsMatch(tensor_t *a, tensor_t *b) {
    if (aNumberOfDims != bNumberOfDims) {
        PRINT_ERROR("Rank mismatch: %lu vs %lu", aNumberOfDims, bNumberOfDims);
        exit(1); // terminate the program immediately
    }
    /* ... */
}

// Another pattern: guard at the top of a function
void convertTensor(tensor_t *in, tensor_t *out) {
    if (in == NULL || out == NULL) {
        PRINT_ERROR("NULL pointer passed");
        exit(1);
    }
    /* safe to proceed */
}

```

Guard checks validate preconditions at the start of a function. On a microcontroller there is no exception system and no debugger. If an invalid argument slips through silently, the program produces wrong results with no indication of what went wrong. `exit(1)` immediately terminates the program with a non-zero status code, signalling failure. Combined with `PRINT_ERROR` which prints a message over UART, this gives the developer a clear error location to investigate. The pattern `!condition` means 'if the condition is NOT met, something is wrong'.

2.13 do { ... } while(false) — The Macro Safety Pattern

```

#define PRINT_ERROR(str, ...) \
do { \
    if (DLEVEL >= 1) { \
        printf("[%s] ", SOURCE_FILE); \
        printf(str, ##__VA_ARGS__); \
        printf("\n"); \
    } \
} while (false)

// WHY? Consider a macro without do-while:
// #define BAD_MACRO(x) if(x) doA(); doB(); // doB ALWAYS runs!
// With do-while, the macro is always a single statement:
if (error) PRINT_ERROR("oops"); // safe – no dangling else

```

The `do { ... } while(false)` wrapper makes a multi-statement macro behave like a single statement. Without it, writing `'if (x) MACRO(y); else ...'` could produce unexpected results because the macro expands to multiple statements and the `'else'` would attach to the wrong `'if'`. The backslash at the end of each line is a line-continuation character — it tells the preprocessor that the macro definition continues on the next line. `##__VA_ARGS__` handles zero or more additional arguments to `printf`-style macros.

2.14 #define Macros — Compile-Time Substitution

```
// Object-like macro (no parameters):
#define SOURCE_FILE "TENSOR"
#define DLEVEL 3
// Function-like macro (with parameters):
#define PRINT_DEBUG(str, ...) do { ... } while(false)
// Conditional compilation:
#ifdef DEBUG_MODE_DEBUG
#define DLEVEL 3
#elif defined(DEBUG_MODE_INFO)
#define DLEVEL 2
#else
#define DLEVEL 0
#endif
```

A `#define` macro is expanded by the C preprocessor BEFORE compilation. The preprocessor performs textual substitution: every occurrence of the macro name in the source code is replaced with its definition. `#ifdef` / `#elif` / `#else` / `#endif` forms conditional compilation blocks — the compiler only sees the code inside the branch whose condition is true. In `Common.h`, whether `DEBUG_MODE_DEBUG` is defined at compile time determines the debug level, and all the `PRINT_*` macros expand to either `printf` calls or to nothing (empty) depending on `DLEVEL`.

2.15 `_Static_assert` — Compile-Time Assertion

```
// At the bottom of TensorConversion.c:
_Static_assert(BOOL + 1 == 6,
"conversionMatrix is 6x6; extend it when adding a new qtype_t entry");
// Equivalent in C11:
static_assert(BOOL + 1 == 6, "extend conversionMatrix");
```

`_Static_assert` checks a condition at COMPILE TIME, not at runtime. If the condition is false, the compiler stops and prints the message. This is a zero-cost safety net: it catches bugs before the program ever runs. In `TensorConversion.c` it checks that the last value of the `qtype_t` enum (`BOOL`) plus 1 equals 6 — the number of rows/columns in `conversionMatrix`. If a developer adds a new `qtype_t` value without extending the matrix, this assertion fails immediately and reminds them to update the table.

2.16 `union` — One Memory Location, Multiple Types

```
typedef union layerConfig {
    linearConfig_t *linear;
    reluConfig_t *relu;
    softmaxConfig_t *softmax;
    conv1dConfig_t *conv1d;
    /* ... */
} layerConfig_t;
// Only ONE field is valid at any time:
layerConfig_t cfg;
cfg.linear = myLinearConfig; // now cfg.relu is garbage – never read it
```

A union is like a struct, but all fields share the SAME memory location. The size of the union is the size of its largest field. Only one field should be read at a time — whichever one was most recently written. `Layer.h` uses a union because a `layer_t` can be any one of many layer types (`Linear`, `Relu`, `Conv1d`, etc.) and each type has a different config struct. The `layer_t.type` field (an enum) tells you WHICH union member is currently valid.

2.17 Declaration vs Definition

```
// DECLARATION – in Tensor.h: tells the compiler the function EXISTS
size_t calcNumberOfElementsByTensor(tensor_t *tensor);
// DEFINITION – in Tensor.c: the actual code
size_t calcNumberOfElementsByTensor(tensor_t *tensor) {
    return calcNumberOfElementsByShape(tensor->shape);
}
// Rules:
// 1. Name must be identical in declaration and definition
// 2. Parameter types must match (names can differ)
// 3. Return type must match
// 4. One definition only; declarations can be repeated
```

A declaration tells the compiler: 'this function exists and has this signature'. A definition provides the body (the actual code). Declarations go in .h files so any .c file can call the function by including the header. Definitions go in the corresponding .c file. The linker resolves the connection between a call to the function and its definition.

Chapter 3 — Common.h

PURPOSE

Common.h is the debug-printing infrastructure for the entire library. It defines three print macros (PRINT_DEBUG, PRINT_INFO, PRINT_ERROR) and PRINT_BYTE_ARRAY. Every .c file includes Common.h and gets coloured, level-controlled console output that automatically shows the source file name and calling function name.

WHY THIS FILE EXISTS

On a microcontroller you cannot attach a debugger and step through code. The only feedback is what you print over UART. A single uniform macro that includes the file name, function name, and message — and that can be silenced completely by omitting -DDEBUG_MODE_* — is essential for embedded debugging. Without Common.h every .c file would contain its own printf boilerplate, making it impossible to control log verbosity centrally.

DEPENDS ON

Standard library only: stdio.h, string.h, stdbool.h. No other project file.

USED BY

Every .c file in the project. The most widely included header in the codebase.

3.1 How Debug Levels Work

```
// Compiler command to enable full debug output:
// gcc -DDEBUG_MODE_DEBUG -DSOURCE_FILE='"MYFILE"' ...
// Resulting DLEVEL value:
// -DDEBUG_MODE_DEBUG → DLEVEL = 3 (all messages)
// -DDEBUG_MODE_INFO → DLEVEL = 2 (info + errors)
// -DDEBUG_MODE_ERROR → DLEVEL = 1 (errors only)
// (nothing defined) → DLEVEL = 0 (silence – no code generated)
// The do-while(false) means each macro is zero bytes when DLEVEL < threshold:
PRINT_DEBUG("value = %d", x); // if DLEVEL < 3, this compiles to nothing
```

3.2 Functions / Macros in Common.h

SYNTAX

```
#define PRINT_DEBUG(str, ...)
```

PURPOSE

Prints a YELLOW coloured debug message. Only active when DLEVEL >= 3 (-DDEBUG_MODE_DEBUG passed at compile time). Format: [SourceFile: FunctionName] your message

PARAMETERS

str: a printf-style format string (e.g. "value = %d"). ...: zero or more extra arguments matching the format specifiers.

NOTE

SOURCE_FILE must be #define'd in each .c file BEFORE including Common.h. Example: #define SOURCE_FILE "TENSOR". __FUNCTION__ is a C99 predefined identifier that expands to the name of the current function as a string literal at compile time.

EXAMPLE

```
PRINT_DEBUG("numberOfElements = %lu", n); // Output: [ARITHMETIC: calcTensorIndexByIndices]
numberOfElements = 12
```

SYNTAX

```
#define PRINT_INFO(str, ...)
```

PURPOSE

Prints a plain (no colour) informational message. Active when DLEVEL >= 2. Same format as PRINT_DEBUG. Use for non-error status updates.

PARAMETERS

str: format string.: variadic printf arguments.

EXAMPLE

```
PRINT_INFO("Loading weights from file: %s", filename);
```

SYNTAX

```
#define PRINT_ERROR(str, ...)
```

PURPOSE

Prints a RED coloured error message. Active when DLEVEL >= 1. This is the most important macro in the file — it is always paired with exit(1) in the codebase to report fatal errors before terminating.

PARAMETERS

str: format string.: variadic printf arguments.

NOTE

PRINT_ERROR does NOT call exit(1) itself. The calling function must call exit(1) after PRINT_ERROR. This separation lets the function do any cleanup before terminating, and makes the macro reusable for non-fatal warnings too.

EXAMPLE

```
PRINT_ERROR("Rank mismatch: %lu vs %lu", a, b); exit(1);
```

SYNTAX

```
#define PRINT_BYTE_ARRAY(prefix, byteArray, numberOfBytes)
```

PURPOSE

Dumps a raw byte array to stdout in hexadecimal format. Example output: [TENSOR: printTensor] raw data: 0x1A 0x00 0xFF 0x3C Used to inspect the raw memory content of a tensor's data field for debugging.

PARAMETERS

prefix: a label string printed before the hex values (e.g. "raw data: "). byteArray: pointer to the uint8_t array to dump. numberOfBytes: how many bytes to print.

NOTE

Always prints regardless of DLEVEL — it has no level check. Use it only during active debugging, then remove the call.

EXAMPLE

```
PRINT_BYTE_ARRAY("weights: ", tensor->data, calcBytesPerTensor(tensor));
```

Chapter 4 — DTypes.h / DTypes.c

PURPOSE

DTypes bridges the gap between the raw byte storage inside a tensor and the typed C values (`int32_t`, `float`) that arithmetic code needs. A tensor stores all its data as a `uint8_t` byte array. Before adding two elements you must reassemble 4 bytes back into an `int32_t` or `float`. DTypes provides that conversion and nothing else.

WHY THIS FILE EXISTS

C strict aliasing rules forbid casting a `uint8_t*` directly to `int32_t*` — the behaviour is undefined (the compiler may generate wrong code). The correct solution is `memcpy`, which DTypes wraps in named functions so the rest of the code stays readable. Having these wrappers in one file means you can change the byte order or packing format in exactly one place if hardware requires it.

DEPENDS ON

`stddef.h` (for `size_t`), `stdint.h` (for `uint8_t`, `int32_t`). No project dependencies.

USED BY

`Tensor.c`, `TensorConversion.c`, `Arithmetic.c`, `Serialize.c`, `Deserialize.c`, `NPYLoader.c` — every file that reads or writes typed values to/from byte buffers.

4.1 How Byte Reassembly Works

```
// A tensor stores the integer value 1000 as 4 bytes:
// bytes[0] = 0xE8, bytes[1] = 0x03, bytes[2] = 0x00, bytes[3] = 0x00
// (little-endian: least significant byte first)
// readBytesAsInt32 uses memcpy to reassemble them:
int32_t readBytesAsInt32(uint8_t *bytes) {
    int32_t value;
    memcpy(&value, bytes, sizeof(int32_t)); // safe, defined behaviour
    return value;
}

// Called in Arithmetic.c for every element:
int32_t aValue = readBytesAsInt32(&aTensor->data[aByteIndex]);
int32_t bValue = readBytesAsInt32(&bTensor->data[bByteIndex]);
int32_t result = arithmeticFunc(aValue, bValue);
writeInt32ToByteArray(result, &outputTensor->data[outputByteIndex]);
```

4.2 Functions in DTypes.h / DTypes.c

SYNTAX

```
int32_t readBytesAsInt32(uint8_t *bytes)
```

PURPOSE

Reads exactly 4 bytes from the memory address 'bytes' and reassembles them into a signed 32-bit integer using memcpy. This is the single most-called function in the library — every arithmetic operation on INT32 or SYM_INT32 tensors calls it once per element.

PARAMETERS

bytes: pointer to the first of the 4 bytes to read. In practice always called as readBytesAsInt32(&tensor->data[byteIndex]) where byteIndex = elementIndex * sizeof(int32_t) = elementIndex * 4.

RETURNS

int32_t: the 32-bit value stored at that memory location.

NOTE

Uses memcpy internally to comply with C strict aliasing rules. No bounds checking — the caller must ensure bytes points to 4 valid bytes.

EXAMPLE

```
// Element 3 of an int32 tensor: size_t byteIndex = 3 * sizeof(int32_t); // = 12 int32_t val =  
readBytesAsInt32(&tensor->data[byteIndex]);
```

SYNTAX

```
int32_t readNumberOfBytesAsInt32(uint8_t *data, size_t numberOfBytes)
```

PURPOSE

Reads a specified number of bytes (1, 2, 3, or 4) and sign-extends the result to int32_t. Used for tensors that pack data at less than 32 bits per element (e.g. 8-bit symmetric quantization stored compactly).

PARAMETERS

data: pointer to byte array. numberOfBytes: how many bytes to read (1–4). Reading more than 4 is undefined.

RETURNS

int32_t: sign-extended integer value.

NOTE

Sign extension means the most significant bit of the source value is propagated into all upper bits of the int32_t result, preserving the sign of negative numbers.

EXAMPLE

```
// Read a 1-byte signed value at position 5: int32_t v = readNumberOfBytesAsInt32(&data[5], 1);
```

SYNTAX

```
void readBytesAsInt32Array(size_t numberOfValues, uint8_t *bytes, int32_t *outputArray)
```

PURPOSE

Converts an entire contiguous byte buffer into an array of `int32_t` values by calling `readBytesAsInt32` in a loop — one call per 4-byte group. Equivalent to calling `readBytesAsInt32` `numberOfValues` times manually.

PARAMETERS

`numberOfValues`: number of `int32_t` values to extract. `bytes`: source byte buffer (must be at least `numberOfValues × 4` bytes long). `outputArray`: pre-allocated `int32_t` array that receives the results (must hold at least `numberOfValues` elements).

RETURNS

void — writes into `outputArray`.

NOTE

The caller must allocate `outputArray` before calling this function.

EXAMPLE

```
int32_t out[10]; readbytesAsInt32Array(10, tensor->data, out); // out[0] = first element, out[9] = tenth
```

SYNTAX

```
float readBytesAsFloat(uint8_t *bytes)
```

PURPOSE

Reads 4 bytes and reassembles them as an IEEE 754 single-precision float. The float companion of `readBytesAsInt32` — used for `FLOAT32` tensors.

PARAMETERS

`bytes`: pointer to 4 bytes in the tensor's data buffer.

RETURNS

float: the floating-point value stored at that address.

NOTE

IEEE 754 single precision: 1 sign bit, 8 exponent bits, 23 mantissa bits. The bit pattern is the same 4 bytes as an `int32_t` — only the interpretation differs.

EXAMPLE

```
float val = readBytesAsFloat(&tensor->data[elementIndex * 4]);
```

SYNTAX

```
void readBytesAsFloatArray(size_t numberOfValues, uint8_t *bytes, float *outputArray)
```

PURPOSE

Bulk-converts a byte buffer into a float array. Calls `readBytesAsFloat` in a loop `numberOfValues` times.

PARAMETERS

`numberOfValues`: element count. `bytes`: source buffer. `outputArray`: destination float array.

RETURNS

void.

SYNTAX

```
void writeInt32ToByteArray(int32_t value, uint8_t *bytes)
```

PURPOSE

Writes a 32-bit signed integer into 4 consecutive bytes. This is the inverse of `readBytesAsInt32`. Called after every arithmetic operation to store the result back into the output tensor's data buffer.

PARAMETERS

value: the `int32_t` result to store. bytes: pointer to 4 bytes in the output tensor's data field where the value is written.

RETURNS

void — modifies the 4 bytes in place.

NOTE

Always call as `writeInt32ToByteArray(result, &tensor->data[elementIndex * 4])`. Uses `memcpy` internally for alias safety.

EXAMPLE

```
int32_t result = addInt32s(a, b); writeInt32ToByteArray(result, &outputTensor->data[j * sizeof(int32_t)]);
```

SYNTAX

```
void writeInt32ArrayToByteArray(size_t numberOfValues, int32_t *valueArray, uint8_t *bytes)
```

PURPOSE

Bulk-writes an array of `int32_t` values into a byte buffer. Calls `writeInt32ToByteArray` in a loop.

PARAMETERS

numberOfValues: count. valueArray: source `int32_t` array. bytes: destination byte buffer.

RETURNS

void.

SYNTAX

```
void writeFloatToByteArray(float value, uint8_t *bytes)
```

PURPOSE

Writes a float into 4 consecutive bytes. The inverse of `readBytesAsFloat`. Used when writing float results to a `FLOAT32` tensor.

PARAMETERS

value: the float to store. bytes: destination in the tensor data buffer.

RETURNS

void.

NOTE

Uses `memcpy` for alias safety.

SYNTAX

```
void writeFloatArrayToByteArray(size_t numberOfValues, float *valueArray, uint8_t *bytes)
```

PURPOSE

Bulk float-to-bytes write. Calls `writeFloatToByteArray` in a loop.

PARAMETERS

numberOfValues, valueArray, bytes.

RETURNS

void.

Chapter 5 — Quantization.h / Quantization.c

PURPOSE

Quantization.h defines the type system describing HOW a tensor stores its numbers. It defines: (a) the `qtype_t` enum naming six numeric schemes, (b) three config structs holding scheme-specific parameters (scale, zeroPoint, bit-width), and (c) init functions that populate those structs and wire them into a `quantization_t` wrapper. Every `tensor_t` carries a `quantization_t` pointer so any function can immediately determine how to interpret the tensor's bytes.

WHY THIS FILE EXISTS

On a microcontroller RAM is scarce. A 32-bit float takes 4 bytes; an 8-bit quantized integer takes 1 byte — 4× smaller. Quantization trades a small amount of precision for huge memory and compute savings. Without a uniform type system, every function would need special-case code for each numeric format. The `quantization_t` struct bundles the format (`qtype`) with its parameters (`qConfig`) so functions can dispatch on type without knowing the implementation details.

DEPENDS ON

Rounding.h (for `roundingMode_t`: `HALF_AWAY` or `SR_HALF_AWAY`).

USED BY

Tensor.h (embedded as a field in `tensor_t`), TensorConversion.h/c, all layer files, the user API `QuantizationApi`.

5.1 The Six Quantization Types

```
typedef enum qtype {
    INT32, // 0: raw signed 32-bit int, no scale (intermediate accumulation)
    FLOAT32, // 1: IEEE 754 single-precision float (training, reference)
    SYM_INT32, // 2: symmetric – real = mantissa × scale, centred at 0
    SYM, // 3: symmetric with configurable bit-width (e.g. 8-bit)
    ASYM, // 4: asymmetric – real = (mantissa - zeroPoint) × scale
    BOOL // 5: packed 1-bit boolean (used in sparsity masks, Dropout)
} qtype_t;

// For SYM_INT32: mantissa is stored as int32_t; real = mantissa × scale
// Example: scale=0.001, mantissa=1000 → real = 1.0
// qMax = 2^(qMaxBits-1) - 1, qMin = -2^(qMaxBits-1)
// For qMaxBits=16: qMax=32767, qMin=-32768
```

5.2 Config Structs

```
typedef struct symInt32QConfig {
    float scale; // maps integer → real: real = mantissa × scale
    roundingMode_t roundingMode; // HALF_AWAY or SR_HALF_AWAY
    uint8_t qMaxBits; // bit depth for clamping: qMax = 2^(qMaxBits-1)-1
} symInt32QConfig_t;

typedef struct asymQConfig {
    float scale; // real = (mantissa - zeroPoint) × scale
    int16_t zeroPoint; // offset to shift the zero point
    uint8_t qBits; // total bits per element
    roundingMode_t roundingMode;
} asymQConfig_t;

typedef struct quantization {
    qtype_t type; // which scheme is in use
    void *qConfig; // pointer to symInt32QConfig_t / asymQConfig_t / NULL
} quantization_t;
```

The `quantization_t` wrapper uses a `void*` for `qConfig` so it can point to any of the three config structs (or `NULL` for `INT32/FLOAT32/BOOL` which have no parameters). The code casts the `void*` to the correct type after checking the `qtype`.

5.3 Functions in `Quantization.h` / `Quantization.c`

SYNTAX

```
void initSymInt32QConfig(roundingMode_t roundingMode, symInt32QConfig_t *symInt32QConfig)
```

PURPOSE

Fills a `symInt32QConfig_t` with default values: `scale = 1.0f`, `qMaxBits = 16`. The default scale of 1.0 means the mantissa equals the real value initially. The scale is updated later by `requantSymInt32Tensor` when the actual data range is known.

PARAMETERS

`roundingMode`: `HALF_AWAY` (standard round-half-away-from-zero) or `SR_HALF_AWAY` (stochastic rounding — adds uniform noise before rounding). `symInt32QConfig`: pointer to an already-allocated `symInt32QConfig_t` to fill.

RETURNS

`void` — modifies `*symInt32QConfig` in place.

NOTE

With `qMaxBits=16`: `qMax=32767`, `qMin=-32768`. The 32-bit mantissa can hold values well beyond this range, but clamping occurs during quantization.

EXAMPLE

```
symInt32QConfig_t cfg; initSymInt32QConfig(HALF_AWAY, &cfg); // cfg.scale=1.0, cfg.qMaxBits=16
```

SYNTAX

```
void initSymInt32QConfigWithQMaxBits(roundingMode_t roundingMode, symInt32QConfig_t *symInt32QConfig, uint8_t qMaxBits)
```

PURPOSE

Same as `initSymInt32QConfig` but sets a custom bit depth. Use this when you want a narrower range (e.g. 8-bit: `qMax=127`, `qMin=-128`).

PARAMETERS

`roundingMode`, `symInt32QConfig`: same as above. `qMaxBits`: bit depth for clamping range. `qMax = 2^(qMaxBits-1) - 1`.

RETURNS

`void`.

NOTE

`qMaxBits=8`: `qMax=127`, `qMin=-128`. `qMaxBits=16`: `qMax=32767`, `qMin=-32768`. `qMaxBits=32`: full `int32` range.

EXAMPLE

```
initSymInt32QConfigWithQMaxBits(HALF_AWAY, &cfg, 8); // qMax=127, qMin=-128
```

SYNTAX

```
void initSymQConfig(uint8_t qBits, roundingMode_t roundingMode, symQConfig_t *symQConfig)
```

PURPOSE

Initialises a symQConfig_t for symmetric quantization at a given bit-width. Used for SYM type tensors (e.g. 8-bit weights).

PARAMETERS

qBits: total bits per element (e.g. 8). roundingMode: rounding mode. symQConfig: target struct.

RETURNS

void.

SYNTAX

```
void initAsymQConfig(uint8_t qBits, roundingMode_t roundingMode, asymQConfig_t *asymQConfig)
```

PURPOSE

Initialises an asymQConfig_t with default scale=1.0 and zeroPoint=0. Asymmetric quantization is useful for activations that are non-negative (e.g. after ReLU) where a non-zero offset allows better use of the integer range.

PARAMETERS

qBits: bits per element. roundingMode. asymQConfig: target struct.

RETURNS

void.

NOTE

After init, the caller must set scale and zeroPoint based on the actual data range.

EXAMPLE

```
asymQConfig_t cfg; initAsymQConfig(8, HALF_AWAY, &cfg); // cfg.scale=1.0, cfg.zeroPoint=0, cfg.qBits=8
```

SYNTAX

```
void initInt32Quantization(quantization_t *quantization)
```

PURPOSE

Sets quantization->type = INT32 and quantization->qConfig = NULL. Use this for tensors that hold raw 32-bit integers with no scale factor (e.g. accumulation buffers in matrix multiply before requantization).

PARAMETERS

quantization: pointer to the quantization_t to initialise.

RETURNS

void.

NOTE

INT32 means the bytes are plain int32_t values. No scale, no zero-point. readBytesAsInt32 / writeInt32ToByteArray are used directly.

EXAMPLE

```
quantization_t q; initInt32Quantization(&q); // q.type=INT32, q.qConfig=NULL
```

SYNTAX

```
void initFloat32Quantization(quantization_t *quantization)
```

PURPOSE

Sets type = FLOAT32, qConfig = NULL. Use this for standard floating-point tensors (training mode, reference results).

PARAMETERS

quantization.

RETURNS

void.

SYNTAX

```
void initBoolQuantization(quantization_t *quantization)
```

PURPOSE

Sets type = BOOL, qConfig = NULL. BOOL tensors store packed 1-bit values. Used for sparsity masks and Dropout masks.

PARAMETERS

quantization.

RETURNS

void.

SYNTAX

```
void initSymInt32Quantization(symInt32QConfig_t *symInt32QConfig, quantization_t *quantization)
```

PURPOSE

Sets type = SYM_INT32 and qConfig = symInt32QConfig (cast to void*). This is the standard setup for quantized weight and activation tensors in the ODT library. After this call the quantization_t is fully wired: the type field drives dispatch and the qConfig pointer leads to the scale and bit-width.

PARAMETERS

symInt32QConfig: pre-filled config struct (call initSymInt32QConfig first). quantization: target wrapper struct.

RETURNS

void.

NOTE

Always initialise the qConfig struct BEFORE calling this function.

EXAMPLE

```
symInt32QConfig_t cfg; initSymInt32QConfig(HALF_AWAY, &cfg); quantization_t q; initSymInt32Quantization(&cfg, &q); // q.type=SYM_INT32, q.qConfig=&cfg
```

SYNTAX

```
void initSymQuantization(symQConfig_t *symQConfig, quantization_t *quantization)
```

PURPOSE

Sets type = SYM, qConfig = symQConfig.

PARAMETERS

symQConfig: pre-filled struct. quantization: wrapper.

RETURNS

void.

SYNTAX

```
void initAsymQuantization(asymQConfig_t *asymQConfig, quantization_t *quantization)
```

PURPOSE

Sets type = ASYM, qConfig = asymQConfig.

PARAMETERS

asymQConfig: pre-filled struct. quantization: wrapper.

RETURNS

void.

Chapter 6 — Tensor.h / Tensor.c

PURPOSE

Tensor.h defines `tensor_t` — the fundamental data structure of the entire library. A tensor is an N-dimensional array of numbers. `tensor_t` bundles the raw byte buffer (data), shape metadata, quantization descriptor, and optional sparsity info. `Tensor.c` implements all utility functions: size calculations, index arithmetic, shape setup, bool bit-packing, and transpose.

WHY THIS FILE EXISTS

Every layer, every loss function, every arithmetic operation passes tensor_t pointers. Keeping the data as uint8_t* and attaching type info via quantization_t makes the same struct hold floats, ints, or quantized integers without changing any layer code. The layer inspects quantization->type to know how to interpret the bytes.

DEPENDS ON

Common.h, DTypes.h, Quantization.h. Standard: stdbool.h, stddef.h, stdint.h.

USED BY

Every single other file in the project. This is the most important header.

6.1 Core Structs

```
// ■■ shape_t ████████████████████████████████████████████████████████████████████████  
typedef struct shape {  
    size_t numberOfDimensions; // rank: 1=vector, 2=matrix, 3=3-D tensor  
    size_t *dimensions; // [d0, d1, ...] PHYSICAL storage sizes  
    size_t *orderOfDimensions; // logical→physical axis mapping (for transpose)  
} shape_t;  
  
// ■■ tensor_t ████████████████████████████████████████████████████████████████████████  
typedef struct tensor {  
    uint8_t *data; // raw byte storage: all elements packed  
    shape_t *shape; // dimension metadata  
    quantization_t *quantization; // numeric scheme + parameters  
    sparsity_t *sparsity; // NULL if dense  
} tensor_t;  
  
// ■■ parameter_t ████████████████████████████████████████████████████████████████████████  
typedef struct parameter {  
    tensor_t *param; // the weight tensor (stores learned values)  
    tensor_t *grad; // the gradient tensor (same shape as param)  
} parameter_t;
```

The `orderOfDimensions` array makes zero-cost transpose possible. For a non-transposed 3x4 matrix: `dimensions=[3,4]`, `orderOfDimensions=[0,1]`. After `transposeTensor(0,1)`: `dimensions=[3,4]` unchanged, `orderOfDimensions=[1,0]`. When reading element `[row,col]` we look up the physical position using `orderOfDimensions`, not the logical indices directly. No data is moved — only the metadata changes.

6.2 Functions in Tensor.h / Tensor.c

SYNTAX

```
uint32_t getBitmask(uint32_t startbit, uint32_t endbit)
```

PURPOSE

Builds a 32-bit integer with 1s in bit positions startbit through endbit (inclusive) and 0s everywhere else. Used by writeByte and readByte for BOOL tensor bit manipulation.

PARAMETERS

startbit: lowest bit position to set (0 = LSB = least significant bit). endbit: highest bit position to set (31 = MSB).

RETURNS

uint32_t: the bitmask.

EXAMPLE

getBitmask(2, 5) → 0b00111100 = 0x3C Bits 2,3,4,5 are 1; all others are 0.

SYNTAX

```
uint8_t writeByte(uint8_t existingData, uint8_t data, uint8_t startbit, uint8_t endbit)
```

PURPOSE

Writes bits from 'data' into positions startbit..endbit of 'existingData', leaving all other bits in existingData unchanged. Used to pack boolean values into bytes for BOOL tensors (8 booleans per byte).

PARAMETERS

existingData: the byte being modified. data: the value to write into the bit range (right-aligned). startbit / endbit: the bit range within the byte to overwrite.

RETURNS

uint8_t: the modified byte with the new bits written in.

NOTE

Step 1: clear the target bits in existingData using ~getBitmask. Step 2: shift data left by startbit to align it. Step 3: OR them together.

EXAMPLE

// Pack boolean 'true' into bit 3 of byte 0xFF: writeByte(0xFF, 1, 3, 3) // existingData=11111111, set bit 3 to 1 → 11111111 (unchanged) writeByte(0xFF, 0, 3, 3) → 11110111 = 0xF7 // cleared bit 3

SYNTAX

```
uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit)
```

PURPOSE

Extracts bits startbit..endbit from 'data' and returns them right-aligned as a uint8_t.

PARAMETERS

data: the byte to extract from. startbit / endbit: the bit range to extract.

RETURNS

uint8_t: the extracted bit field (shifted right so bit startbit is at position 0).

EXAMPLE

readByte(0b10110100, 2, 4) → extracts bits 2,3,4 → 0b101 = 5

SYNTAX

```
void byteConversion(uint8_t *dataIn, size_t dataInBits, uint8_t *dataOut, size_t dataOutBits, size_t numValues)
```

PURPOSE

Converts between different per-element bit widths. For example: expanding a 1-bit-per-element BOOL array to 8-bit storage, or packing 8-bit values into a 1-bit mask.

PARAMETERS

dataIn: source byte array. dataInBits: bits per element in source (e.g. 1 for BOOL, 8 for SYM 8-bit). dataOut: destination byte array. dataOutBits: bits per element in destination. numValues: number of logical values to convert.

RETURNS

void.

SYNTAX

```
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex)
```

PURPOSE

Reads one boolean value from a BOOL tensor at the given flat element index. Since BOOL tensors pack 8 values per byte, this computes the correct byte and bit position from flatIndex and extracts the bit using readByte.

PARAMETERS

tensor: pointer to a BOOL tensor (quantization->type must be BOOL). flatIndex: zero-based element index (0 = first element, 1 = second, etc.).

RETURNS

bool: true if the bit at that position is 1, false if 0.

NOTE

byte position = flatIndex / 8; bit position = flatIndex % 8.

EXAMPLE

```
// Read element 11 of a BOOL tensor: bool b = tensorBoolGet(tensor, 11); // byte = 11/8 = 1, bit = 11%8 = 3 → reads bit 3 of tensor->data[1]
```

SYNTAX

```
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value)
```

PURPOSE

Writes one boolean value into a BOOL tensor at flatIndex.

PARAMETERS

tensor: BOOL tensor to modify. flatIndex: element position to write. value: true (1) or false (0).

RETURNS

void.

NOTE

Uses writeByte internally to modify only the target bit without touching adjacent bits.

EXAMPLE

```
tensorBoolSet(mask, 5, true); // set bit 5 to 1 tensorBoolSet(mask, 5, false); // clear bit 5
```

SYNTAX

```
tensor_t *getParamFromParameter(parameter_t *parameter)
```

PURPOSE

Returns the weight tensor pointer from a parameter_t. Convenience accessor used in layer backward passes to update weights.

PARAMETERS

parameter: pointer to a parameter_t.

RETURNS

tensor_t *: the weight tensor (parameter->param).

SYNTAX

```
tensor_t *getGradFromParameter(parameter_t *parameter)
```

PURPOSE

Returns the gradient tensor pointer from a parameter_t. Used in the backward pass to accumulate gradients and in the optimizer step.

PARAMETERS

parameter: pointer to a parameter_t.

RETURNS

tensor_t *: the gradient tensor (parameter->grad).

SYNTAX

```
size_t calcBytesPerElement(quantization_t *quantization)
```

PURPOSE

Returns how many bytes one element occupies based on the quantization type. INT32 → 4, FLOAT32 → 4, SYM_INT32 → 4, SYM → qBits/8 (rounded up), ASYM → qBits/8 (rounded up), BOOL → 0 (sub-byte, use calcBitsPerElement).

PARAMETERS

quantization: the quantization descriptor of a tensor.

RETURNS

size_t: bytes per element. Returns 0 for BOOL.

NOTE

The result × numberOfElements gives total data buffer size.

EXAMPLE

```
// For SYM_INT32 tensor: size_t bpe = calcBytesPerElement(tensor->quantization); // = 4
```

SYNTAX

```
size_t calcBitsPerElement(quantization_t *quantization)
```

PURPOSE

Returns bits per element. Needed for BOOL (1 bit) and sub-byte types.

PARAMETERS

quantization.

RETURNS

size_t: bits per element. INT32/FLOAT32/SYM_INT32 return 32. BOOL returns 1.

SYNTAX

```
size_t calcBytesPerTensor(tensor_t *tensor)
```

PURPOSE

Returns the total byte count of a tensor's data buffer: $= \text{numberOfElements} \times \text{bytesPerElement}$ (rounded up for sub-byte types).

PARAMETERS

tensor: any fully initialised tensor_t.

RETURNS

size_t: total bytes needed for the data field.

EXAMPLE

```
// 3x4 SYM_INT32 tensor: 12 elements x 4 bytes = 48 bytes size_t total = calcBytesPerTensor(t);
```

SYNTAX

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements)
```

PURPOSE

Lower-level version of calcBytesPerTensor: given a quantization scheme and element count, returns the byte count. Used before the tensor_t is fully built.

PARAMETERS

q: quantization descriptor. numberOfElements: how many values to accommodate.

RETURNS

size_t: byte count.

SYNTAX

```
size_t calcNumberOfElementsByShape(shape_t *shape)
```

PURPOSE

Multiplies all dimension sizes together to get total element count. For shape [3, 4, 5]: returns $3 \times 4 \times 5 = 60$.

PARAMETERS

shape: pointer to a shape_t.

RETURNS

size_t: total element count.

EXAMPLE

```
shape: numberOfDimensions=2, dimensions=[3,4] calcNumberOfElementsByShape(&shape) → 12
```

SYNTAX

```
size_t calcNumberOfElementsByTensor(tensor_t *tensor)
```

PURPOSE

Convenience wrapper: calls `calcNumberOfElementsByShape(tensor->shape)`.

PARAMETERS

tensor: any `tensor_t`.

RETURNS

size_t: total element count.

NOTE

This is the most commonly called function for getting element count in loops.

EXAMPLE

```
size_t n = calcNumberOfElementsByTensor(t); for (size_t i = 0; i < n; i++) { ... }
```

SYNTAX

```
void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index)
```

PURPOSE

Swaps two logical axes by swapping their entries in `orderOfDimensions`. No data is moved — only metadata changes. Any subsequent call to `calcElementIndexByIndices` automatically respects the new axis order. This is a zero-cost transpose: $O(1)$ regardless of tensor size.

PARAMETERS

tensor: the tensor to transpose (modified in place). dim0Index: logical index of the first axis to swap. dim1Index: logical index of the second axis to swap.

RETURNS

void — modifies `tensor->shape->orderOfDimensions`.

NOTE

After transpose, the `dimensions[]` array is unchanged. Only `orderOfDimensions` changes. A non-transposed tensor has `orderOfDimensions = [0, 1, 2, ...]`. After `transposeTensor(0, 1)` on a 2D tensor: `orderOfDimensions = [1, 0]`.

EXAMPLE

```
// Transpose a [3×4] matrix to [4×3] logical view: transposeTensor(matrix, 0, 1); // dimensions still [3,4];  
orderOfDimensions now [1,0] // Element [row=1, col=2] now reads from physical position [row=2, col=1]
```

SYNTAX

```
void setTensorValues(tensor_t *tensor, uint8_t *data, shape_t *shape, quantization_t *quantization, sparsity_t *sparsity)
```

PURPOSE

Fills all four fields of a tensor_t at once. This is the standard way to initialise a tensor after the caller has allocated all sub-structs.

PARAMETERS

tensor: the tensor_t to fill. data: pre-allocated byte buffer (size must be calcNumberOfBytesForData(quant, nElements)). shape: pre-filled shape_t. quantization: pre-filled quantization_t. sparsity: sparsity_t pointer, or NULL if the tensor is dense.

RETURNS

void.

NOTE

The order is always: (1) prepare the sub-structs, (2) call setTensorValues. Never set tensor->data directly — always use this function for clarity.

EXAMPLE

```
setTensorValues(&myTensor, buffer, &myShape, &myQ, NULL);
```

SYNTAX

```
void setTensorValuesForConversion(uint8_t *data, quantization_t *q, tensor_t *originalTensor, tensor_t *outputTensor)
```

PURPOSE

Sets up outputTensor with new data buffer and quantization, but copies shape and sparsity from originalTensor. Used in TensorConversion.c when producing a converted tensor that has the same logical shape as the input but different numeric type.

PARAMETERS

data: new byte buffer for the output. q: the output quantization (different type than input). originalTensor: source of shape and sparsity. outputTensor: tensor_t to configure.

RETURNS

void.

SYNTAX

```
void setOrderOfDimsForNewTensor(size_t numberOfDimensions, size_t *orderOfDimensions)
```

PURPOSE

Fills orderOfDimensions with the identity ordering [0, 1, 2, ..., N-1]. Call this when creating a new tensor that is not transposed.

PARAMETERS

numberOfDimensions: number of axes. orderOfDimensions: array of size numberOfDimensions to fill.

RETURNS

void.

NOTE

After this call, orderOfDimensions[i] == i for all i. This means the logical axis i maps to physical axis i — no transpose.

EXAMPLE

```
size_t order[3]; setOrderOfDimsForNewTensor(3, order); // order = [0, 1, 2]
```

SYNTAX

```
void setShape(shape_t *shape, size_t *dims, size_t numberOfDims, size_t *orderOfDims)
```

PURPOSE

Fills all three fields of a `shape_t` at once. Convenience wrapper.

PARAMETERS

`shape`: target `shape_t`. `dims`: array of dimension sizes. `numberOfDims`: rank. `orderOfDims`: ordering array (usually set with `setOrderOfDimsForNewTensor` first).

RETURNS

void.

EXAMPLE

```
size_t dims[2] = {3, 4}; size_t order[2]; setOrderOfDimsForNewTensor(2, order); setShape(&s, dims, 2, order); // s:  
3x4, identity ordering
```

SYNTAX

```
void printTensor(tensor_t *t)
```

PURPOSE

Prints tensor shape and raw byte data to stdout. For debugging — not for production use.

PARAMETERS

`t`: any `tensor_t`.

RETURNS

void.

SYNTAX

```
void copyTensor(tensor_t *dest, tensor_t *src)
```

PURPOSE

Shallow-copies all four pointer fields from `src` into `dest`. Both tensors then point to the same data, shape, and quantization buffers.

PARAMETERS

`dest`: destination. `src`: source.

RETURNS

void.

NOTE

SHALLOW copy only — does not duplicate the underlying data or shape arrays. Modifying `dest->data[i]` will also modify `src->data[i]`.

Chapter 7 — TensorConversion.h / TensorConversion.c

PURPOSE

TensorConversion converts a tensor from one numeric type to another. It answers: 'I have a FLOAT32 tensor; I need a SYM_INT32 tensor with the same logical values.' The conversionMatrix[6][6] dispatch table holds a function pointer for every [fromType][toType] pair. The single public entry convertTensor() looks up and calls the right converter.

WHY THIS FILE EXISTS

Neural-network training produces activations as floats but hardware stores weights as quantized integers. A dispatch table avoids switch-case chains and makes adding a new type O(1): fill one new row and column in the matrix.

DEPENDS ON

Common.h, DTypes.h, Tensor.h, Quantization.h.

USED BY

Layer files that requantize activations, QuantizationApi, training loop.

7.1 The Dispatch Table — conversionMatrix[6][6]

```
// Declaration in header – extern means defined in TensorConversion.c:
extern conversionFunction_t conversionMatrix[6][6];
// Definition in TensorConversion.c using designated initialisers:
conversionFunction_t conversionMatrix[6][6] = {
    [INT32 ][FLOAT32 ] = convertInt32TensorToFloatTensor,
    [FLOAT32][SYM_INT32] = convertFloatTensorToSymInt32Tensor,
    [SYM_INT32][FLOAT32] = convertSymInt32TensorToFloat32Tensor,
    [SYM_INT32][SYM_INT32] = requantSymInt32Tensor,
    /* ... all 30 valid pairs ... */
};
// Look up a converter directly:
conversionFunction_t fn = conversionMatrix[SYM_INT32][FLOAT32];
fn(inputTensor, outputTensor); // call it
```

Designated initialisers ([INDEX] = value) set specific elements of an array by index. All unspecified entries default to NULL (which maps to unsupportedConversionTypes() via convertTensor's guard).

7.2 Two-Pass Dynamic Requantization — Step by Step

requantSymInt32Tensor uses a two-pass algorithm. Here is a worked example:

```
// Tensor: [100, -200, 50], inScale=0.01, output qMaxBits=16
// qMax = 2^(16-1) - 1 = 32767, qMin = -32768
// PASS A – find absMax of real values:
// real[0] = 100 × 0.01 = 1.0 absMax = max(1.0, 2.0, 0.5) = 2.0
// real[1] = -200 × 0.01 = -2.0
// real[2] = 50 × 0.01 = 0.5
// Compute fresh scale:
// scale = absMax / qMax = 2.0 / 32767 ≈ 0.0000610
// PASS B – requantize each element:
// out[0] = round(1.0 / 0.0000610) = round(16385) = 16385
// out[1] = round(-2.0 / 0.0000610) = round(-32770) → clamp → -32768 (qMin)
// out[2] = round(0.5 / 0.0000610) = round(8193) = 8193
// Write fresh scale to output qConfig:
// outputTensor->quantization->qConfig->scale = 0.0000610
```

The two passes ensure the fresh scale is computed from the FULL range of the input before any element is written. If it were done in one pass, elements written early would use a scale computed before the maximum was known.

7.3 Functions in TensorConversion.h / TensorConversion.c

SYNTAX

```
typedef void (*conversionFunction_t)(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Defines the type of every converter function in conversionMatrix. A function of this type reads from inputTensor->data and writes to outputTensor->data. Both tensors must be pre-allocated with correct byte counts before calling.

PARAMETERS

inputTensor: source tensor (data field read, not modified). outputTensor: destination tensor (data field written, quantization already set).

RETURNS

void — output is written into outputTensor.

NOTE

This is a typedef — it creates a TYPE NAME. You do not call this directly. You call a function that matches this signature, e.g. requantSymInt32Tensor.

EXAMPLE

```
conversionFunction_t fn = conversionMatrix[FLOAT32][SYM_INT32]; fn(in, out);
```

SYNTAX

```
void convertTensor(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Main entry point for all type conversions. First checks if input and output have the same qtype_t — if so calls convertTensorsWithSameType (memmove data + copy shape metadata). Otherwise looks up conversionMatrix[inputType][outputType] and calls that function. If the entry is NULL, prints an error and exits.

PARAMETERS

inputTensor: source tensor. outputTensor: pre-allocated output tensor with the desired quantization already set.

RETURNS

void.

NOTE

The same-type shortcut uses memmove (not memcpy) to handle in-place conversion where inputTensor == outputTensor safely.

EXAMPLE

```
// Convert FLOAT32 → SYM_INT32: convertTensor(floatTensor, symTensor); // Internally calls  
conversionMatrix[FLOAT32][SYM_INT32](floatTensor, symTensor)
```


SYNTAX

```
void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Re-quantizes `SYM_INT32` → `SYM_INT32` with a DYNAMICALLY computed fresh scale. Two-pass: Pass A finds `absMax`; computes `scale = absMax/qMax`; Pass B requantizes. Writes the fresh scale to `outputTensor`'s `qConfig`. Wired into `conversionMatrix[SYM_INT32][SYM_INT32]`.

PARAMETERS

`inputTensor`: `SYM_INT32` source tensor. `outputTensor`: `SYM_INT32` destination with `qMaxBits` already set in `qConfig`.

RETURNS

void.

NOTE

IN-PLACE SAFE: `inputTensor == outputTensor` is allowed. Pass A only reads; Pass B does same-index read-then-write over `int32` storage. NEVER saturates by construction: `absMax` maps exactly to $\pm qMax$.

EXAMPLE

```
// Re-scale weights after gradient update: requantSymInt32Tensor(weights, weights); // in-place
```

SYNTAX

```
void requantSymInt32TensorToScale(tensor_t *inputTensor, tensor_t *outputTensor)
```

PURPOSE

Re-quantizes `SYM_INT32` → `SYM_INT32` into a PRE-SET target scale. One pass: reads each mantissa, converts to real, divides by `targetScale`, rounds and clamps. The target scale must be set in `outputTensor->quantization->qConfig->scale` BEFORE calling this function.

PARAMETERS

`inputTensor`: `SYM_INT32` source. `outputTensor`: `SYM_INT32` destination with scale already set by caller.

RETURNS

void.

NOTE

SATURATES BY DESIGN: values outside `[qMin, qMax]` clamp to `qMin/qMax`. NOT wired into `conversionMatrix` — must be called directly. Use this when you have a known target scale (e.g. from a pre-trained model configuration).

EXAMPLE

```
// Requantize to a known scale: outConfig->scale = 0.001f; requantSymInt32TensorToScale(src, dst);
```

SYNTAX

```
char *quantTypeToString(qtype_t t)
```

PURPOSE

Returns a human-readable string for a `qtype_t` value. `INT32`→"`INT32`", `FLOAT32`→"`FLOAT32`", `SYM_INT32`→"`SYM_INT32`", etc. Used in error messages and `PRINT_*` calls.

PARAMETERS

`t`: any `qtype_t` value.

RETURNS

`char*`: a static string literal (do not free or modify).

EXAMPLE

```
printf("Input type: %s\n", quantTypeToString(tensor->quantization->type));
```

Chapter 8 — Arithmetic.h / Arithmetic.c

PURPOSE

Arithmetic.h / Arithmetic.c provides the generic engine for element-wise (point-wise) operations on tensors. Instead of implementing 'add', 'subtract', 'multiply' as separate loops, this file implements ONE loop that takes a function pointer as a parameter. You pass `int32_t addInt32s(a,b)` and the loop calls it for every corresponding element pair. All specialised operation files (Add.c, Sub.c, Mul.c, etc.) use this engine.

WHY THIS FILE EXISTS

Without this engine, every operation file would duplicate hundreds of lines of index-calculation code. Transpose-aware element addressing, dimension validation, and byte I/O live in exactly one place. Each concrete operation is just a two-line function.

DEPENDS ON

Common.h, DTypes.h, Tensor.h, Matmul.h (for `calcNumberOfElementsByTensor`).

USED BY

Add.c, Sub.c, Mul.c, Div.c, and all other operation files. Layer.c. Loss functions.

8.1 Function-Pointer Types

```
// Type for int32 element-wise operations:
typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t a, int32_t b);
// Type for float element-wise operations:
typedef float (*floatElementArithmeticFunc_t)(float a, float b);
// How Add.c uses this pattern:
int32_t addInt32s(int32_t a, int32_t b) { return a + b; }
void addInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out) {
    int32PointWiseArithmetic(a, b, addInt32s, out); // pass the function
}
```

8.2 Index Calculation — Full Worked Example

For a 3×4 matrix (12 elements) with flat index `i=6`:

```

// STEP 1 – calcIndicesByRowIndex(2, dims=[3,4], rowIndex=6, indices)
// offset = 1
// Loop: offset *= dims[0]=3 → 3; offset *= dims[1]=4 → 12
// (offset = total elements = 12)
//
// restIndex = 6
// i=0: offset /= dims[0]=3 → 4; indices[0] = 6/4 = 1; restIndex = 6 - 1*4 = 2
// i=1: offset /= dims[1]=4 → 1; indices[1] = 2/1 = 2; restIndex = 2 - 2*1 = 0
// Result: indices = [1, 2] (row=1, col=2)
// STEP 2 – calcElementIndexByIndices for NON-transposed tensor:
// orderOfDimensions = [0, 1]
// orderedIndices[orderOfDimensions[0]=0] = indices[0] = 1
// orderedIndices[orderOfDimensions[1]=1] = indices[1] = 2
// orderedIndices = [1, 2]
// offset = 12; i=0: offset/=3→4; outputIndex += 1*4 = 4
// i=1: offset/=4→1; outputIndex += 2*1 = 6
// Result: physical index = 6 (no transpose – same as flatIndex)
// STEP 2b – calcElementIndexByIndices for TRANSPOSED tensor:
// orderOfDimensions = [1, 0] (transposed: logical row→physical col)
// orderedIndices[orderOfDimensions[0]=1] = indices[0] = 1 → orderedIndices[1]=1
// orderedIndices[orderOfDimensions[1]=0] = indices[1] = 2 → orderedIndices[0]=2
// orderedIndices = [2, 1]
// offset = 12; i=0: offset/=3→4; outputIndex += 2*4 = 8
// i=1: offset/=4→1; outputIndex += 1*1 = 9
// Result: physical index = 9 (transposed!)

```

8.3 Four Operation Patterns

Pattern	Description
PointWise	$A[i] \text{ op } B[i] \rightarrow \text{output}[i]$ (two in, one out, separate result tensor)
PointWiseInplace	$A[i] \text{ op } B[i] \rightarrow A[i]$ (result overwrites tensor A)
ElementWithTensor	$x \text{ op } A[i] \rightarrow \text{output}[i]$ (scalar combined with every element)
ElementWithTensorInplace	$x \text{ op } A[i] \rightarrow A[i]$ (scalar op, result overwrites tensor)

8.4 Functions in Arithmetic.h / Arithmetic.c

SYNTAX

size_t getDimensionsByIndex(tensor_t *tensor, size_t index)

PURPOSE

Finds the LOGICAL dimension size for axis 'index'. Loops through orderOfDimensions[] to find which physical slot has logical axis 'index', then returns dimensions[that slot]. Transpose-aware.

PARAMETERS

tensor: any tensor. index: logical axis number (0 = outermost / rows).

RETURNS

size_t: size of that logical axis. Exits with PRINT_ERROR if not found.

EXAMPLE

// Non-transposed 3×4 tensor: getDimensionsByIndex(t, 0)=3, getDimensionsByIndex(t, 1)=4 // After transpose(0,1):
getDimensionsByIndex(t, 0)=4, getDimensionsByIndex(t, 1)=3

SYNTAX

```
void orderDims(tensor_t *tensor, size_t *orderedDims)
```

PURPOSE

Fills `orderedDims[i] = getDimensionsByIndex(tensor, i)` for `i` in `0..N-1`. Produces a transpose-aware snapshot of the shape in logical order.

PARAMETERS

`tensor`: any tensor. `orderedDims`: caller-allocated array of size `numberOfDimensions`.

RETURNS

`void`.

NOTE

Called in `doDimensionsMatch` and all `PointWise` functions to normalise the shape before comparison or iteration.

EXAMPLE

```
size_t dims[2]; orderDims(tensor, dims); // dims[0]=rows, dims[1]=cols (respects transpose)
```

SYNTAX

```
bool doDimensionsMatch(tensor_t *a, tensor_t *b)
```

PURPOSE

Checks that two tensors have the same rank AND matching logical dimension sizes. Exits with `PRINT_ERROR + exit(1)` if ranks differ. Returns `false` (without exiting) if any dimension size differs.

PARAMETERS

`a`, `b`: two tensors to compare.

RETURNS

`true` if all logical dimensions match; `false` otherwise.

NOTE

Called as the first guard check inside every `PointWise` function. A false return from this function means the operation cannot proceed.

EXAMPLE

```
if (!doDimensionsMatch(a, b)) { PRINT_ERROR("shape mismatch"); exit(1); }
```

SYNTAX

```
size_t calcTensorIndexByIndices(size_t numberOfDimensions, size_t *dimensions, size_t *indices)
```

PURPOSE

Converts multi-dimensional indices `[i0, i1, ..., iN-1]` to a flat row-major index. Formula: starts from last dimension, accumulates backwards. For 2D: $\text{flat} = i0 \times \text{dim1} + i1$. For 3D: $\text{flat} = i0 \times (\text{dim1} \times \text{dim2}) + i1 \times \text{dim2} + i2$.

PARAMETERS

`numberOfDimensions`: rank. `dimensions`: array of dimension sizes. `indices`: array of per-axis positions.

RETURNS

`size_t`: the flat element index.

EXAMPLE

```
// 3x4 tensor, indices=[1,2]: calcTensorIndexByIndices(2, [3,4], [1,2]) = 1x4 + 2 = 6
```

SYNTAX

```
void calcIndicesByRawIndex(size_t numberOfDims, size_t *dims, size_t rawIndex, size_t *indices)
```

PURPOSE

INVERSE of calcTensorIndexByIndices: converts a flat index back to per-axis indices. First computes total = product of all dims. Then peels one axis at a time: $indices[i] = rawIndex / (total/dims[i])$, then $rawIndex -= indices[i] * (total/dims[i])$.

PARAMETERS

numberOfDims: rank. dims: dimension sizes. rawIndex: flat position. indices: array to fill.

RETURNS

void — writes per-axis indices into the indices array.

NOTE

This is the first of two functions called inside the PointWise loop for each element. It converts the loop counter i (a flat index) into logical row/col indices.

EXAMPLE

```
// dims=[3,4], rawIndex=9: // total=12; peel dim0: 9/4=2, rest=9-8=1; // peel dim1: 1/1=1 → indices=[2,1] (row=2, col=1)
```

SYNTAX

```
size_t calcElementIndexByIndices(size_t numberOfDims, size_t *dims, size_t *indices, size_t *orderOfDimensions)
```

PURPOSE

TRANSPOSE-AWARE flat index. First remaps logical indices to physical order using orderOfDimensions, then computes row-major flat index. This is the second function called in the PointWise loop — it gives the physical byte position for a given logical element.

PARAMETERS

numberOfDims, dims: shape. indices: logical per-axis positions (output of calcIndicesByRawIndex).
orderOfDimensions: orderOfDimensions from the tensor's shape struct.

RETURNS

size_t: physical flat index into the data buffer.

NOTE

For a non-transposed tensor this returns the same value as calcTensorIndexByIndices. For a transposed tensor it swaps axes according to orderOfDimensions.

EXAMPLE

```
// Non-transposed: returns same as calcTensorIndexByIndices // Transposed [1,0]: row and col axes are swapped before computing flat index
```

SYNTAX

```
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor, int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE

The core element-wise loop for int32 tensors, writing to a SEPARATE output. For each flat position i (0 to `numberOfElements-1`): 1. `calcIndicesByRowIndex` → logical indices for A 2. `calcElementIndexByIndices` → physical byte index for A 3. Repeat for B 4. `readBytesAsInt32` from both A and B 5. Call `arithmeticFunc(a, b)` → result 6. `writeInt32ToByteArray` to `outputTensor` at sequential position i

PARAMETERS

`aTensor`: first operand (INT32 or SYM_INT32). `bTensor`: second operand (must match A's shape). `arithmeticFunc`: the operation — add, sub, mul, div, etc. `outputTensor`: pre-allocated result tensor (written sequentially, non-transposed).

RETURNS

void.

NOTE

The output is ALWAYS written sequentially ($i \times 4$ bytes) regardless of A and B transpose state. The output receives a logical, non-transposed layout.

EXAMPLE

```
void addInts(int32_t a, int32_t b) { return a+b; } int32PointWiseArithmetic(a, b, addInts, out);
```

SYNTAX

```
void int32PointWiseArithmeticInplace(tensor_t *aTensor, tensor_t *bTensor, int32ElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE

Same logic as `int32PointWiseArithmetic` but writes the result back into `aTensor->data`. No separate output tensor — `aTensor` serves as both first operand and output.

PARAMETERS

`aTensor`: first operand AND output (modified in place). `bTensor`: second operand. `arithmeticFunc`: the element-wise operation.

RETURNS

void.

NOTE

Use this when the original `aTensor` values are no longer needed after the operation. Saves the memory cost of an additional output buffer.

EXAMPLE

```
int32PointWiseArithmeticInplace(weights, gradients, subInt32s); // weights -= gradients
```

SYNTAX

```
void floatPointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor, floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE

Identical to int32PointWiseArithmetic but for FLOAT32 tensors. Uses readBytesAsFloat / writeFloatToByteArray instead of int32 variants.

PARAMETERS

aTensor, bTensor: FLOAT32 tensors. arithmeticFunc: float function. outputTensor: FLOAT32 result.

RETURNS

void.

SYNTAX

```
void floatPointWiseArithmeticInplace(tensor_t *aTensor, tensor_t *bTensor, floatElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE

Float in-place version. Result overwrites aTensor.

PARAMETERS

aTensor: in/out. bTensor: second operand. arithmeticFunc: float operation.

RETURNS

void.

SYNTAX

```
void int32ElementWithTensorArithmetic(tensor_t *aTensor, int32_t x, int32ElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE

Applies arithmeticFunc(element, x) to EVERY element of aTensor, writing results to outputTensor. Simpler than PointWise — sequential loop, no dimension check, because x is the same for every element.

PARAMETERS

aTensor: input tensor. x: the scalar operand applied to every element. arithmeticFunc: operation (e.g. multiply all by x). outputTensor: result.

RETURNS

void.

NOTE

No doDimensionsMatch call needed because one operand is a scalar.

EXAMPLE

```
// Multiply every element by 2: int32ElementWithTensorArithmetic(t, 2, mulInt32s, out);
```

SYNTAX

```
void int32ElementWithTensorArithmeticInplace(tensor_t *aTensor, int32_t x, int32ElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE

Same scalar operation but result overwrites aTensor.

PARAMETERS

aTensor: in/out. x: scalar. arithmeticFunc: operation.

RETURNS

void.

SYNTAX

```
void floatElementWithTensorArithmetic(tensor_t *aTensor, float x, floatElementArithmeticFunc_t arithmeticFunc, tensor_t *outputTensor)
```

PURPOSE

Float scalar operation: applies arithmeticFunc(element, x) to every float element.

PARAMETERS

aTensor: FLOAT32 input. x: float scalar. arithmeticFunc. outputTensor: result.

RETURNS

void.

SYNTAX

```
void floatElementWithTensorArithmeticInplace(tensor_t *aTensor, float x, floatElementArithmeticFunc_t arithmeticFunc)
```

PURPOSE

Float scalar in-place version.

PARAMETERS

aTensor: FLOAT32 in/out. x: float scalar. arithmeticFunc.

RETURNS

void.

Chapter 9 — Specialised Arithmetic Operation Files

Each file below wraps one concrete math operation. The pattern is always: (1) define a small function matching `int32ElementArithmeticFunc_t` or `floatElementArithmeticFunc_t`, then (2) call the generic engine in `Arithmetic.c`. All files include `Arithmetic.h` and `Tensor.h`.

9.1 Add.h / Add.c — Addition

DEPENDS ON

`Arithmetic.h`, `Tensor.h`.

```
float addFloat32s(float a, float b)
```

Returns $a + b$. The leaf function passed to `floatPointWiseArithmetic` for float tensor addition.

```
int32_t addInt32s(int32_t a, int32_t b)
```

Returns $a + b$. Used for INT32 and SYM_INT32 tensor addition.

```
void addInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *outputTensor)
```

Adds two INT32 tensors element-wise. Calls `int32PointWiseArithmetic` with `addInt32s`.

```
void addInt32TensorsInplace(tensor_t *a, tensor_t *b)
```

In-place add: $a[i] += b[i]$. Calls `int32PointWiseArithmeticInplace`.

```
void addInt32ElementWithInt32Tensor(tensor_t *a, int32_t b, tensor_t *outputTensor)
```

Adds scalar b to every element of tensor a . Calls `int32ElementWithTensorArithmetic`.

```
void addInt32ElementWithInt32TensorInplace(tensor_t *a, int32_t b)
```

In-place scalar add: $a[i] += b$. Calls `int32ElementWithTensorArithmeticInplace`.

```
void addFloat32Tensors(tensor_t *a, tensor_t *b, tensor_t *outputTensor)
```

Adds two FLOAT32 tensors element-wise.

```
void addFloat32TensorsInplace(tensor_t *a, tensor_t *b)
```

In-place float add.

```
void addFloat32ElementWithFloat32Tensor(tensor_t *a, float b, tensor_t *outputTensor)
```

Adds float scalar b to every element of tensor a .

```
void addFloat32ElementWithFloat32TensorInplace(tensor_t *a, float b)
```

In-place float scalar add.

```
void addInt32TensorToSymInt32TensorInplace(tensor_t *symInt32Tensor, tensor_t *int32Tensor)
```

Adds an INT32 bias tensor to a SYM_INT32 activation tensor in place. Used in `linearForward` after `matmul` to add the bias.

```
void addFloat32TensorToSymInt32TensorInplace(tensor_t *symInt32, tensor_t *float32)
```

Adds a FLOAT32 tensor to a SYM_INT32 tensor in place.

```
void addSymInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

Adds two SYM_INT32 tensors element-wise.

```
void addSymInt32TensorsInplace(tensor_t *a, tensor_t *b)
```

In-place SYM_INT32 addition.

```
size_t getAddInstructionCounter()
```

Returns a counter of how many add operations have been executed. Used for profiling and unit tests that verify instruction counts.

9.2 Sub.h / Sub.c — Subtraction

DEPENDS ON

`Arithmetic.h`, `Tensor.h`.

```
int32_t subInt32s(int32_t a, int32_t b)
```

Returns $a - b$.

```
float subFloat32s(float a, float b)
```

Returns $a - b$.

```
void subInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

Element-wise $a - b \rightarrow out$.

```
void subInt32TensorsInplace(tensor_t *a, tensor_t *b)
In-place:  $a[i] -= b[i]$ .

void subInt32ElementWithInt32Tensor(tensor_t *a, int32_t b, tensor_t *out)
Scalar:  $a[i] - b \rightarrow out$ .

void subInt32ElementWithInt32TensorInplace(tensor_t *a, int32_t b)
In-place:  $a[i] -= b$ .

void subFloat32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
Float element-wise subtract.

void subFloat32TensorsInplace(tensor_t *a, tensor_t *b)
Float in-place subtract.

void subFloat32ElementWithFloat32Tensor(tensor_t *a, float b, tensor_t *out)
Float scalar subtract.

void subFloat32ElementWithFloat32TensorInplace(tensor_t *a, float b)
Float scalar in-place.

void subSymInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
SYM_INT32 subtract.

void subSymInt32TensorsInplace(tensor_t *a, tensor_t *b)
SYM_INT32 in-place subtract.

size_t getSubInstructionCounter()
Profiling counter for subtract operations.
```

9.3 Mul.h / Mul.c — Multiplication

DEPENDS ON

Arithmetic.h, Tensor.h.

```
int32_t mulInt32s(int32_t a, int32_t b)
Returns  $a * b$ .

float mulFloat32s(float a, float b)
Returns  $a * b$ .

void mulInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
Element-wise  $a * b \rightarrow out$ .

void mulInt32TensorsInplace(tensor_t *a, tensor_t *b)
In-place:  $a[i] *= b[i]$ .

void mulInt32ElementWithInt32Tensor(tensor_t *a, int32_t b, tensor_t *out)
Scale tensor by scalar.

void mulInt32ElementWithInt32TensorInplace(tensor_t *a, int32_t b)
In-place scalar scale.

void mulFloat32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
Float element-wise multiply.

void mulFloat32TensorsInplace(tensor_t *a, tensor_t *b)
Float in-place multiply.

void mulFloat32ElementWithFloat32Tensor(tensor_t *a, float b, tensor_t *out)
Float scalar multiply.

void mulFloat32ElementWithFloat32TensorInplace(tensor_t *a, float b)
Float scalar in-place.

void mulSymInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
SYM_INT32 element-wise multiply.

void mulSymInt32TensorsInplace(tensor_t *a, tensor_t *b)
SYM_INT32 in-place multiply.

size_t getMulInstructionCounter()
Profiling counter.
```

9.4 Div.h / Div.c — Division

DEPENDS ON

Arithmetic.h, Tensor.h.

```
int32_t divInt32s(int32_t a, int32_t b)
```

Returns a / b (integer division, truncates toward zero).

```
float divFloat32s(float a, float b)
```

Returns a / b .

```
void divInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

Element-wise $a/b \rightarrow out$.

```
void divInt32TensorsInplace(tensor_t *a, tensor_t *b)
```

In-place: $a[i] /= b[i]$.

```
void divInt32ElementWithInt32Tensor(tensor_t *a, int32_t b, tensor_t *out)
```

Divide tensor by scalar.

```
void divInt32ElementWithInt32TensorInplace(tensor_t *a, int32_t b)
```

In-place tensor/scalar.

```
void divFloat32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

Float element-wise divide.

```
void divFloat32TensorsInplace(tensor_t *a, tensor_t *b)
```

Float in-place divide.

```
void divFloat32ElementWithFloat32Tensor(tensor_t *a, float b, tensor_t *out)
```

Float scalar divide.

```
void divFloat32ElementWithFloat32TensorInplace(tensor_t *a, float b)
```

Float in-place scalar divide.

```
void divSymInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

SYM_INT32 element-wise divide.

```
void divSymInt32TensorsInplace(tensor_t *a, tensor_t *b)
```

SYM_INT32 in-place divide.

```
size_t getDivInstructionCounter()
```

Profiling counter.

9.5 Matmul.h / Matmul.c — Matrix Multiplication

DEPENDS ON

DTypes.h, Tensor.h.

NOTE

Matmul uses its own inner loop (NOT the element-wise engine) because matrix multiplication reads the SAME row many times and the SAME column many times — a plain element-wise loop cannot express this access pattern.

```
void matmulInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

Multiplies two INT32 matrices: $C[i][j] = \sum_k A[i][k] * B[k][j]$. Output shape: $[a.rows, b.cols]$.

```
void matmulFloat32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

Float matrix multiply.

```
void matmulSymInt32Tensors(tensor_t *a, tensor_t *b, tensor_t *out)
```

SYM_INT32 matrix multiply. Used in linearForwardSymInt32.

```
void matmulFloat32TensorsWithBias(tensor_t *a, tensor_t *b, tensor_t *out, tensor_t *bias)
```

Float matmul + add bias vector in one call. $out = A @ B + bias$.

```
void matmulSymInt32TensorsWithBias(tensor_t *a, tensor_t *b, tensor_t *out, tensor_t *bias)
```

SYM_INT32 matmul + add bias.

```
size_t getMatmulInstructionCounter()
```

Profiling counter for matmul operations.

9.6 MinMax.h / MinMax.c — Finding Extremes

float findAbsMaxFloat(uint8_t *bytes, size_t numberOfElements)

Scans all float elements and returns the maximum absolute value. Used in requantSymInt32Tensor Pass A to find absMax for scale computation.

float findMaxFloat(uint8_t *bytes, size_t numberOfElements)

Returns the maximum float value (signed, not absolute).

float findMinFloat(uint8_t *bytes, size_t numberOfElements)

Returns the minimum float value.

int32_t findMaxInt32(uint8_t *bytes, size_t numberOfElements)

Returns the maximum int32_t value in the byte buffer.

int32_t findMinInt32(uint8_t *bytes, size_t numberOfElements)

Returns the minimum int32_t value.

9.7 Rounding.h / Rounding.c

int32_t roundByMode(float input, roundingMode_t roundingMode)

Rounds a float to int32_t using HALF_AWAY (standard) or SR_HALF_AWAY (stochastic). Used in all quantization operations to convert real values to integer mantissas. Stochastic rounding adds uniform noise in [-0.5, 0.5) before rounding, which reduces systematic bias during training.

float clamp(float input, float min, float max)

Clamps input to the range [min, max]. Returns min if input < min, max if input > max, otherwise input unchanged. Used in requantSymInt32Tensor to keep quantized values within [qMin, qMax].

9.8 Sum.h / Sum.c, Log.h / Log.c

int32_t sumint32(int32_t *values, size_t numberOfValues)

Sums all values in an int32_t array. Used in Softmax and LayerNorm.

float sumFloat(float *values, size_t numberOfValues)

Sums all values in a float array.

float logFloat(float x)

Returns the natural logarithm ln(x). Used in CrossEntropy loss: -sum(label * log(output)).

Chapter 10 — Neural-Network Layer Files

PURPOSE

Layer files implement the forward (inference) and backward (gradient) passes for each neural-network operation. They build on the arithmetic engine (Layer 4) and the conversion system (Layer 3). The base Layer.h defines function-pointer types so layers are interchangeable at runtime.

DEPENDS ON

Tensor.h, Arithmetic.h, TensorConversion.h, and each other's specialised headers.

10.1 Layer.h / Layer.c — Base Layer Abstraction

```
// The base layer struct:
typedef struct layer {
    layerType_t type; // LINEAR, RELU, CONV1D, SOFTMAX, ...
    layerConfig_t *config; // union: points to the specific config struct
} layer_t;

// Function-pointer types for forward/backward dispatch:
typedef void (*forwardFn_t)(layer_t *layer, tensor_t *input, tensor_t *output);
typedef void (*backwardFn_t)(layer_t *layer, tensor_t *forwardInput,
    tensor_t *loss, tensor_t *propLoss);
typedef void (*calcOutputShapeFn_t)(layer_t *layer, shape_t *in, shape_t *out);
// layerFunctions[] dispatch table (indexed by layerType_t):
extern layerFunctions_t layerFunctions[]; // set up in Layer.c
// To run forward pass on any layer:
layerFunctions[layer->type].forward(layer, input, output);
```

SYNTAX

```
void initLayer(layer_t *layer, layerType_t layerType, layerConfig_t *config)
```

PURPOSE

Sets layer->type and layer->config. The starting point for creating any layer.

PARAMETERS

layer: target struct. layerType: e.g. LINEAR, RELU, CONV1D. config: pointer to the specific config struct (linear, relu, conv1d, etc.) wrapped in a layerConfig_t union.

RETURNS

void.

10.2 Linear.h / Linear.c — Fully-Connected Layer

A Linear (fully-connected) layer computes: $\text{output} = \text{input} \times \text{weights}^T + \text{bias}$. In matrix notation for input $[1 \times \text{in_features}]$: $\text{output}[j] = \sum_i (\text{input}[i] \times \text{weight}[j][i]) + \text{bias}[j]$.

```
void linearInitConfig(linearConfig_t *cfg, parameter_t *weights, parameter_t *bias, quantization_t
    *forwardQ, quantization_t *weightGradQ, quantization_t *biasGradQ, quantization_t *propLossQ)
Fills all fields of a linearConfig_t. Must be called before using the layer. weights holds the learned weight matrix [out_features × in_features]. bias holds the learned bias vector [out_features]. forwardQ specifies quantization of the forward-pass output. weightGradQ / biasGradQ specify quantization of gradient tensors. propLossQ specifies quantization of the propagated loss (input gradient).
```

```
void linearForwardFloat(tensor_t *w, tensor_t *b, tensor_t *input, tensor_t *output)
Forward pass assuming all tensors are FLOAT32. Calls matmulFloat32TensorsWithBias. Fastest path for floating-point training.
```

```
void linearForwardSymInt32(tensor_t *w, tensor_t *b, tensor_t *input, tensor_t *output)
Forward pass assuming all tensors are SYM_INT32. Calls matmulSymInt32TensorsWithBias then requantSymInt32Tensor.
```

```
void linearForward(layer_t *linearLayer, tensor_t *input, tensor_t *output)
```

General forward pass that handles any quantization combination by checking forwardQ->type and dispatching to the correct specialised forward function.

```
void linearBackward(layer_t *layer, tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss)
```

Backward pass: computes weight gradients, bias gradients, and propagated loss (gradient w.r.t. the layer input) for use by the previous layer.

```
void linearCalcWeightGradsFloat32(tensor_t *loss, tensor_t *forwardInput, tensor_t *weightGrads)
```

Computes weight gradients for float mode: $dW = loss^T \times forwardInput$. A matrix multiply — each output error is distributed back to the weights that contributed to it.

```
void linearCalcBiasGradsFloat32(tensor_t *biasGrads, tensor_t *loss)
```

Computes bias gradients: $db = \text{sum of loss across the batch dimension}$.

```
void linearCalcPropLossFloat32(tensor_t *weights, tensor_t *loss, tensor_t *propLoss)
```

Computes propagated loss: $dX = loss \times weights$. This is the gradient w.r.t. the layer input, passed back to the previous layer.

```
void linearCalcOutputShape(layer_t *linearLayer, shape_t *inputShape, shape_t *outputShape)
```

Computes the output shape for a given input shape. output shape = [batch, out_features].

10.3 Conv1d.h / Conv1d.c — 1D Convolution Layer

Conv1d slides a learnable filter of size K over a 1D sequence of length L. For each position, it computes a weighted sum of the input values in the filter window. Used in audio/speech processing and signal classification.

```
void initConv1dConfigWithWeightsAndBias(conv1dConfig_t *cfg, kernel_t *kernel, parameter_t *weights, parameter_t *bias, size_t groups, quantization_t *forwardQ, quantization_t *weightGradQ, quantization_t *biasGradQ, quantization_t *propLossQ)
```

Fills a conv1dConfig_t. kernel describes stride/padding/dilation. weights has shape [Cout, Cin/groups, K]. bias has shape [Cout] or is NULL.

```
void conv1dForward(layer_t *layer, tensor_t *input, tensor_t *output)
```

General forward pass for 1D convolution. Dispatches based on quantization type.

```
void conv1dForwardFloat(layer_t *layer, tensor_t *input, tensor_t *output)
```

FLOAT32 specialised forward pass using Conv1dKernel.

```
void conv1dBackward(layer_t *layer, tensor_t *forwardInput, tensor_t *lossGrad, tensor_t *propLoss)
```

Backward pass: computes filter gradients and propagates loss to previous layer.

```
void conv1dCalcOutputShape(layer_t *layer, shape_t *inputShape, shape_t *outputShape)
```

Computes output length from input length, kernel size, stride, and padding.

10.4 Relu.h / Relu.c — Rectified Linear Unit

ReLU computes $output[i] = \max(0, input[i])$. It is the standard non-linearity that introduces the ability to learn non-linear functions. In the backward pass, the gradient flows only for elements where $input > 0$.

```
void reluInit(reluConfig_t *cfg, quantization_t *forwardQ, quantization_t *backwardQ)
```

Fills a reluConfig_t.

```
void reluForwardFloat(tensor_t *input, tensor_t *output)
```

ReLU for FLOAT32: $output[i] = \max(0.0f, input[i])$.

```
void reluForwardAsym(tensor_t *input, tensor_t *output)
```

ReLU for ASYM quantized tensors.

```
void reluForward(layer_t *layer, tensor_t *input, tensor_t *output)
```

General forward — dispatches to float or asym version based on quantization.

```
void reluBackwardFloat(tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss)
```

Backward pass: $propLoss[i] = loss[i]$ if $forwardInput[i] > 0$, else 0.

```
void reluBackward(layer_t *layer, tensor_t *forwardInput, tensor_t *loss, tensor_t *propLoss)
```

General backward — dispatches to float or asym version.

```
void reluCalcOutputShape(layer_t *layer, shape_t *in, shape_t *out)
```

Output shape is identical to input shape (ReLU is shape-preserving).

10.5 Softmax.h / Softmax.c — Probability Distribution

Softmax converts a vector of raw scores into a probability distribution: $\text{output}[i] = \exp(\text{input}[i]) / \sum_j \exp(\text{input}[j])$. All outputs are positive and sum to 1. Used as the final layer in classifiers.

```
void softmaxInitConfig(softmaxConfig_t *cfg, quantization_t *forwardQ, quantization_t *backwardQ)
```

Fills a softmaxConfig_t.

```
void softmaxForward(layer_t *layer, tensor_t *input, tensor_t *output)
```

Runs softmax: $\exp(x) / \sum(\exp(x))$. Numerically stable implementation subtracts max before exponentiating.

```
void softmaxBackward(layer_t *layer, tensor_t *input, tensor_t *loss, tensor_t *propLoss)
```

Backward pass through softmax — combined with cross-entropy this simplifies to $\text{propLoss}[i] = \text{output}[i] - \text{label}[i]$.

```
void softmaxCalcOutputShape(layer_t *layer, shape_t *in, shape_t *out)
```

Output shape equals input shape.

Chapter 11 — Training Infrastructure

11.1 LossFunction.h — Loss Computation Base

The loss function measures how wrong the model's predictions are. A smaller loss means better predictions. During training, the gradient of the loss with respect to the output is used to update the weights.

```
typedef float (*lossFwdFn_t)(tensor_t *output, tensor_t *label, reduction_t reduction)
Function-pointer type for loss forward pass. Returns a scalar loss value. reduction: REDUCTION_MEAN divides by number of
elements; REDUCTION_SUM does not.

typedef void (*lossBwdFn_t)(tensor_t *output, tensor_t *label, tensor_t *result)
Function-pointer type for loss backward pass. Writes per-element gradients to result.

typedef float (*computeMeanScaleFn_t)(size_t totalSamples, tensor_t *modelOutput)
Computes the 1/N scale factor for MEAN reduction. Called by the training loop before applying gradients when backwardReduction
== REDUCTION_MEAN.

lossConfig_t defaultLossConfig(lossFuncType_t funcType)
Returns a default-initialised lossConfig_t with backwardReduction=REDUCTION_MEAN.

extern lossFunctions_t lossFunctions[]
Dispatch table indexed by lossFuncType_t (MSE=0, CROSS_ENTROPY=1). Each entry holds forward, backward, and
computeMeanScale function pointers.
```

11.2 Optimizer.h / Sgd.h — Weight Update

The optimizer applies the computed gradients to update the model weights. SGD (Stochastic Gradient Descent):
 $\text{weight} -= \text{learningRate} \times \text{gradient}$.

```
void sgdInit(sgd_t *sgd, float learningRate, float momentumFactor, float weightDecay)
Initialises an SGD optimiser. learningRate: step size (e.g. 0.001). momentumFactor: fraction of previous gradient to add (0.0 = no
momentum, 0.9 = typical). weightDecay: L2 regularisation strength (0.0 = no regularisation).

void sgdStep(optimizer_t *optimizer)
Performs one optimiser step: for each parameter, param -= learningRate × gradient. Calls subFloat32Tensors or
subSymInt32Tensors.

void sgdStepM(optimizer_t *optimizer)
SGD with momentum step. Updates a velocity buffer and applies it to the weights.

void sgdZeroGrad(optimizer_t *optimizer)
Sets all gradient tensors to zero. Call this before each training batch to prevent gradient accumulation across batches.

size_t calcTotalNumberOfStates(layer_t **model, size_t sizeModel)
Counts total state buffers needed for all parameters in the model. Used to pre-allocate momentum buffers for SGD with momentum.
```

11.3 DataLoader.h — Batched Dataset Iteration

DataLoader manages loading samples from storage, batching them, shuffling the order between epochs, and applying transforms.

```
typedef tensor_t *(*transformFn_t)(tensor_t *tensor)
Type for a transform function applied to samples (e.g. normalisation).

typedef sample_t *(*getSampleFn_t)(size_t id)
Type for a function that loads sample 'id' from storage.

typedef size_t (*getDatasetSizeFn_t)()
Type for a function that returns the total number of samples.

typedef batch_t *(*getBatchFn_t)(dataLoader_t *loader, size_t index)
Type for a function that assembles a batch of batchSize samples.
```



```
void initDataLoader(dataLoader_t *loader, getSampleFn_t getSample, getDatasetSizeFn_t getSize,
getBatchFn_t getBatch, uint16_t batchSize, transformFn_t transform, transformFn_t targetTransform,
bool shuffle, uint64_t shuffleSeed, size_t *indices, bool dropLast)
```

Initialises the DataLoader with all its function pointers and configuration. shuffle=true randomises sample order each epoch using shuffleSeed. dropLast=true discards the final incomplete batch if dataset size is not divisible by batchSize.

11.4 RNG.h / RNG.c — Random Number Generator

A lightweight pseudo-random number generator used by Dropout (random masking) and data shuffling. Module-global state — not thread-safe.

```
void rngSetSeed(uint32_t seed)
```

Sets the RNG seed. Call once at the start of training for reproducible results.

```
uint32_t rngGetSeed(void)
```

Returns the current seed value.

```
float rngNextFloat(void)
```

Returns a pseudo-random float uniformly distributed in [0.0, 1.0). Used by Bernoulli to decide whether each element is masked.

```
void rngShuffleIndices(size_t *indices, size_t n)
```

Shuffles the indices array in-place using Fisher-Yates algorithm. Called by DataLoader at the start of each epoch when shuffle=true.

11.5 Serialize.h / Serialize.c — Model Storage

Serialization writes weight tensors to a binary file (flash or SD card) so a trained model can be reloaded on the next boot.

```
void serializeTensor(tensor_t *tensor, FILE *f)
```

Writes a tensor's data, shape, and quantization fields to file f as binary. The file pointer f must already be open in write-binary mode ("wb").

```
void serializeParameter(parameter_t *parameter, FILE *f)
```

Serializes both the param and grad tensors of a parameter_t.

```
void serializeModel(layer_t **model, size_t sizeModel, FILE *f)
```

Serializes all learnable parameters in the entire model by iterating over layers and calling serializeParameter for each one.

Chapter 12 — User API (src/userApi/)

PURPOSE

The userApi folder exposes the library to application code. It wraps all internal details behind clean, stable function signatures. An application should never include internal headers like Arithmetic.h directly — it includes only userApi headers.

API Header	Purpose
InferenceApi.h	Run a forward pass through the full model: predict(model, input, output).
TrainingLoopApi.h	Run one training epoch: iterate batches, forward, loss, backward, optimiser step.
StateDictApi.h	Load or save named parameter tensors (like PyTorch state_dict).
StorageApi.h	Abstract over flash / file-system storage for serialised models.
TensorApi.h	Create, fill, and inspect tensors without touching internal struct fields.
QuantizationApi.h	Initialise quantization configs and apply quantization to tensors.
LayerWeightsApi.h	Read or write weight tensors of a specific layer by index.
ModelValidationApi.h	Run the model on a validation set and report accuracy/loss.
LinearApi.h	Create and configure a Linear layer.
Conv1dApi.h	Create and configure a Conv1d layer.
ReluApi.h	Create a Relu layer.
SoftmaxApi.h	Create a Softmax layer.
DropoutApi.h	Create a Dropout layer with a given drop probability.
SgdApi.h	Create and configure an SGD optimiser.
DataLoaderApi.h	Create and configure a DataLoader.

12.1 How to Use the API — Training Loop Example

```
#include "TrainingLoopApi.h"
#include "InferenceApi.h"
#include "SgdApi.h"
#include "DataLoaderApi.h"
// 1. Create model layers
layer_t linear1, relu1, linear2, softmax1;
createLinearLayer(&linear1, weights1, bias1, &q_linear1);
createReluLayer(&relu1, &q_relu1);
layer_t *model[] = { &linear1, &relu1, &linear2, &softmax1 };
// 2. Create optimiser
sgd_t sgd; optimizer_t optimizer;
sgdInit(&sgd, 0.001f, 0.0f, 0.0f); // lr=0.001, no momentum, no decay
// 3. Training loop
for (int epoch = 0; epoch < 100; epoch++) {
    trainingEpochDefault(model, 4, &optimizer, &loader, &lossCfg);
}
// 4. Run inference on new input
predict(model, 4, &inputTensor, &outputTensor);
// 5. Save weights
FILE *f = fopen("model.bin", "wb");
serializeModel(model, 4, f);
fclose(f);
```

Chapter 13 — Quick Function Reference Index

The table below lists every public function and macro documented in this reference with its file and a one-line description.

Function / Macro / Type	File	One-line description
PRINT_DEBUG(str,...)	Common.h	Yellow debug print (DLEVEL>=3)
PRINT_INFO(str,...)	Common.h	Info print (DLEVEL>=2)
PRINT_ERROR(str,...)	Common.h	Red error print (DLEVEL>=1)
PRINT_BYTE_ARRAY(p,a,n)	Common.h	Hex-dump byte array
readBytesAsInt32	DTypes.h	4 bytes → int32_t (memcpy)
readNumberOfBytesAsInt32	DTypes.h	N bytes → int32_t sign-extended
readBytesAsInt32Array	DTypes.h	Byte buffer → int32_t array
readBytesAsFloat	DTypes.h	4 bytes → float
readBytesAsFloatArray	DTypes.h	Byte buffer → float array
writeInt32ToByteArray	DTypes.h	int32_t → 4 bytes
writeInt32ArrayToByteArray	DTypes.h	int32_t array → byte buffer
writeFloatToByteArray	DTypes.h	float → 4 bytes
writeFloatArrayToByteArray	DTypes.h	float array → byte buffer
qtype_t (enum)	Quantization.h	INT32/FLOAT32/SYM_INT32/SYM/ASYM/BOOL
initSymInt32QConfig	Quantization.h	Fill symInt32QConfig (16-bit default)
initSymInt32QConfigWithQMaxBits	Quantization.h	Fill symInt32QConfig (custom bits)
initSymQConfig	Quantization.h	Fill symQConfig
initAsymQConfig	Quantization.h	Fill asymQConfig
initInt32Quantization	Quantization.h	Set type=INT32
initFloat32Quantization	Quantization.h	Set type=FLOAT32
initBoolQuantization	Quantization.h	Set type=BOOL
initSymInt32Quantization	Quantization.h	Set type=SYM_INT32 + link config
initSymQuantization	Quantization.h	Set type=SYM + link config
initAsymQuantization	Quantization.h	Set type=ASYM + link config
getBitmask	Tensor.h	Build bitmask for bit range
writeByte	Tensor.h	Write bits into a byte
readByte	Tensor.h	Extract bits from a byte
byteConversion	Tensor.h	Convert between bit widths
tensorBoolGet	Tensor.h	Read one bool from BOOL tensor
tensorBoolSet	Tensor.h	Write one bool to BOOL tensor
getParamFromParameter	Tensor.h	Get weight tensor from parameter_t
getGradFromParameter	Tensor.h	Get gradient tensor from parameter_t
calcBytesPerElement	Tensor.h	Bytes per element for given qtype
calcBitsPerElement	Tensor.h	Bits per element for given qtype
calcBytesPerTensor	Tensor.h	Total bytes in data buffer
calcNumberOfBytesForData	Tensor.h	Bytes for N elements of given type
calcNumberOfElementsByShape	Tensor.h	Product of all dimension sizes
calcNumberOfElementsByTensor	Tensor.h	Elements in a tensor
calcNumberOfElementsByParameter	Tensor.h	Elements in parameter weight
transposeTensor	Tensor.h	Swap two axes (zero-cost)
setTensorValues	Tensor.h	Fill all tensor_t fields
setTensorValuesForConversion	Tensor.h	Set data+quant, copy shape

Function / Macro / Type	File	One-line description
setParameterValues	Tensor.h	Fill a parameter_t
setOrderOfDimsForNewTensor	Tensor.h	Init orderOfDimensions to identity
setShape	Tensor.h	Fill a shape_t
printTensor	Tensor.h	Debug-print tensor
printShape	Tensor.h	Debug-print shape
copyTensor	Tensor.h	Shallow copy tensor_t
conversionFunction_t	TensorConversion.h	Function-pointer type for converters
convertTensor	TensorConversion.h	Dispatch conversion via matrix
requantSymInt32Tensor	TensorConversion.h	SYM_INT32→SYM_INT32 dynamic scale
requantSymInt32TensorToScale	TensorConversion.h	SYM_INT32→SYM_INT32 fixed scale
quantTypeToString	TensorConversion.h	qtype_t → readable string
conversionMatrix[6][6]	TensorConversion.h	Dispatch table of all converters
int32ElementArithmeticFunc_t	Arithmetic.h	Function-pointer type (int32 op)
floatElementArithmeticFunc_t	Arithmetic.h	Function-pointer type (float op)
getDimensionsByIndex	Arithmetic.h	Logical dimension size for axis
orderDims	Arithmetic.h	Fill array with logical dim sizes
doDimensionsMatch	Arithmetic.h	Check shapes are compatible
calcTensorIndexByIndices	Arithmetic.h	Multi-dim indices → flat index
calcIndicesByRawIndex	Arithmetic.h	Flat index → multi-dim indices
calcElementIndexByIndices	Arithmetic.h	Transpose-aware physical index
int32PointWiseArithmetic	Arithmetic.h	int32 A op B → output
int32PointWiseArithmeticInplace	Arithmetic.h	int32 A op B → A
floatPointWiseArithmetic	Arithmetic.h	float A op B → output
floatPointWiseArithmeticInplace	Arithmetic.h	float A op B → A
int32ElementWithTensorArithmetic	Arithmetic.h	int32 scalar op tensor → out
int32ElementWithTensorArithmeticInplace	Arithmetic.h	int32 scalar op → tensor
floatElementWithTensorArithmetic	Arithmetic.h	float scalar op tensor → out
floatElementWithTensorArithmeticInplace	Arithmetic.h	float scalar op → tensor
addInt32s / addFloat32s	Add.h	Leaf add function (returns a+b)
addInt32Tensors	Add.h	Element-wise int32 add
addSymInt32Tensors	Add.h	SYM_INT32 element-wise add
subInt32s / subFloat32s	Sub.h	Leaf subtract function
subInt32Tensors	Sub.h	Element-wise int32 subtract
mulInt32s / mulFloat32s	Mul.h	Leaf multiply function
mulInt32Tensors	Mul.h	Element-wise int32 multiply
divInt32s / divFloat32s	Div.h	Leaf divide function
divInt32Tensors	Div.h	Element-wise int32 divide
matmulInt32Tensors	Matmul.h	Matrix multiply int32
matmulFloat32TensorsWithBias	Matmul.h	Float matmul + bias
findAbsMaxFloat	MinMax.h	Max absolute float value
findMaxInt32/findMinInt32	MinMax.h	Max/min int32 in buffer
roundByMode	Rounding.h	Float→int32 round (HALF_AWAY/SR)

Function / Macro / Type	File	One-line description
clamp	Rounding.h	Clamp float to [min, max]
sumint32 / sumFloat	Sum.h	Sum all values in array
logFloat	Log.h	Natural logarithm ln(x)
initLayer	Layer.h	Set layer type and config
layerFunctions[]	Layer.h	Dispatch table: forward/backward by type
linearForward	Linear.h	Linear layer forward pass
linearBackward	Linear.h	Linear layer backward pass
conv1dForward	Conv1d.h	Conv1d forward pass
conv1dBackward	Conv1d.h	Conv1d backward pass
reluForward	Relu.h	ReLU forward pass
reluBackward	Relu.h	ReLU backward pass
softmaxForward	Softmax.h	Softmax forward pass
defaultLossConfig	LossFunction.h	Default loss config (MEAN reduction)
lossFunctions[]	LossFunction.h	Dispatch table: loss fwd/bwd
sgdInit	Sgd.h	Initialise SGD optimiser
sgdStep	Sgd.h	Apply gradients to weights
sgdZeroGrad	Sgd.h	Zero all gradient tensors
initDataLoader	DataLoader.h	Configure dataset iterator
rngSetSeed	RNG.h	Set RNG seed
rngNextFloat	RNG.h	Uniform random float in [0,1)
rngShuffleIndices	RNG.h	Fisher-Yates shuffle indices array
serializeTensor	Serialize.h	Write tensor to binary file
serializeModel	Serialize.h	Serialise all model parameters

Chapter 14 — Line-by-Line Code Walkthrough: Index Functions

This chapter takes the most important functions in the library and explains every single line of code. For each line you will see: the line number, the exact code, and a paragraph explaining what it does, why it exists, and what would break if it were removed.

14.1 calcIndicesByRowIndex — Converting a Flat Index to Per-Axis Indices

This function is the inverse of row-major indexing. Given a flat position (like element 6 of a 3×4 matrix) it recovers which row and column that position corresponds to.

```
void calcIndicesByRowIndex(size_t numberOfDims, size_t *dims,
    size_t rowIndex, size_t *indices) {
    size_t offset = 1;
    for (size_t i = 0; i < numberOfDims; i++) {
        offset *= dims[i];
    }
    size_t restIndex = rowIndex;
    for (size_t i = 0; i < numberOfDims; i++) {
        offset /= dims[i];
        indices[i] = restIndex / offset;
        restIndex -= indices[i] * offset;
    }
}
```

1	<code>void calcIndicesByRowIndex(size_t numberOfDims, size_t *dims,</code>	Function declaration — 'void' means it returns nothing and writes its result directly into the 'indices' array (output parameter). 'size_t numberOfDims' is the rank of the tensor (2 for a matrix, 3 for a 3D tensor). 'size_t *dims' is a pointer to the array of dimension sizes.
2	<code>size_t rowIndex, size_t *indices) {</code>	'size_t rowIndex' is the flat integer position we want to decode — e.g. 6 means 'the 7th element counting from 0'. 'size_t *indices' is the OUTPUT array: the function will fill this with the per-axis positions (e.g. [row=1, col=2]). The caller must have allocated this array.
3	<code>size_t offset = 1;</code>	Initialise offset to 1. This variable will accumulate the total number of elements in the tensor (the product of all dimension sizes). We start at 1 because multiplying by 1 has no effect, so the loop below can start immediately.
4	<code>for (size_t i = 0; i < numberOfDims; i++) {</code>	Loop over all dimensions. size_t is the correct type for loop counters that are used as array indices — it is always unsigned and pointer-wide.
5	<code>offset *= dims[i];</code>	Multiply offset by the size of dimension i. After this loop, offset equals the total number of elements in the tensor. For dims=[3,4]: after i=0: offset=3; after i=1: offset=12. This is equivalent to calcNumberOfElementsByShape.
6	<code>}</code>	End of first loop. offset now equals numberOfElements.
7	<code>size_t restIndex = rowIndex;</code>	Make a mutable copy of rowIndex. We will repeatedly subtract from restIndex as we peel off one axis at a time. We cannot modify rowIndex directly because it is a function parameter (by value in C — the original is unchanged by this assignment, but it is cleaner to have an explicit 'rest' variable).
8	<code>for (size_t i = 0; i < numberOfDims; i++) {</code>	Second loop: peel one axis per iteration, starting from the outermost (axis 0).
9	<code>offset /= dims[i];</code>	Divide offset by the size of the current dimension. After this division, offset equals the number of elements in all REMAINING dimensions. For dims=[3,4], i=0: offset = 12 / 3 = 4 (there are 4 elements per row). For i=1: offset = 4 / 4 = 1 (one element per column position).
10	<code>indices[i] = restIndex / offset;</code>	Integer division: how many complete 'slices' of size offset fit into restIndex? This gives the index along axis i. Example: restIndex=6, offset=4 → 6/4=1 → row=1. Integer division automatically truncates the fractional part, which is what we want.
11	<code>restIndex -= indices[i] * offset;</code>	Subtract the contribution of axis i from restIndex. What remains is the flat index within the subspace of axes i+1, i+2, ... N-1. Example: restIndex = 6 - 1*4 = 2. In the next iteration (i=1, offset=1): indices[1] = 2/1 = 2; restIndex = 2-2=0.
12	<code>}</code>	End of peeling loop. After the loop: indices = [1, 2] for rowIndex=6, dims=[3,4]. This means: element 6 is at row 1, column 2.
13	<code>}</code>	End of function. The result is in the indices[] array the caller passed in.

EXAMPLE

Complete trace for $\text{rawIndex}=9$, $\text{dims}=[2,3,4]$ (a $2 \times 3 \times 4$ tensor, 24 elements total): First loop: $\text{offset} = 1 \times 2 \times 3 \times 4 = 24$ $\text{restIndex} = 9$ $i=0$: $\text{offset} = 24/2 = 12$; $\text{indices}[0] = 9/12 = 0$; $\text{restIndex} = 9 - 0 \times 12 = 9$ $i=1$: $\text{offset} = 12/3 = 4$; $\text{indices}[1] = 9/4 = 2$; $\text{restIndex} = 9 - 2 \times 4 = 1$ $i=2$: $\text{offset} = 4/4 = 1$; $\text{indices}[2] = 1/1 = 1$; $\text{restIndex} = 1 - 1 \times 1 = 0$ Result: $\text{indices} = [0, 2, 1]$ (depth=0, row=2, col=1) Verification: $0 \times 12 + 2 \times 4 + 1 \times 1 = 0 + 8 + 1 = 9$ ✓

14.2 calcTensorIndexByIndices — Converting Per-Axis Indices to a Flat Index

```
size_t calcTensorIndexByIndices(size_t numberOfDimensions,
size_t *dimensions, size_t *indices) {
size_t index = indices[numberOfDimensions - 1];
size_t offset = dimensions[numberOfDimensions - 1];
for (int i = (int)numberOfDimensions - 2; i >= 0; i--) {
index += indices[i] * offset;
offset *= dimensions[i];
}
return index;
}
```

3	<code>size_t index = indices[numberOfDimensions - 1];</code>	Seed 'index' with the value of the LAST (innermost) dimension's index. In a 2D matrix [3×4], indices[1] is the column. The column index contributes directly to the flat position with no multiplier because adjacent columns are adjacent in memory (row-major order).
4	<code>size_t offset = dimensions[numberOfDimensions - 1];</code>	Seed 'offset' with the size of the last dimension (e.g. 4 columns). This will be the multiplier for the NEXT outer dimension. Interpretation: to advance one step in the second-to-last dimension, you skip 'numCols' elements.
5	<code>for (int i = (int)numberOfDimensions - 2; i >= 0; i--) {</code>	Work BACKWARDS from the second-to-last dimension to dimension 0. Note the cast to (int): numberOfDimensions is size_t (unsigned), but the loop must go down to -1 to detect termination — 'i >= 0' would never be false for an unsigned type (wraps to a huge positive number). The cast makes i a signed int so the >= 0 check works correctly.
6	<code>index += indices[i] * offset;</code>	Add the contribution of dimension i: how far into axis i we are, times how many elements are in all dimensions i+1, i+2, ..., N-1. For a 3×4 matrix and indices=[1,2]: i=0 → index += 1 × 4 = 4; total = 4+2 = 6.
7	<code>offset *= dimensions[i];</code>	Grow offset by the current dimension size. After this, offset represents how many elements are in dimensions i, i+1, ..., N-1. This will be the correct multiplier for the next (more outer) dimension.
9	<code>return index;</code>	Return the flat position. For indices=[1,2] in a [3×4] tensor: returns 6. This means element [row=1, col=2] is at byte offset 6 × sizeof(element).

EXAMPLE

Trace for a 2×3×4 tensor, indices=[0, 2, 1]: Start: index = indices[2] = 1; offset = dimensions[2] = 4 i=1: index += indices[1] × offset = 2 × 4 = 8 → index = 9 offset *= dimensions[1] = 4 × 3 = 12 i=0: index += indices[0] × offset = 0 × 12 = 0 → index = 9 offset *= dimensions[0] = 12 × 2 = 24 Result: flat index = 9 (matches the inverse function above ✓)

14.3 requantSymInt32Tensor — Two-Pass Dynamic Re-Quantization

This is the most complex function in TensorConversion.c. It re-scales a SYM_INT32 tensor so the full representable range is used. Below is a simplified but faithful rendering of the key steps.

```
void requantSymInt32Tensor(tensor_t *inputTensor, tensor_t *outputTensor) {
    symInt32QConfig_t *inCfg = inputTensor->quantization->qConfig;
    symInt32QConfig_t *outCfg = outputTensor->quantization->qConfig;
    float inScale = inCfg->scale;
    int32_t qMax = (int32_t)((1u << (outCfg->qMaxBits - 1)) - 1);
    int32_t qMin = -(int32_t)(1u << (outCfg->qMaxBits - 1));
    size_t n = calcNumberOfElementsByTensor(inputTensor);
    /* PASS A: find absMax */
    float absMax = 0.0f;
    for (size_t i = 0; i < n; i++) {
        int32_t m = readBytesAsInt32(&inputTensor->data[i * 4]);
        float rv = (float)m * inScale;
        if (rv < 0.0f) rv = -rv;
        if (rv > absMax) absMax = rv;
    }
    float newScale = (absMax == 0.0f) ? 1.0f : absMax / (float)qMax;
    /* PASS B: requantize */
    for (size_t i = 0; i < n; i++) {
        int32_t m = readBytesAsInt32(&inputTensor->data[i * 4]);
        float rv = (float)m * inScale;
        float q = rv / newScale;
        if (q > (float)qMax) q = (float)qMax;
        if (q < (float)qMin) q = (float)qMin;
        int32_t out = roundByMode(q, outCfg->roundingMode);
        writeInt32ToByteArray(out, &outputTensor->data[i * 4]);
    }
    outCfg->scale = newScale;
}
```

2	<code>symInt32QConfig_t *inCfg = inputTensor->quantization->qConfig;</code>	Cast the void* qConfig pointer to symInt32QConfig_t*. We know it is safe to cast here because this function is only called when both tensors have type SYM_INT32 (enforced by convertTensor or direct call). The -> operator follows the pointer chain: inputTensor → quantization field → qConfig field → then dereference.
4	<code>float inScale = inCfg->scale;</code>	Latch the INPUT scale into a local variable BEFORE the loop begins. This is critical for in-place safety: when inputTensor == outputTensor, outCfg->scale is the same memory location as inCfg->scale. Writing to outCfg->scale (line 30) at the end would overwrite it. By reading it into a local float first, we are safe.
5	<code>int32_t qMax = (int32_t)((1u << (outCfg->qMaxBits - 1)) - 1);</code>	Compute the maximum representable value. '1u << (qMaxBits-1)' shifts the unsigned integer 1 left by qMaxBits-1 positions = 2^(qMaxBits-1). Subtracting 1 gives 2^(qMaxBits-1) - 1 = 32767 for 16-bit. The cast to int32_t is needed because 1u is unsigned.
6	<code>int32_t qMin = -(int32_t)(1u << (outCfg->qMaxBits - 1));</code>	Minimum = -2^(qMaxBits-1) = -32768 for 16-bit. Note the negation AFTER the cast — negating unsigned values can cause unsigned overflow on some compilers.

11	<pre>int32_t m = readBytesAsInt32(&inputTensor->data[i * 4]);</pre>	Read element i from the input tensor. $i * 4$ is the byte offset because each <code>int32_t</code> element occupies 4 bytes. <code>readBytesAsInt32</code> uses <code>memcpy</code> internally — safe alias-free byte reassembly.
12	<pre>float rv = (float)m * inScale;</pre>	Convert from quantized mantissa to real value: $\text{real} = \text{mantissa} \times \text{scale}$. The cast <code>(float)</code> is necessary because <code>m</code> is <code>int32_t</code> — without it the multiplication might overflow or produce an integer result depending on compiler.
15	<pre>float newScale = (absMax == 0.0f) ? 1.0f : absMax / (float)qMax;</pre>	Compute the fresh scale. The ternary operator <code>?:</code> handles the degenerate case where all elements are zero — we set <code>scale=1</code> to avoid division by zero. Otherwise: $\text{scale} = \text{absMax} / \text{qMax}$ ensures that the largest real value maps to exactly <code>qMax</code> , using the full representable range.
22	<pre>if (q > (float)qMax) q = (float)qMax;</pre>	Clamp to <code>qMax</code> . By construction of <code>newScale</code> , this should never trigger unless floating-point rounding creates a value microscopically above <code>qMax</code> . The cast <code>(float)qMax</code> converts the <code>int32_t qMax</code> to float for comparison.
25	<pre>int32_t out = roundByMode(q, outCfg->roundingMode);</pre>	Round the float back to an integer using the configured rounding mode. <code>HALF_AWAY</code> rounds 0.5 to 1 and -0.5 to -1 (standard rounding). <code>SR_HALF_AWAY</code> adds uniform noise before rounding to reduce systematic bias.
29	<pre>outCfg->scale = newScale;</pre>	Write the fresh scale to the OUTPUT <code>qConfig</code> . This is done AFTER the loop, not during. Importance: if in-place (<code>inputTensor == outputTensor</code>), <code>inScale</code> was latched on line 4 before any writing, so Pass B used the correct original scale.

14.4 int32PointWiseArithmetic — The Core Element-wise Loop

```
void int32PointWiseArithmetic(tensor_t *aTensor, tensor_t *bTensor,
int32ElementArithmeticFunc_t arithmeticFunc,
tensor_t *outputTensor) {
    if (!doDimensionsMatch(aTensor, bTensor)) {
        PRINT_ERROR("Dimensions don't match!");
        exit(1);
    }
    size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);
    size_t bytesPerElement = sizeof(int32_t); // = 4
    size_t numberOfDims = aTensor->shape->numberOfDimensions;
    size_t *aDims = aTensor->shape->dimensions;
    size_t *bDims = bTensor->shape->dimensions;
    size_t aOrderedDims[numberOfDims]; // VLA – stack allocated
    size_t bOrderedDims[numberOfDims];
    orderDims(aTensor, aOrderedDims);
    orderDims(bTensor, bOrderedDims);
    for (size_t i = 0; i < numberOfElements; i++) {
        size_t aIndices[numberOfDims];
        calcIndicesByRowIndex(numberOfDims, aDims, i, aIndices);
        size_t aElementIndex = calcElementIndexByIndices(
            numberOfDims, aDims, aIndices, aTensor->shape->orderOfDimensions);
        size_t bIndices[numberOfDims];
        calcIndicesByRowIndex(numberOfDims, bDims, i, bIndices);
        size_t bElementIndex = calcElementIndexByIndices(
            numberOfDims, bDims, bIndices, bTensor->shape->orderOfDimensions);
        size_t aByteIndex = aElementIndex * bytesPerElement;
        size_t bByteIndex = bElementIndex * bytesPerElement;
        int32_t aValue = readBytesAsInt32(&aTensor->data[aByteIndex]);
        int32_t bValue = readBytesAsInt32(&bTensor->data[bByteIndex]);
        int32_t result = arithmeticFunc(aValue, bValue);
        size_t outputByteIndex = i * bytesPerElement;
        writeInt32ToByteArray(result, &outputTensor->data[outputByteIndex]);
    }
}
```

4	<pre>if (!doDimensionsMatch(aTensor, bTensor)) {</pre>	Guard check: verify both tensors have the same logical shape. The ! operator negates the boolean return: 'if NOT matching'. This catches shape mismatches early with a clear error message rather than producing silent wrong results.
8	<pre>size_t numberOfElements = calcNumberOfElementsByTensor(aTensor);</pre>	The loop will iterate this many times — once per element pair. We compute it once and store it rather than calling the function on each iteration.
10	<pre>size_t numberOfDims = aTensor->shape->numberOfDimensions;</pre>	The rank. Used to size the VLAs on the next lines and to pass to index functions.
11	<pre>size_t *aDims = aTensor->shape->dimensions;</pre>	A shortcut pointer to avoid writing <code>aTensor->shape->dimensions</code> repeatedly. This is NOT a copy — both <code>aDims</code> and <code>aTensor->shape->dimensions</code> point to the same array. Modifying <code>aDims[i]</code> would modify the tensor's shape (do not do that).

14	<code>size_t aOrderedDims[numberOfDims]; // VLA — stack allocated</code>	Variable Length Array: size determined at runtime from <code>numberOfDims</code> . Allocated on the stack — no malloc needed. Automatically freed when the function returns. Why allocate this but not use it in the loop? It IS used indirectly: <code>orderDims</code> fills it, and <code>doDimensionsMatch</code> already ran using these arrays. They serve as a normalised shape snapshot for verification.
18	<code>for (size_t i = 0; i < numberOfElements; i++) {</code>	The main loop. <code>i</code> is the LOGICAL element counter: element 0, 1, 2, ... N-1. For each <code>i</code> we decode it into per-axis indices and then find the physical byte position in both A and B (which may be transposed differently).
19	<code>size_t aIndices[numberOfDims];</code>	Another VLA — declared INSIDE the loop so it is re-initialised each iteration. Stack allocation is cheap (just adjusting the stack pointer).
20	<code>calcIndicesByRowIndex(numberOfDims, aDims, i, aIndices);</code>	Step 1: decode logical position <code>i</code> into per-axis indices for tensor A. Result: <code>aIndices</code> = [row, col] (for 2D) or [d0, d1, d2] (for 3D).
21	<code>size_t aElementIndex = calcElementIndexByIndices(</code>	Step 2: remap through A's <code>orderOfDimensions</code> to find the PHYSICAL flat index. For a non-transposed tensor this equals <code>i</code> (same as <code>calcTensorIndexByIndices</code>). For a transposed tensor the row and column are swapped first.
29	<code>size_t aByteIndex = aElementIndex * bytesPerElement;</code>	Convert element index to byte index. For <code>int32_t</code> , <code>bytesPerElement=4</code> . Element 6 is at byte offset 24 in the data buffer.
31	<code>int32_t aValue = readBytesAsInt32(&aTensor->data[aByteIndex]);</code>	Read 4 bytes starting at <code>aByteIndex</code> from A's raw data buffer and reassemble them as a signed 32-bit integer. & takes the address of <code>data[aByteIndex]</code> .
34	<code>int32_t result = arithmeticFunc(aValue, bValue);</code>	Call the operation through the function pointer. This one call handles addition, subtraction, multiplication, or any custom operation depending on which function pointer was passed when calling <code>int32PointWiseArithmetic</code> .
36	<code>size_t outputByteIndex = i * bytesPerElement;</code>	Output is always written SEQUENTIALLY: element 0, 1, 2, ... Even if A and B are transposed, the result is stored in logical order. The output tensor ends up non-transposed.

Chapter 15 — Extended Worked Examples

This chapter shows complete end-to-end traces of common operations. Reading through these traces will cement your understanding of how all the components work together.

15.1 Building and Filling a 2×3 SYM_INT32 Tensor

```
// Goal: create a 2×3 SYM_INT32 tensor with values [[1,2,3],[4,5,6]],
// scale=0.1, qMaxBits=16.
// Step 1: allocate the raw data buffer (2×3 = 6 elements × 4 bytes = 24 bytes)
uint8_t dataBuffer[24];
// Step 2: set up quantization
symInt32QConfig_t qCfg;
initSymInt32QConfig(HALF_AWAY, &qCfg); // scale=1.0, qMaxBits=16
qCfg.scale = 0.1f; // override scale to 0.1
quantization_t quant;
initSymInt32Quantization(&qCfg, &quant); // type=SYM_INT32, qConfig=&qCfg
// Step 3: set up shape
size_t dims[2] = {2, 3};
size_t orderOfDims[2];
setOrderOfDimsForNewTensor(2, orderOfDims); // orderOfDims = [0, 1]
shape_t shape;
setShape(&shape, dims, 2, orderOfDims);
// Step 4: create the tensor
tensor_t t;
setTensorValues(&t, dataBuffer, &shape, &quant, NULL);
// Step 5: write values [[1,2,3],[4,5,6]] as mantissas
int32_t values[6] = {1, 2, 3, 4, 5, 6};
for (int i = 0; i < 6; i++) {
    writeInt32ToByteArray(values[i], &t.data[i * 4]);
}
// t.data = [01 00 00 00 | 02 00 00 00 | 03 00 00 00 | ...] (little-endian)
// real values: 1×0.1=0.1, 2×0.1=0.2, ..., 6×0.1=0.6
```

15.2 Transposing and Reading the Tensor

```

// Current state: t is [2x3], orderOfDimensions=[0,1] (not transposed)
// Reading element [row=1, col=2] (should be value 6):
size_t indices[2] = {1, 2};
size_t physIdx = calcElementIndexByIndices(2, dims, indices,
t.shape->orderOfDimensions);
// orderedIndices: [1,2] (unchanged – not transposed)
// physIdx = 1x3 + 2 = 5
int32_t mantissa = readBytesAsInt32(&t.data[physIdx * 4]); // = 6
float realVal = (float)mantissa * qCfg.scale; // = 0.6
// Now transpose: swap axes 0 and 1 → logical shape becomes [3x2]
transposeTensor(&t, 0, 1);
// t.shape->orderOfDimensions now = [1, 0]
// t.shape->dimensions still = [2, 3] (unchanged)
// Reading element [row=2, col=1] in the TRANSPOSED view (same real element):
size_t indices2[2] = {2, 1};
size_t physIdx2 = calcElementIndexByIndices(2, dims, indices2,
t.shape->orderOfDimensions);
// orderOfDimensions=[1,0]:
// orderedIndices[1] = indices2[0] = 2 → orderedIndices[1] = 2
// orderedIndices[0] = indices2[1] = 1 → orderedIndices[0] = 1
// orderedIndices = [1, 2]
// physIdx2 = 1x3 + 2 = 5 ← same physical position!
int32_t mantissa2 = readBytesAsInt32(&t.data[physIdx2 * 4]); // still 6
// The transpose changed the logical view but not the physical data.

```

15.3 Adding Two 2x3 Tensors Element-wise

```

// Tensor A: [[1,2,3],[4,5,6]] as SYM_INT32, scale=0.1
// Tensor B: [[10,20,30],[40,50,60]] as SYM_INT32, scale=0.1
// Expected output: [[11,22,33],[44,55,66]]
// Assuming A and B are already set up as above:
uint8_t outData[24];
tensor_t outTensor;
setTensorValues(&outTensor, outData, &shape, &quant, NULL);
// Call the generic arithmetic engine:
int32PointWiseArithmetic(&A, &B, addInt32s, &outTensor);
// What happens internally for element i=5 (row=1, col=2):
// calcIndicesByRowIndex(2, [2,3], 5, aIndices):
// offset=6; peel: offset/=2=3; aIndices[0]=5/3=1; rest=5-3=2
// offset/=3=1; aIndices[1]=2/1=2; rest=2-2=0
// aIndices = [1, 2]
//
// calcElementIndexByIndices(2, [2,3], [1,2], [0,1]):
// orderOfDimensions=[0,1] (not transposed)
// orderedIndices = [1, 2]
// physIdx = 1x3 + 2 = 5
//
// aByteIndex = 5 × 4 = 20
// aValue = readBytesAsInt32(&A.data[20]) = 6
// bValue = readBytesAsInt32(&B.data[20]) = 60
// result = addInt32s(6, 60) = 66
// writeInt32ToByteArray(66, &outTensor.data[20])
//
// outTensor.data now contains mantissa 66 at byte offset 20.
// real value = 66 × 0.1 = 6.6 ✓

```

15.4 Converting FLOAT32 to SYM_INT32

```

// Float tensor: [1.0, -0.5, 0.25]
// Target: SYM_INT32 with qMaxBits=8 (qMax=127, qMin=-128)
// After convertFloatTensorToSymInt32Tensor:
//
// Step 1 – find absMax:
// |1.0|=1.0, |-0.5|=0.5, |0.25|=0.25 → absMax = 1.0
//
// Step 2 – compute scale:
// scale = absMax / qMax = 1.0 / 127 ≈ 0.00787
//
// Step 3 – quantize each element:
// 1.0 / 0.00787 = 127.0 → round → 127
// -0.5 / 0.00787 = -63.5 → round (HALF_AWAY) → -64
// 0.25 / 0.00787 = 31.75 → round → 32
//
// output mantissas: [127, -64, 32]
// output scale: 0.00787
//
// Verify: 127 × 0.00787 ≈ 0.9994 ≈ 1.0 ✓
// -64 × 0.00787 ≈ -0.5037 ≈ -0.5 ✓
// 32 × 0.00787 ≈ 0.2519 ≈ 0.25 ✓

```

15.5 SGD Weight Update — One Step

```

// Linear layer: weights W, gradients dW, learning rate = 0.01
// Rule: W_new = W - lr × dW
// Implemented in sgdStep → subFloat32Tensors or:
// mulFloat32ElementWithFloat32TensorInplace(&dW, learningRate, &dW);
// // dW[i] = dW[i] × 0.01 (scale the gradient)
// subFloat32TensorsInplace(&W, &dW);
// // W[i] = W[i] - dW[i] (update the weight)
// Example: W[3] = 0.5, dW[3] = 0.1, lr = 0.01
// scaled gradient = 0.1 × 0.01 = 0.001
// new weight = 0.5 - 0.001 = 0.499
// After all weight updates, zero the gradients before the next batch:
sgdZeroGrad(&optimizer);
// Calls memset(grad->data, 0, calcBytesPerTensor(grad)) for each parameter.

```


Chapter 16 — Common Mistakes and How to Avoid Them

This chapter documents the most common errors developers make when working with the ODT library, and how the library's design prevents or detects them.

Forgetting to set `orderOfDimensions`

EXPLANATION

If you call `setTensorValues` but never call `setOrderOfDimsForNewTensor`, the `orderOfDimensions` pointer is uninitialised. `calcElementIndexByIndices` will read garbage values and produce wrong indices. Always call `setOrderOfDimsForNewTensor` before `setShape`, and `setShape` before `setTensorValues`.

CORRECT CODE PATTERN

```
size_t order[2]; setOrderOfDimsForNewTensor(2, order); // DO THIS FIRST setShape(&shape, dims, 2, order);
```

Wrong byte buffer size

EXPLANATION

The data buffer must hold `numberOfElements × bytesPerElement` bytes. Use `calcNumberOfBytesForData(quantization, numberOfElements)` to compute the correct size. A buffer that is too small causes writes beyond the allocated memory (undefined behaviour).

CORRECT CODE PATTERN

```
size_t nBytes = calcNumberOfBytesForData(&quant, 6); uint8_t buf[nBytes]; // NOT: uint8_t buf[6] for int32_t data!
```

Reading a transposed tensor with the wrong index function

EXPLANATION

Do NOT use `calcTensorIndexByIndices` to read from a transposed tensor — it ignores `orderOfDimensions` and gives the PHYSICAL position without the axis swap. Always use `calcElementIndexByIndices`, which accepts `orderOfDimensions` as a parameter.

CORRECT CODE PATTERN

```
// WRONG for transposed tensor: size_t physIdx = calcTensorIndexByIndices(n, dims, indices); // CORRECT: size_t  
physIdx = calcElementIndexByIndices(n, dims, indices, orderOfDimensions);
```

Reading `inScale` after starting Pass B in requantisation

EXPLANATION

If you write to a `SYM_INT32` tensor in-place during requantisation and read `scale` after the first write, you will read the UPDATED `scale`, not the original. Always latch `inScale = inCfg->scale` before the first loop, as `requantSymInt32Tensor` does.

CORRECT CODE PATTERN

```
float inScale = inCfg->scale; // latch BEFORE any writes for (...) { /* now safe to write */ }
```

Calling `exit(1)` without `PRINT_ERROR`

EXPLANATION

Always call `PRINT_ERROR` before `exit(1)` so the developer knows why the program stopped. A bare `exit(1)` with no message is very hard to debug on a UART-only embedded system.

CORRECT CODE PATTERN

```
if (!doDimensionsMatch(a, b)) { PRINT_ERROR("shape mismatch: [%lu] vs [%lu]", na, nb); exit(1); }
```

Not zeroing gradients between batches

EXPLANATION

Calling `sgdStep` adds the current gradients to the weights. If you do not call `sgdZeroGrad` before the next batch, gradients accumulate and the model diverges. Always zero gradients at the start (or end) of each batch loop.

CORRECT CODE PATTERN

```
sgdZeroGrad(&optimizer); // zero BEFORE computing new gradients forwardPass(...); backwardPass(...);
sgdStep(&optimizer);
```

Mismatched VLA size

EXPLANATION

When declaring `size_t aIndices[numberOfDims]` inside a loop, ensure `numberOfDims` is correctly initialised before the declaration. If `numberOfDims` is 0, the VLA has size 0 — writing to it is undefined behaviour.

CORRECT CODE PATTERN

```
// Safe: read numberOfDims once before entering the loop
size_t nd = tensor->shape->numberOfDimensions;
for (...)
{ size_t indices[nd]; ... }
```

Using memcpy instead of memmove when regions overlap

EXPLANATION

When `inputTensor == outputTensor`, the source and destination memory may overlap. `memcpy` is undefined for overlapping regions; `memmove` is always safe. `convertTensorsWithSameType` uses `memmove` for this reason.

CORRECT CODE PATTERN

```
// WRONG for overlapping buffers: memcpy(output->data, input->data, n);
// CORRECT: memmove(output->data, input->data, n);
```

Chapter 17 — C Syntax Quick-Reference Table

A condensed reference for every C syntax pattern that appears in this library. Use this as a lookup when you encounter unfamiliar syntax in the source files.

Syntax Pattern	Meaning	Example from the library
type *name	Pointer to type	tensor_t *tensor — pointer to a tensor_t struct
name->field	Access field of struct through pointer	tensor->shape->numberOfDimensions
(*name).field	Same as name->field	(*tensor).quantization — equivalent to tensor->quantization
&variable	Address-of: get a pointer to variable	&tensor->data[byteIndex]
*pointer	Dereference: read value at pointer address	*((int32_t*)bytes) — unsafe; use memcpy instead
sizeof(type)	Compile-time size of type in bytes	sizeof(int32_t) = 4
(type)value	Cast: convert value to type	(float)mantissa * scale
typedef T name	Create type alias: 'name' is same as T	typedef uint8_t byte_t — byte_t is now an alias for uint8_t
typedef struct { } T	Define struct AND create alias	typedef struct tensor { ... } tensor_t
typedef enum { } T	Define enum AND create alias	typedef enum qtype { INT32, FLOAT32, ... } qtype_t
typedef R (*T)(P)	Function-pointer type: T points to function R(P)	typedef int32_t (*int32ElementArithmeticFunc_t)(int32_t, int32_t)
union { ... }	One memory location shared by all members	typedef union layerConfig { linearConfig_t *linear; ... } layerConfig_t
type arr[N]	Fixed-size array	size_t dims[3] — array of 3 size_t values
type arr[n]	VLA: size determined at runtime	size_t indices[numberOfDims] — stack array, size known at runtime
extern T name	Declare a variable defined in another file	extern conversionFunction_t conversionMatrix[6][6]
static void fn()	File-private function (not callable from other .c files)	static void zeroTensorData(tensor_t *tensor)
1u << n	Bit shift: compute 2^n as unsigned int	1u << (qMaxBits-1) = 2^(qMaxBits-1)
x ? a : b	Ternary: if x is true return a, else return b	absMax == 0.0f ? 1.0f : absMax / qMax
a &= mask	Bitwise AND-assign: clear bits	byte &= ~getBitmask(start, end) — clear target bit range
a = val	Bitwise OR-assign: set bits	byte = (data << startbit) — set target bit range
#ifdef / #endif	Conditional compilation	#ifdef DEBUG_MODE_DEBUG → DLEVEL=3
_Static_assert(c,m)	Compile-time assertion	_Static_assert(BOOL+1==6, "extend matrix")
do { } while(false)	Single-statement macro wrapper	#define PRINT_ERROR(s,...) do { ... } while(false)
##_VA_ARGS__	Variadic macro arguments	printf(str, ##_VA_ARGS__) — passes 0+ extra args to printf
int i = (int)n-2; i>=0; i--	Signed loop going to -1 (requires cast from size_t)	for (int i=(int)nDims-2; i>=0; i--) — backward loop in calcTensorIndexByIndices

Chapter 18 — Data Flow in a Training Step

This chapter traces what happens to the DATA (the bytes inside tensors) during one complete training step: forward pass → loss → backward → weight update.

```
■■■■■■■■■■■■■■■■■■■■ input tensor (FLOAT32 or SYM_INT32)
■ Input Data ■ bytes: [e0, e1, e2, ..., eN]
■■■■■■■■■■■■■■■■■■■■
■ readBytesAsFloat / readBytesAsInt32 [DTypes.c]
▼
■■■■■■■■■■■■■■■■■■■■ Linear Layer Forward:
■ Matmul ■ output[j] =  $\sum$  input[i] × weight[j][i] + bias[j]
■ + Bias ■ → matmulSymInt32TensorsWithBias [Matmul.c]
■■■■■■■■■■■■■■■■■■■■ → writeInt32ToByteArray [DTypes.c]
■
▼
■■■■■■■■■■■■■■■■■■■■ Requantize activation to fresh scale:
■ requantSym ■ Pass A: find absMax; compute newScale
■ Int32Tensor ■ Pass B: mantissa = round(real / newScale)
■■■■■■■■■■■■■■■■■■■■ [TensorConversion.c]
■
▼
■■■■■■■■■■■■■■■■■■■■ ReLU: output[i] = max(0, input[i])
■ Relu ■ → reluForwardSymInt32 or reluForwardFloat
■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■ Softmax: output[i] =  $\exp(x[i]) / \sum \exp(x[j])$ 
■ Softmax ■ → softmaxForward [Softmax.c]
■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■ CrossEntropy: loss =  $-\sum \text{label}[i] \times \log(\text{output}[i])$ 
■ Loss Forward ■ → lossFunctions[CROSS_ENTROPY].forward
■■■■■■■■■■■■■■■■■■■■ [CrossEntropy.c, Log.c]
■ gradient = output - label (for softmax+CE combined)
▼
■■■■■■■■■■■■■■■■■■■■ Backward through each layer (reverse order):
■ Backward ■ propLoss = gradient × weightsT [Linear.c]
■ Passes ■ weightGrad = forwardInputT × gradient
■■■■■■■■■■■■■■■■■■■■
■
▼
■■■■■■■■■■■■■■■■■■■■ SGD update: weight -= lr × weightGrad
■ sgdStep ■ → mulFloat32ElementWithFloat32TensorInplace [Mul.c]
■ ■ → subFloat32TensorsInplace [Sub.c]
■■■■■■■■■■■■■■■■■■■■
```

Every box in the diagram corresponds to one or more function calls. Every arrow represents bytes moving through readBytesAs* and writeXxxToByteArray. The tensor_t struct travels through the entire chain — only its data bytes change at each step; the struct itself (shape, quantization pointers) stays the same.

Chapter 19 — Glossary of Terms

Definitions of every technical term used in this document.

Term	Definition
absMax	The absolute maximum value in a set — $\max$ of $ x $ over all elements x . Used in <code>requantSymInt32Tensor</code> to determine the scale factor.
Arithmetic engine	The generic element-wise loop in <code>Arithmetic.c</code> . Takes a function pointer and iterates over all elements, calling the function on each pair.
Asymmetric quantization	Quantization where the integer range $[0, 2^n-1]$ is shifted by a <code>zeroPoint</code> : $\text{real} = (\text{mantissa} - \text{zeroPoint}) \times \text{scale}$. Allows representing non-zero-centred distributions.
Backward pass	Computing gradients. Starting from the loss, derivatives are propagated backwards through each layer using the chain rule of calculus.
Batch	A collection of training samples processed together. Gradients are averaged over the batch before updating weights.
Byte buffer	A contiguous array of <code>uint8_t</code> (unsigned 8-bit bytes). Used as the raw storage format for all tensor data.
Cast	In C, explicitly converting a value from one type to another: <code>(float)intValue</code> . Important for preventing integer overflow in arithmetic.
Clamp	Restrict a value to a range $[\text{min}, \text{max}]$. Values below <code>min</code> become <code>min</code> ; values above <code>max</code> become <code>max</code> .
Convolutional layer (Conv1d)	Applies a sliding filter of fixed size K to a 1D input sequence to extract local patterns (e.g. audio features).
Cross-entropy loss	Loss for classification: $-\sum \text{label}[i] \times \log(\text{prediction}[i])$. Measures how different the predicted probability distribution is from the true distribution.
Dense layer	See <code>Linear layer</code> .
Dispatch table	An array of function pointers indexed by a type enum. Avoids switch-case chains — looking up the right function is $O(1)$.
Dropout	Randomly zeroes out a fraction (p) of activation values during training. Prevents overfitting by forcing the network to not rely on any single neuron.
Element-wise	An operation applied independently to each pair of corresponding elements: $\text{result}[i] = \text{op}(\text{A}[i], \text{B}[i])$. Also called point-wise.
Epoch	One complete pass through the training dataset.
Flat index	A single integer identifying an element in a multi-dimensional array by counting elements sequentially in row-major order.
Forward pass	Computing the model output from an input. Each layer transforms its input to produce its output.
Function pointer	A pointer that holds the address of a function. Allows passing functions as parameters to other functions.
Gradient	The derivative of the loss with respect to a weight — how much the loss would change if that weight were changed by a small amount.
Guard check	A validation at the start of a function that exits with an error if a precondition is not met.
Header guard	<code>#ifndef / #define / #endif</code> wrapper that prevents a <code>.h</code> file from being included more than once in the same translation unit.
In-place	An operation where the result overwrites one of the input tensors, saving the memory cost of an additional output buffer.
Layer	One transformation stage in a neural network (<code>Linear</code> , <code>Relu</code> , <code>Conv1d</code> , <code>Softmax</code> , etc.).
LayerNorm	Layer Normalisation: normalises activations across the feature dimension for stable training of deep networks.
Learning rate	The step size used by the optimiser: $\text{weight} -= \text{lr} \times \text{gradient}$. Too large $\rightarrow$ training diverges; too small $\rightarrow$ training is slow.
Linear layer	Computes $\text{output} = \text{input} \times \text{weights}^T + \text{bias}$. Also called a fully-connected or dense layer.
Little-endian	Byte ordering where the least significant byte is stored first. The STM32 ARM processor (and most modern hardware) is little-endian.
Logical index / Logical axis	The axis numbering as the user sees it (e.g. axis 0 = rows). After transpose, logical axis 0 may correspond to physical axis 1.
Mantissa	The integer part of a <code>SYM_INT32</code> quantized value. Real value = $\text{mantissa} \times \text{scale}$.

Matrix multiply (Matmul)	$C[i][j] = \sum_k A[i][k] \times B[k][j]$. The core operation of a Linear layer.
memcpy	C standard library function that copies n bytes from src to dst. Required for safe byte-to-typed-value conversion.
memmove	Like memcpy but safe when source and destination overlap.
memset	Fills n bytes with a constant value. Used to zero gradient buffers.
MSE loss	Mean Squared Error: $\sum (\text{prediction}[i] - \text{label}[i])^2 / N$. Used for regression tasks.
Optimizer	Algorithm that updates model weights using computed gradients. SGD is the simplest optimizer.
orderOfDimensions	Array in shape_t that maps logical axis i to physical slot j. Enables zero-cost transpose.
Physical index / Physical axis	The actual position in the data buffer, as determined by the orderOfDimensions mapping.
Pointer arithmetic	Adding an integer to a pointer advances it by that many elements: data + 4 points to the 5th byte.
PointWise	See element-wise.
qConfig	void* field in quantization_t pointing to the scheme-specific config struct.
qMax / qMin	The maximum and minimum representable integer values for a given bit depth: qMax = $2^{(qMaxBits-1)}-1$, qMin = $-2^{(qMaxBits-1)}$.
qMaxBits	Bit depth controlling the quantization range (e.g. 16 $\rightarrow$ range ± 32767).
qtype_t	Enum with six values: INT32, FLOAT32, SYM_INT32, SYM, ASYM, BOOL.
Quantization	Representing floating-point values as integers with a scale factor, reducing memory and compute at the cost of some precision.
Rank	Number of dimensions of a tensor (also called numberOfDimensions). Scalar=0, vector=1, matrix=2, 3D tensor=3.
ReLU	Rectified Linear Unit: output = max(0, input). The standard non-linearity.
Requantization	Re-scaling a SYM_INT32 tensor to a new scale, either dynamically (two-pass) or to a fixed target scale.
Row-major order	Memory layout where adjacent elements of the last dimension are stored contiguously. C arrays use row-major order.
scale	The factor mapping between integer mantissa and real value: real = mantissa $\times$ scale. A smaller scale allows higher precision but smaller range.
SGD	Stochastic Gradient Descent: weight -= learningRate $\times$ gradient. The simplest and most widely used optimizer.
shape_t	Struct holding the rank and dimension sizes of a tensor, plus orderOfDimensions.
Softmax	Converts raw scores to probabilities: $\text{output}[i] = \exp(x[i]) / \sum \exp(x[j])$.
Sparsity	When most tensor values are zero. The sparsity_t struct describes the sparsity pattern. Dense tensors have sparsity=NULL.
static (function)	C keyword making a function visible only within its .c file. Equivalent to 'private' in object-oriented languages.
Stochastic rounding (SR_HALF_AWAY)	Rounding mode that adds uniform noise in [-0.5,0.5) before rounding. Reduces systematic quantization bias during training.
symInt32QConfig_t	Config struct for SYM_INT32: holds scale, roundingMode, qMaxBits.
symmetric quantization	Quantization centred at zero: real = mantissa $\times$ scale. The integer range is symmetric: [-qMax, qMax].
tensor_t	The core data structure: holds data buffer, shape, quantization, and sparsity.
Transpose	Swapping axes of a tensor. In this library: zero-cost, done by modifying orderOfDimensions only, without moving any data.
Translation unit	One .c file plus all the headers it includes, compiled together. Each .c file is one translation unit.
uint8_t	Unsigned 8-bit integer (0–255). Used as the universal raw storage type for tensor data buffers.
VLA	Variable Length Array: a stack-allocated array whose size is a runtime variable (C99 feature). No malloc/free needed; freed automatically on function return.
void*	A generic pointer that can point to any type. Used in quantization_t.qConfig to hold any of the three config struct types.
zeroPoint	In asymmetric quantization: the integer value that maps to real value 0. real = (mantissa - zeroPoint) $\times$ scale.

Chapter 20 — Line-by-Line: linearForward

linearForward is the core computation of the Linear (fully-connected) layer: it computes $\text{output} = \text{matmul}(\text{input}, \text{weights}^T) + \text{bias}$. This chapter walks through every line of the forward pass implementation.

```
void linearForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    linearConfig_t *config = layer->config.linear;
    tensor_t *weights = config->weights;
    tensor_t *bias = config->bias;
    /* Step 1: transpose weights to shape [outFeatures, inFeatures]^T */
    /* We need weights in shape [inFeatures, outFeatures] for matmul */
    transposeTensor(weights, 0, 1);
    /* Step 2: matmul output[batch, outF] =  $\sum_k \text{input}[\text{batch}, k] \times \text{weights}^T[k, \text{outF}]$  */
    if (weights->quantization->qType == FLOAT32) {
        matmulFloat32TensorsWithBias(input, weights, bias, output);
    } else {
        matmulSymInt32TensorsWithBias(input, weights, bias, output);
    }
    /* Step 3: un-transpose weights back to original shape */
    transposeTensor(weights, 0, 1);
    /* Step 4: re-quantize output if quantized */
    if (output->quantization->qType == SYM_INT32) {
        tensor_t *preActBuffer = config->preActivationBuffer;
        copyTensor(output, preActBuffer);
        requantSymInt32Tensor(preActBuffer, output);
    }
}
```

1	<code>void linearForward(layer_t *layer, tensor_t *input, tensor_t *output) {</code>	The function signature. Takes the generic <code>layer_t*</code> (contains config and weights), the input tensor (activations from the previous layer), and writes its result into the pre-allocated output tensor. Returning void keeps the API consistent across all layers.
2	<code>linearConfig_t *config = layer->config.linear;</code>	Extract the layer-specific config from the generic union field. <code>layer->config</code> is a <code>layerConfig_t</code> union; accessing <code>.linear</code> gives a <code>linearConfig_t*</code> stored as one member of the union. This is safe only when <code>layer->layerType == LINEAR</code> .
3	<code>tensor_t *weights = config->weights;</code>	Short alias for the weight tensor. <code>weights->shape = [outFeatures, inFeatures]</code> in storage (because that is how they were initialised and serialised). The matmul expects weights transposed, so we flip them in the next step.
8	<code>transposeTensor(weights, 0, 1);</code>	Zero-cost transpose: swap axes 0 and 1 by exchanging <code>orderOfDimensions</code> entries. After this, logical axis 0 = original axis 1 (inFeatures), logical axis 1 = original axis 0 (outFeatures). No bytes are moved — only the <code>orderOfDimensions</code> metadata is swapped.
11	<code>if (weights->quantization->qType == FLOAT32) {</code>	Dispatch to the correct matmul based on the weight quantization type. <code>FLOAT32</code> weights use floating-point matmul (faster but more memory). <code>SYM_INT32</code> weights use integer matmul (slower but can run on hardware without FPU). The <code>qType</code> field is set when the layer is initialised.
12	<code>matmulFloat32TensorsWithBias(input, weights, bias, output);</code>	Calls the floating-point matrix multiply + bias add. Computes: $\text{output}[b, j] = \sum_k \text{input}[b, k] \times \text{weights}[k, j] + \text{bias}[j]$ for every batch item <code>b</code> and output feature <code>j</code> .

14	<code>matmulSymInt32TensorsWithBias(input, weights, bias, output);</code>	Same computation but using integer arithmetic on SYM_INT32 tensors. Each element multiply is $\text{int32} \times \text{int32} \rightarrow \text{int64}$ (widening), then accumulated and scaled back. The bias is added as an integer after scaling.
18	<code>transposeTensor(weights, 0, 1);</code>	Un-transpose: restore the weights to their original shape [outFeatures, inFeatures]. This must be done AFTER the matmul. Because transposeTensor only swaps metadata, calling it twice returns to the original state. Failing to un-transpose would corrupt subsequent uses of these weights (e.g. the backward pass, or saving to flash).
21	<code>if (output->quantization->qType == SYM_INT32) {</code>	If the output is quantized, re-quantize it to a fresh scale. The matmul accumulates into a large dynamic range — values can be much larger than the input. Re-quantising compresses the range back.
22	<code>tensor_t *preActBuffer = config->preActivationBuffer;</code>	A scratch tensor allocated when the layer was created. It has the same shape as output. We copy the raw matmul result here so requantSymInt32Tensor can read from it while writing to output.
23	<code>copyTensor(output, preActBuffer);</code>	Copy the raw output into the scratch buffer. This is necessary because requantSymInt32Tensor's Pass A reads ALL elements to find absMax — if it were reading from the same buffer it is writing to, Pass B could corrupt elements before Pass A has seen them all.
24	<code>requantSymInt32Tensor(preActBuffer, output);</code>	Two-pass re-quantisation: finds the best scale for the current data and re-encodes the output tensor with that scale. The result has the same values but compressed into the representable integer range.

Chapter 21 — Line-by-Line: ReLU Forward and Backward

ReLU is the simplest non-linearity: it passes positive values unchanged and clamps negative values to zero. Despite its simplicity, its backward pass introduces a subtle mask — the backward gradient only flows where the forward input was positive.

21.1 reluForwardFloat32 — Forward Pass

```
void reluForwardFloat32(tensor_t *input, tensor_t *output) {
    size_t n = calcNumberOfElementsByTensor(input);
    for (size_t i = 0; i < n; i++) {
        size_t byteIdx = i * sizeof(float);
        float val = readBytesAsFloat(&input->data[byteIdx]);
        if (val < 0.0f) val = 0.0f;
        writeFloatToByteArray(val, &output->data[byteIdx]);
    }
}
```

5	float val = readBytesAsFloat(&input->data[byteIdx]);	Read one float element from the input tensor's raw byte buffer. sizeof(float) == 4, so byteIdx = i*4. readBytesAsFloat uses memcpy to reassemble 4 bytes into a float (safe strict-aliasing).
6	if (val < 0.0f) val = 0.0f;	The entire ReLU computation in one line: clip negative values to zero. Positive values (and exactly zero) pass through unchanged. This is equivalent to max(0, val) but avoids a function call.
7	writeFloatToByteArray(val, &output->data[byteIdx]);	Write the result back to the output tensor. output may be the same tensor as input (in-place), which is fine because the read and write access the same element.

21.2 reluBackwardFloat32 — Backward Pass

```
void reluBackwardFloat32(tensor_t *forwardInput, tensor_t *propagatedLoss,
    tensor_t *outputGrad) {
    size_t n = calcNumberOfElementsByTensor(forwardInput);
    for (size_t i = 0; i < n; i++) {
        size_t byteIdx = i * sizeof(float);
        float fwdVal = readBytesAsFloat(&forwardInput->data[byteIdx]);
        float lossVal = readBytesAsFloat(&propagatedLoss->data[byteIdx]);
        float grad = (fwdVal > 0.0f) ? lossVal : 0.0f;
        writeFloatToByteArray(grad, &outputGrad->data[byteIdx]);
    }
}
```

6	float fwdVal = readBytesAsFloat(&forwardInput->data[byteIdx]);	Read the value from the FORWARD PASS input — the activation that entered ReLU during forward. This is stored in a buffer saved by the layer during forward. It is needed here to reconstruct the ReLU mask: where was the input > 0?
7	float lossVal = readBytesAsFloat(&propagatedLoss->data[byteIdx]);	Read the incoming gradient (the loss gradient propagated from the NEXT layer). The chain rule says: $d_loss/d_x = d_loss/d_y \times d_y/d_x$. lossVal is d_loss/d_y . We multiply by the ReLU derivative d_y/d_x on the next line.

8	<pre>float grad = (fwdVal > 0.0f) ? lossVal : 0.0f;</pre>	The ReLU derivative is: 1 when fwdVal > 0, else 0. So: grad = lossVal × 1 = lossVal (when fwdVal > 0) grad = lossVal × 0 = 0 (when fwdVal ≤ 0). The ternary operator ? : implements this branch in one line. This is called the 'gradient mask' — gradients are blocked wherever the forward input was non-positive. Dead neurons (always ≤ 0) get zero gradient forever (the 'dying ReLU' problem).
---	--	---

EXAMPLE

Example backward trace for a 3-element vector: Forward input: [-0.3, 0.5, -1.2] Forward output (ReLU): [0.0, 0.5, 0.0] Propagated loss gradient from next layer: [0.4, 0.2, 0.7] Backward computation: i=0: fwdVal=-0.3 (≤0) → grad = 0.0 i=1: fwdVal=+0.5 (>0) → grad = 0.2 (gradient passes through) i=2: fwdVal=-1.2 (≤0) → grad = 0.0 outputGrad = [0.0, 0.2, 0.0] Interpretation: only neuron 1 (which was active during forward) contributes to the weight gradients in the previous layer.

Chapter 22 — The Backward Pass: Theory and Implementation

This chapter explains the mathematics behind backpropagation and shows exactly how the ODT library implements it. You do not need calculus beyond the chain rule.

22.1 The Chain Rule

Suppose you have a network: Input $\rightarrow$ Layer A $\rightarrow$ Layer B $\rightarrow$ Loss. The chain rule says: $d_loss/d_A_output = d_loss/d_B_output \times d_B_output/d_A_output$. In words: the gradient of the loss with respect to A's output equals the gradient of the loss with respect to B's output (which we already have) times how much A's output affects B's output (the local derivative).

```
Backward pass order:
Layer B backward:
input: propagatedLoss (= d_loss/d_B_output) – from loss function
output: propLoss_for_A (= d_loss/d_A_output) – sent to layer A
side-effect: weightGrad_B += input^T × propagatedLoss
Layer A backward:
input: propLoss_for_A
output: propLoss_for_input – sent to previous layer
side-effect: weightGrad_A += input^T × propLoss_for_A
Optimizer step:
weights_A -= lr × weightGrad_A
weights_B -= lr × weightGrad_B
```

22.2 Linear Layer Backward — Mathematics

For a linear layer: $output = input \times weights^T + bias$. Given d_loss/d_output (the incoming gradient), we need to compute: (1) d_loss/d_input , to continue back-propagating; (2) $d_loss/d_weights$, to update the weights; and (3) d_loss/d_bias , to update the bias.

```
Mathematical derivation:
Let L = loss, Y = output, X = input (batch×inF), W = weights (outF×inF)
(1)  $dL/dX = (dL/dY) \times W$ 
Shape: (batch×outF) × (outF×inF) = (batch×inF) ← propagated gradient
(2)  $dL/dW = (dL/dY)^T \times X$ 
Shape: (outF×batch) × (batch×inF) = (outF×inF) ← weight gradient
(3)  $dL/d\_bias[j] = \sum_b dL/dY[b,j]$  (sum over batch)
Shape: (outF,) ← bias gradient
```

22.3 Implementation in linearBackward

```

void linearBackward(layer_t *layer, tensor_t *propLoss, tensor_t *outputPropLoss) {
    linearConfig_t *config = layer->config.linear;
    tensor_t *weights = config->weights;
    tensor_t *weightGrad = config->weightGradient;
    tensor_t *biasGrad = config->biasGradient;
    tensor_t *savedInput = config->savedInput;
    /* (1) propagate loss backward through weights: propLoss × W */
    matmulFloat32TensorsWithBias(propLoss, weights, NULL, outputPropLoss);
    /* (2) compute weight gradient: savedInput^T × propLoss */
    transposeTensor(savedInput, 0, 1);
    matmulFloat32TensorsWithBias(savedInput, propLoss, NULL, weightGrad);
    transposeTensor(savedInput, 0, 1); // restore
    /* (3) compute bias gradient: sum propLoss over batch dimension */
    sumFloat(propLoss, 0, biasGrad);
}

```

6	tensor_t *savedInput = config->savedInput;	The forward pass saved a copy of its INPUT before computing the output. The backward pass needs this to compute the weight gradient (step 2). Without saving it, the backward pass would have no access to the original input.
9	matmulFloat32TensorsWithBias(propLoss, weights, NULL, outputPropLoss);	Compute $d_L/d_{input} = propLoss \times W$. Passing NULL for bias means no bias is added in this matmul. The result is the gradient to propagate to the previous layer.
12	transposeTensor(savedInput, 0, 1);	Transpose the saved input from [batch×inF] to [inF×batch] so the matmul $d_L/d_W = savedInput^T \times propLoss$ has compatible shapes: $(inF \times batch) \times (batch \times outF) = (inF \times outF)$, then transpose to get $(outF \times inF)$.
13	matmulFloat32TensorsWithBias(savedInput, propLoss, NULL, weightGrad);	Compute the weight gradient. The result is ADDED to (accumulated into) weightGrad, not assigned. This allows gradient accumulation over multiple batches or microbatches before an optimizer step.
17	sumFloat(propLoss, 0, biasGrad);	Compute d_L/d_{bias} by summing propLoss over dimension 0 (the batch dimension). Each output feature j gets gradient = $\sum_b propLoss[b,j]$.

Chapter 23 — Understanding the Optimizer: SGD and SGD with Momentum

23.1 Vanilla SGD

Stochastic Gradient Descent is the simplest optimizer. It moves each weight in the direction that reduces the loss, by a step proportional to the gradient.

```
/* SGD update rule: */
weight[i] = weight[i] - learningRate × gradient[i]
/* Implemented as two operations: */
/* Step 1: scale gradient in place */
mulFloat32ElementWithFloat32TensorInplace(&gradient, learningRate, &gradient);
/* gradient[i] = gradient[i] × learningRate for all i */
/* Step 2: subtract from weights */
subFloat32TensorsInplace(&weights, &gradient);
/* weights[i] = weights[i] - gradient[i] for all i */
```

The learning rate is a critical hyperparameter. If it is too large, the optimizer overshoots the minimum and training diverges. If it is too small, training is slow. A typical value for classification is between 0.001 and 0.1.

23.2 SGD with Momentum

SGD with momentum adds a 'velocity' term that accumulates gradients over time. This helps the optimizer build up speed in consistent directions and dampen oscillations.

```
/* SGD with momentum update rule: */
velocity[i] = momentum × velocity[i] + learningRate × gradient[i]
weight[i] = weight[i] - velocity[i]
/* In code (sgdStepM): */
/* Step 1: scale velocity by momentum factor (e.g. 0.9) */
mulFloat32ElementWithFloat32TensorInplace(&velocity, momentum, &velocity);
/* Step 2: scale gradient by learning rate */
mulFloat32ElementWithFloat32TensorInplace(&gradient, learningRate, &tempGrad);
/* Step 3: add scaled gradient to velocity */
addFloat32TensorsInplace(&velocity, &tempGrad);
/* Step 4: subtract velocity from weights */
subFloat32TensorsInplace(&weights, &velocity);
```

EXAMPLE

Momentum example: weight=-0.5, gradient=0.3, lr=0.1, momentum=0.9 Iteration 1 (velocity starts at 0): velocity = $0.9 \times 0 + 0.1 \times 0.3 = 0.03$ weight = $-0.5 - 0.03 = -0.53$ Iteration 2 (same gradient): velocity = $0.9 \times 0.03 + 0.1 \times 0.3 = 0.027 + 0.03 = 0.057$ weight = $-0.53 - 0.057 = -0.587$ Iteration 3: velocity = $0.9 \times 0.057 + 0.03 = 0.0513 + 0.03 = 0.0813$ weight = $-0.587 - 0.0813 = -0.668$ Without momentum: after 3 iterations weight = $-0.5 - 3 \times 0.03 = -0.59$ With momentum: after 3 iterations weight = -0.668 (moves faster in consistent direction)

23.3 Zero-Gradient After Each Step

Gradients accumulate in the weightGradient and biasGradient tensors during backprop. After calling sgdStep (or sgdStepM), these buffers must be zeroed before the next batch. sgdZeroGrad handles this — it calls `memset(grad->data, 0, bytes)` for every parameter.

```
/* Typical training loop: */
for (int batch = 0; batch < numBatches; batch++) {
/* Always zero gradients at the start of each batch: */
sgdZeroGrad(&optimizer);
/* Forward pass: compute predictions */
forwardPass(&model, &batchInput, &predictions);
/* Loss: compute scalar loss and loss gradient */
lossFunctions[CROSS_ENTROPY].forward(&predictions, &labels, &lossVal);
lossFunctions[CROSS_ENTROPY].backward(&predictions, &labels, &lossGrad);
/* Backward pass: propagate gradients through each layer */
backwardPass(&model, &lossGrad, &propLoss);
/* Optimizer step: update weights using accumulated gradients */
sgdStep(&optimizer); /* or sgdStepM for momentum */
}
```

Chapter 24 — How to Write a New Layer

This chapter explains the design pattern for adding a new layer type to the ODT library. Every layer follows the same pattern: a config struct, a forward function, a backward function, and an init function.

24.1 Step 1: Define the Config Struct

```
/* In MyLayer.h */
#ifndef MY_LAYER_H
#define MY_LAYER_H
#include "Layer.h"
#include "Tensor.h"
/* Config holds all learnable parameters and buffers for this layer. */
typedef struct myLayerConfig {
    tensor_t *weights; /* learnable: [outputSize, inputSize] */
    tensor_t *weightGradient; /* gradient buffer for weights */
    tensor_t *savedInput; /* saved forward input for backward pass */
    float myHyperParam; /* any layer-specific constant */
} myLayerConfig_t;
/* Declare the forward and backward functions: */
void myLayerForward(layer_t *layer, tensor_t *input, tensor_t *output);
void myLayerBackward(layer_t *layer, tensor_t *propLoss, tensor_t *outPropLoss);
/* Init function: allocates config and sets function pointers in layer: */
void initMyLayer(layer_t *layer, float hyperParam);
#endif /* MY_LAYER_H */
```

24.2 Step 2: Implement Forward

```
/* In MyLayer.c */
#include "MyLayer.h"
#include "Matmul.h"
void myLayerForward(layer_t *layer, tensor_t *input, tensor_t *output) {
    myLayerConfig_t *cfg = layer->config.custom; /* custom union field */
    /* IMPORTANT: save input for backward pass */
    copyTensor(input, cfg->savedInput);
    /* Compute the forward transformation */
    matmulFloat32TensorsWithBias(input, cfg->weights, NULL, output);
    /* Apply any element-wise activation here (e.g. tanh, sigmoid) */
    for (size_t i = 0; i < calcNumberOfElementsByTensor(output); i++) {
        float v = readBytesAsFloat(&output->data[i*4]);
        v = v * cfg->myHyperParam; /* example: scale by hyperParam */
        writeFloatToByteArray(v, &output->data[i*4]);
    }
}
```

24.3 Step 3: Implement Backward

```

void myLayerBackward(layer_t *layer, tensor_t *propLoss, tensor_t *outPropLoss) {
    myLayerConfig_t *cfg = layer->config.custom;
    /* Step A: compute gradient through activation (e.g. scale derivative) */
    /*  $d_L/d_{pre\_activation} = propLoss \times hyperParam$  */
    mulFloat32ElementWithFloat32TensorInplace(propLoss, cfg->myHyperParam, propLoss);
    /* Step B: propagate gradient through weights to previous layer */
    matmulFloat32TensorsWithBias(propLoss, cfg->weights, NULL, outPropLoss);
    /* Step C: accumulate weight gradient */
    transposeTensor(cfg->savedInput, 0, 1);
    addFloat32TensorsInplace(cfg->weightGradient,
        matmulResult(cfg->savedInput, propLoss));
    transposeTensor(cfg->savedInput, 0, 1);
}

```

24.4 Step 4: Init and Register

```

void initMyLayer(layer_t *layer, float hyperParam) {
    /* Allocate config on the heap or in a static pool */
    myLayerConfig_t *cfg = malloc(sizeof(myLayerConfig_t));
    cfg->myHyperParam = hyperParam;
    /* allocate weights, gradients, buffers ... */
    /* Wire up the layer struct */
    initLayer(layer, MY_LAYER_TYPE);
    layer->config.custom = cfg;
    layer->forward = myLayerForward;
    layer->backward = myLayerBackward;
}

/* In Layer.h: add MY_LAYER_TYPE to layerType_t enum */
/* In layerConfig_t union: add myLayerConfig_t *custom field */

```


Chapter 25 — Random Number Generation: The Xorshift Algorithm

The RNG module uses a xorshift32 algorithm — a fast, compact pseudo-random number generator suitable for embedded systems. It has no heap allocation, requires only 4 bytes of state, and is reproducible (same seed → same sequence).

25.1 Xorshift32 Explained

```
uint32_t xorshift32(uint32_t *state) {
    uint32_t x = *state;
    x ^= x << 13; /* XOR with left-shifted copy */
    x ^= x >> 17; /* XOR with right-shifted copy */
    x ^= x << 5; /* XOR with left-shifted copy */
    *state = x;
    return x;
}
```

2	<code>uint32_t x = *state;</code>	Read the current state. The state is the entire random number generator — 4 bytes. This makes the RNG extremely lightweight.
3	<code>x ^= x << 13;</code>	XOR-assign: $x = x \oplus (x \text{ shifted left by } 13)$. The bit shift creates a copy of x with all bits moved 13 positions left (zeros fill from the right). XOR mixes the shifted copy back into x . This creates non-linear dependencies between bits.
4	<code>x ^= x >> 17;</code>	Right-shift by 17, then XOR. Together with the left shifts, this ensures every bit of the output depends on every bit of the input in a complex pattern. The specific constants (13, 17, 5) are chosen for maximum-length period ($2^{32} - 1$ unique values before repeating).
6	<code>*state = x;</code>	Update the state with the new value. The output IS the new state — xorshift is a full-period generator.

25.2 Converting to Float and Shuffling

```
float rngNextFloat(rng32_t *rng) {
    uint32_t raw = xorshift32(&rng->state);
    /* Map [0, 2^32-1] to [0, 1): divide by 2^32 */
    return (float)raw / 4294967296.0f; /* 4294967296 = 2^32 */
}

void rngShuffleIndices(rng32_t *rng, size_t *indices, size_t n) {
    /* Fisher-Yates shuffle: O(n), unbiased */
    for (size_t i = n - 1; i > 0; i--) {
        size_t j = (size_t)(rngNextFloat(rng) * (float)(i + 1));
        /* swap indices[i] and indices[j] */
        size_t temp = indices[i];
        indices[i] = indices[j];
        indices[j] = temp;
    }
}
```

`rngShuffleIndices` implements the Fisher-Yates shuffle. Starting from the last element, it picks a random position j from $[0, i]$ and swaps elements i and j . Each permutation is equally likely (unbiased). This is used by

`DataLoader` to shuffle sample indices at the start of each epoch.

Chapter 26 — Model Serialisation: Saving and Loading Weights

The `serialize` module writes tensor data to a byte stream (UART, flash, or memory). On the STM32, trained weights can be transferred over UART to a host computer or saved to flash for later inference.

26.1 Serialisation Format

```
/* Tensor serialisation format: */
[ 4 bytes ] numberOfDimensions
[ 4 bytes ] dimensions[0]
[ 4 bytes ] dimensions[1]
... dimensions[numberOfDimensions-1]
[ 4 bytes ] qType (0=INT32, 1=FLOAT32, 2=SYM_INT32, ...)
[ 4 bytes ] scale (for SYM_INT32, as raw float bits)
[ 4 bytes ] qMaxBits (for SYM_INT32)
[ n×4 bytes ] tensor data (n = product of all dimensions)
/* serializeTensor writes this structure to a uint8_t output buffer. */
/* deserializeTensor reads it back and reconstructs the tensor_t. */
```

26.2 `serializeTensor` Implementation

```
size_t serializeTensor(tensor_t *tensor, uint8_t *outputBuffer) {
    uint8_t *ptr = outputBuffer;
    size_t nDims = tensor->shape->numberOfDimensions;
    /* Write numberOfDimensions */
    writeInt32ToByteArray((int32_t)nDims, ptr); ptr += 4;
    /* Write each dimension */
    for (size_t i = 0; i < nDims; i++) {
        writeInt32ToByteArray((int32_t)tensor->shape->dimensions[i], ptr);
        ptr += 4;
    }
    /* Write quantization type */
    writeInt32ToByteArray((int32_t)tensor->quantization->qType, ptr); ptr += 4;
    /* Write scale (as raw float bits) */
    if (tensor->quantization->qType == SYM_INT32) {
        symInt32QConfig_t *cfg = tensor->quantization->qConfig;
        writeFloatToByteArray(cfg->scale, ptr); ptr += 4;
        writeInt32ToByteArray((int32_t)cfg->qMaxBits, ptr); ptr += 4;
    }
    /* Write data bytes */
    size_t dataBytes = calcNumberOfBytesForData(tensor->quantization,
        calcNumberOfElementsByTensor(tensor));
    memcpy(ptr, tensor->data, dataBytes);
    ptr += dataBytes;
    return (size_t)(ptr - outputBuffer); /* total bytes written */
}
```

The function uses a 'running pointer' pattern: `ptr` starts at `outputBuffer` and advances by 4 bytes (or `dataBytes`) after each write. The return value is `ptr - outputBuffer`, which equals the total number of bytes written. The caller needs this to know how many bytes to transmit or save.

EXAMPLE

Serialising a 2×3 SYM_INT32 tensor (scale=0.1, qMaxBits=16): Offset Content 0-3 02 00 00 00
(numberOfDimensions = 2) 4-7 02 00 00 00 (dimensions[0] = 2) 8-11 03 00 00 00 (dimensions[1] = 3) 12-15 02 00 00
00 (qType = 2 = SYM_INT32) 16-19 CD CC CC 3D (scale = 0.1 in IEEE 754 little-endian) 20-23 10 00 00 00
(qMaxBits = 16) 24-47 data bytes (6 × 4 = 24 bytes of int32_t mantissas) Total: 48 bytes

Chapter 27 — DataLoader: Batching and Augmentation

The `DataLoader` module handles epoch-level iteration: it shuffles the dataset, applies transforms to each sample, and assembles mini-batches of training data. This separates I/O and preprocessing logic from the training loop itself.

27.1 DataLoader Fields Explained

Field	Type	Purpose
<code>indices</code>	<code>size_t*</code>	Array of sample indices [0, N-1], shuffled each epoch
<code>currentIndex</code>	<code>size_t</code>	Pointer into indices: which sample is next
<code>batchSize</code>	<code>size_t</code>	How many samples per batch
<code>numSamples</code>	<code>size_t</code>	Total dataset size N
<code>getSample</code>	<code>getSampleFn_t</code>	User callback: loads one sample by index
<code>transform</code>	<code>transformFn_t</code>	User callback: applies augmentation to a sample
<code>rng</code>	<code>rng32_t*</code>	RNG used to shuffle indices each epoch

27.2 The Training Loop with DataLoader

```
/* Typical usage pattern: */
dataLoader_t loader;
initDataLoader(&loader, numSamples, batchSize, getSample, transform, &rng);
for (int epoch = 0; epoch < numEpochs; epoch++) {
    /* Shuffle indices at the start of each epoch */
    shuffleDataLoader(&loader);
    loader.currentIndex = 0;
    while (dataLoaderHasNext(&loader)) {
        /* Fill batchInput and batchLabels tensors */
        dataLoaderNext(&loader, &batchInput, &batchLabels);
        /* dataLoaderNext internally calls: */
        /* for each sample in batch: */
        /* idx = loader.indices[currentIndex++] */
        /* getSample(idx, &sampleTensor) */
        /* transform(&sampleTensor, &augmented) */
        /* copy augmented into batch slot */
        /* Train on this batch: */
        sgdZeroGrad(&optimizer);
        forwardPass(&model, &batchInput, &predictions);
        /* ... loss + backward + sgdStep ... */
    }
}
```

The `getSample` callback is user-defined. On an STM32 it typically reads a sample from flash memory or receives it over UART. The `transform` callback applies data augmentation (e.g. random noise, time-shift) to improve generalisation. Separating these as callbacks means the `DataLoader` code never needs to know where the data comes from — it just calls the provided function pointer.

Chapter 28 — Quantization Deep Dive: Why Scale Matters

This chapter works through the quantization math in detail, showing what happens to accuracy when scale is chosen well versus poorly.

28.1 Choosing Scale — The Two Extremes

```
/* Scenario: float tensor = [0.001, 0.002, 0.003, ... , 1.000] */
/* qMaxBits = 8 → qMax = 127 */
/* Case A: scale too large (scale = 10.0) */
/* mantissa = round(1.0 / 10.0) = 0 */
/* All values round to 0! (catastrophic precision loss) */
/* Case B: scale too small (scale = 0.00001) */
/* mantissa = round(1.0 / 0.00001) = 100000 */
/* Exceeds qMax=127 → clamped to 127 for 1.0, and also */
/* 0.001/0.00001=100, 0.002/0.00001=200 (also clamped) */
/* Saturation: all large values collapse to 127 */
/* Case C: optimal scale (scale = 1.0 / 127 ≈ 0.00787) */
/* mantissa = round(1.0 / 0.00787) = round(127.0) = 127 */
/* mantissa = round(0.5 / 0.00787) = round(63.5) = 64 */
/* mantissa = round(0.001 / 0.00787) = round(0.13) = 0 */
/* Very small values (< scale/2) lose to rounding, */
/* but the full range is used efficiently. */
```

28.2 Quantization Error as a Function of Scale

The maximum representable real value is $qMax \times scale$. Values larger than this are clamped (saturation error). The spacing between adjacent representable values is $scale$. Values are rounded to the nearest multiple of $scale$ (rounding error $\leq scale/2$). The optimal scale minimises the sum of saturation and rounding error. For a uniform distribution over $[-A, A]$, the optimal scale is $A / qMax$. `requantSymInt32Tensor` automatically finds this by computing $absMax / qMax$.

28.3 SYM vs ASYM Quantization

```
/* Symmetric (SYM_INT32): */
real = mantissa × scale
range: [-qMax×scale, qMax×scale] – centred at zero
Best for: weights, gradients, activations after ReLU is NOT used
/* Asymmetric (ASYM): */
real = (mantissa - zeroPoint) × scale
range: [-zeroPoint×scale, (2^n - 1 - zeroPoint)×scale]
Best for: activations after ReLU (range is [0, max], non-symmetric)
/* Example: ReLU activations [0, 0, 0.3, 0.7, 1.0] */
/* SYM would waste half the integer range (negative side) on zero. */
/* ASYM sets zeroPoint so that 0.0 maps to integer 0: */
/* zeroPoint = 0, scale = 1.0/255 for uint8 – range [0, 1.0] */
```

Chapter 29 — FQT and RigL: Library Design in Context

This chapter explains how the ODT library's architecture was specifically designed to support Fixed-point Quantization Training (FQT) and the RigL sparse training method for deployment on the STM32 Nucleo-F746ZG microcontroller.

29.1 What is FQT?

Fixed-point Quantization Training (following Deutel et al.) trains the model from scratch using integer arithmetic. Unlike post-training quantization (which quantizes after training and often degrades accuracy), FQT trains the model to be robust to quantization by keeping activations and weights in SYM_INT32 throughout training. The two-pass requantisation in `requantSymInt32Tensor` is the core enabler: it continuously re-scales layer outputs to prevent overflow while maintaining maximum precision.

```
/* FQT training loop (simplified): */
/* */
/* All tensors: SYM_INT32 with qMaxBits=16 */
/* */
/* Forward pass: */
/* linearForward → matmulSymInt32TensorsWithBias */
/* → requantSymInt32Tensor (re-scale) */
/* reluForward → reluForwardSymInt32 (clip negatives to 0) */
/* softmaxForward → convert to float for exp, back to SYM */
/* */
/* Backward pass: */
/* gradients computed in FLOAT32 (for precision) */
/* weights updated in FLOAT32, then re-quantized before next */
/* forward pass */
/* */
/* Key insight: forward pass uses integer math (fast on STM32), */
/* backward pass uses float (can run on the FPU). */
```

29.2 What is RigL?

RigL (Rigging the Lottery, Evci et al. 2020) is a sparse training algorithm. It starts with a randomly pruned (sparse) network and periodically regrows connections: at each regrowth step it prunes the smallest-magnitude weights and adds new connections at the positions with the largest gradient magnitudes. The `sparsity_t` struct in `tensor_t` supports this: it stores the mask of which weights are active. The optimizer only updates active weights.

```
/* RigL update cycle (every T iterations): */
/* 1. Compute gradient magnitudes for ALL potential connections */
/* (including currently pruned ones – zero weights still get */
/* non-zero gradients from the backward pass) */
findAbsMaxFloat(weightGradTensor, &gradMax);
/* 2. Prune: remove k% of smallest-magnitude active weights */
/* Update sparsity mask to mark these as inactive */
pruneSmallestWeights(layer, pruneRatio);
/* 3. Regrow: activate the k% connections with largest */
/* gradient magnitude among currently inactive positions */
regrowLargestGradients(layer, pruneRatio);
/* 4. Zero the newly grown weights (they start at 0.0) */
zeroNewWeights(layer);
```

29.3 How the Library Supports Both

The `sparsity_t` field in `tensor_t` holds the active-weight mask. All arithmetic functions check for sparsity before operating: if sparsity is not NULL, only active elements are read/written. The `SYM_INT32` quantization type supports the FQT forward pass with integer matmul. The `FLOAT32` type supports the gradient backward pass with FPU arithmetic. The `requantSymInt32Tensor` function bridges between the two: it accepts float-updated weights and converts them back to `SYM_INT32` for the next forward pass. This design allows the ODT library to run FQT+RigL on the STM32 with minimal memory overhead — no separate quantized and float copies of the model are needed simultaneously.

Chapter 30 — Complete Function Index with File Cross-Reference

This table lists every public function in the library, the file that declares it, the file that defines it (implementation), and which other functions call it.

Function	Declared In	Defined In	Called By
readBytesAsInt32	DTypes.h	DTypes.c	requantSymInt32Tensor, int32PointWise*, linearForward
readBytesAsFloat	DTypes.h	DTypes.c	floatPointWise*, softmaxForward, sgdStep
writeInt32ToByteArray	DTypes.h	DTypes.c	requantSymInt32Tensor, int32PointWise*, matmulSym*
writeFloatToByteArray	DTypes.h	DTypes.c	floatPointWise*, softmaxForward, serializeTensor
initSymInt32QConfig	Quantization.h	Quantization.c	user init code
initSymInt32Quantization	Quantization.h	Quantization.c	user init code
initFloat32QConfig	Quantization.h	Quantization.c	user init code
initFloat32Quantization	Quantization.h	Quantization.c	user init code
calcNumberOfBytesForData	Quantization.h	Quantization.c	setTensorValues, serializeTensor
setTensorValues	Tensor.h	Tensor.c	user init code
setShape	Tensor.h	Tensor.c	user init code
setOrderOfDimsForNewTensor	Tensor.h	Tensor.c	user init code
transposeTensor	Tensor.h	Tensor.c	linearForward, linearBackward, matmulSym*
calcNumberOfElementsByTensor	Tensor.h	Tensor.c	int32PointWise*, floatPointWise*, requant*, sgdStep
calcIndicesByRowIndex	Tensor.h	Tensor.c	int32PointWise*, floatPointWise*
calcTensorIndexByIndices	Tensor.h	Tensor.c	calcElementIndexByIndices
calcElementIndexByIndices	Tensor.h	Tensor.c	int32PointWise*, floatPointWise*
copyTensor	Tensor.h	Tensor.c	linearForward (pre-act copy), linearBackward (savedInput)
zeroTensorData	Tensor.h	Tensor.c	sgdZeroGrad
convertTensor	TensorConversion.h	TensorConversion.c	user code, layer transitions
requantSymInt32Tensor	TensorConversion.h	TensorConversion.c	linearForward, conv1dForward
requantSymInt32TensorToScale	TensorConversion.h	TensorConversion.c	user code, fixed-scale paths
convertFloatTensorToSymInt32Tensor	TensorConversion.h	TensorConversion.c	FQT weight conversion
int32PointWiseArithmetic	Arithmetic.h	Arithmetic.c	addInt32Tensors, subInt32Tensors, ...
floatPointWiseArithmetic	Arithmetic.h	Arithmetic.c	addFloat32Tensors, subFloat32Tensors, ...
addInt32s	Add.h	Add.c	addInt32Tensors, addInt32TensorsInplace
addFloat32s	Add.h	Add.c	addFloat32Tensors, addFloat32TensorsInplace
addInt32Tensors	Add.h	Add.c	arithmetic operations
addFloat32Tensors	Add.h	Add.c	sgdStep, linearBackward
subFloat32TensorsInplace	Sub.h	Sub.c	sgdStep, sgdStepM
subInt32Tensors	Sub.h	Sub.c	arithmetic operations
mulFloat32ElementWithFloat32TensorInplace	Mul.h	Mul.c	sgdStep, sgdStepM
mullInt32Tensors	Mul.h	Mul.c	arithmetic operations
matmulSymInt32TensorsWithBias	Matmul.h	Matmul.c	linearForward, conv1dForward
matmulFloat32TensorsWithBias	Matmul.h	Matmul.c	linearForward, linearBackward

findAbsMaxFloat	MinMax.h	MinMax.c	requant*, convertFloat*
findMaxInt32	MinMax.h	MinMax.c	user code
findMinInt32	MinMax.h	MinMax.c	user code
roundByMode	Rounding.h	Rounding.c	requantSymInt32Tensor
sumFloat	Sum.h	Sum.c	linearBackward (bias grad), softmaxForward
logFloat	Log.h	Log.c	CrossEntropy loss forward
initLayer	Layer.h	Layer.c	initLinear, initConv1d, initRelu, ...
initLinear	Linear.h	Linear.c	user network init
linearForward	Linear.h	Linear.c	layer dispatch table
linearBackward	Linear.h	Linear.c	backward pass loop
sgdInit	Sgd.h	Sgd.c	user init
sgdStep	Sgd.h	Sgd.c	training loop
sgdStepM	Sgd.h	Sgd.c	training loop (momentum)
sgdZeroGrad	Sgd.h	Sgd.c	training loop (every batch)
serializeTensor	Serialize.h	Serialize.c	serializeParameter, serializeModel
serializeParameter	Serialize.h	Serialize.c	serializeModel
serializeModel	Serialize.h	Serialize.c	user code (save weights)
rngSetSeed	RNG.h	RNG.c	user init
rngNextFloat	RNG.h	RNG.c	rngShuffleIndices, Bernoulli
rngShuffleIndices	RNG.h	RNG.c	shuffleDataLoader
initDataLoader	DataLoader.h	DataLoader.c	user init
dataLoaderNext	DataLoader.h	DataLoader.c	training loop
shuffleDataLoader	DataLoader.h	DataLoader.c	training loop (start of epoch)

Chapter 31 — Line-by-Line: softmaxForward

Softmax converts a vector of raw scores (logits) to a probability distribution: every element is positive and they all sum to 1. It is always the last layer before the cross-entropy loss in a classifier.

31.1 The Numerical Stability Problem

The naive formula $\text{output}[i] = \exp(x[i]) / \sum_j(\exp(x[j]))$ overflows when $x[i]$ is large (e.g. $\exp(1000) = +\text{inf}$ in float32). The standard fix is to subtract the maximum value first: $\text{output}[i] = \exp(x[i] - \text{max_x}) / \sum_j(\exp(x[j] - \text{max_x}))$. This is mathematically identical but numerically safe because all exponents are now ≤ 0 , so $\exp() \leq 1$.

```

void softmaxForward(tensor_t *input, tensor_t *output) {
    size_t n = calcNumberOfElementsByTensor(input);
    /* Pass 1: find max (for numerical stability) */
    float maxVal = readBytesAsFloat(&input->data[0]);
    for (size_t i = 1; i < n; i++) {
        float v = readBytesAsFloat(&input->data[i * sizeof(float)]);
        if (v > maxVal) maxVal = v;
    }
    /* Pass 2: compute exp(x[i] - maxVal) and accumulate sum */
    float sumExp = 0.0f;
    for (size_t i = 0; i < n; i++) {
        float v = readBytesAsFloat(&input->data[i * sizeof(float)]);
        float e = expf(v - maxVal);
        writeFloatToByteArray(e, &output->data[i * sizeof(float)]);
        sumExp += e;
    }
    /* Pass 3: divide each exp by sumExp to normalise */
    for (size_t i = 0; i < n; i++) {
        float e = readBytesAsFloat(&output->data[i * sizeof(float)]);
        writeFloatToByteArray(e / sumExp, &output->data[i * sizeof(float)]);
    }
}

```

4	float maxVal = readBytesAsFloat(&input->data[0]);	Seed maxVal with the FIRST element. Starting at element 0 ensures the loop comparison always has a valid reference. If we started with 0.0f or -FLT_MAX, we could fail for unusual distributions.
5	for (size_t i = 1; i < n; i++) {	Start from i=1 — we already have element 0 as the initial maxVal.
11	float sumExp = 0.0f;	Accumulator for the denominator. Must be initialised to 0 before the loop.
14	float e = expf(v - maxVal);	expf is the single-precision (float) version of exp. Using expf instead of exp prevents implicit conversion to double which would waste FPU cycles on ARM Cortex-M7 (which has a single-precision FPU). Subtracting maxVal ensures $v - \text{maxVal} \leq 0$, so $e = \exp(v - \text{maxVal})$ is in (0, 1]. This prevents the overflow that would occur with large logits.
15	writeFloatToByteArray(e, &output->data[i * sizeof(float)]);	Write the UNNORMALIZED exp value to the output tensor. We cannot normalise yet because we don't know sumExp until the loop finishes. The output temporarily holds exp values — they are overwritten by normalised probabilities in Pass 3.
21	float e = readBytesAsFloat(&output->data[i * sizeof(float)]);	Read back the unnormalized exp that was written in Pass 2.
22	writeFloatToByteArray(e / sumExp, &output->data[i * sizeof(float)]);	Divide by sumExp to normalise. After this, all output[i] sum to 1.0 (barring floating-point rounding error, typically < 1e-6).

EXAMPLE

Trace for input = [2.0, 1.0, 0.1] (3 logits for 3 classes): Pass 1: maxVal = 2.0 Pass 2: i=0: $e = \exp(2.0 - 2.0) = \exp(0) = 1.000$; sumExp = 1.000 i=1: $e = \exp(1.0 - 2.0) = \exp(-1) = 0.368$; sumExp = 1.368 i=2: $e = \exp(0.1 - 2.0) = \exp(-1.9) = 0.150$; sumExp = 1.518 Pass 3 (divide by 1.518): output[0] = $1.000 / 1.518 = 0.659$ output[1] = $0.368 / 1.518 = 0.242$ output[2] = $0.150 / 1.518 = 0.099$ Sum check: $0.659 + 0.242 + 0.099 = 1.000$ ✓ Interpretation: class 0 has 65.9% probability, class 1 has 24.2%, class 2 has 9.9%.

Chapter 32 — Memory Layout on the STM32: No Dynamic Allocation

Embedded systems like the STM32 Nucleo-F746ZG have 320 KB of SRAM and no operating system memory manager. The ODT library is designed to avoid malloc/free entirely — all tensor buffers, config structs, and working buffers are either statically allocated (global or function-scope static) or declared as stack VLAs inside functions. This section explains the memory layout and why each choice was made.

32.1 STM32F746ZG Memory Map (Relevant Regions)

```
FLASH (1 MB, 0x08000000 - 0x080FFFFF):  
- Program code (.text)  
- Read-only constants (.rodata)  
- Constant tensor weights (if stored in flash)  
DTCM RAM (128 KB, 0x20000000 - 0x2001FFFF):  
- Fast access: 1 cycle by Cortex-M7 core  
- Place: hot tensors (input/output/activation buffers)  
SRAM1 (192 KB, 0x20020000 - 0x2004FFFF):  
- General purpose  
- Place: weight tensors, gradient buffers, optimizer state  
Stack (grows down from top of DTCM RAM):  
- VLAs (index arrays, per-function temporaries)  
- Function frames  
- Default size: 8 KB (configurable in linker script)
```

32.2 How the Library Avoids malloc

What	How it is allocated
tensor_t struct	Stack variable or file-scope global in user code
shape_t struct	Stack variable or file-scope global
quantization_t struct	Stack variable or file-scope global
qConfig (symInt32QConfig_t etc.)	Stack variable or global
Data buffer (uint8_t[])	Static array dimensioned at compile time: uint8_t buf[N*4]
dimensions[] array in shape_t	Static array: size_t dims[MAX_DIMS]
orderOfDimensions[] in shape_t	Static array: size_t order[MAX_DIMS]
VLAs inside arithmetic functions	Stack, freed on function return
layerConfig_t (e.g. linearConfig_t)	Static global or placement in static pool
savedInput buffer	Static array allocated once at model init
weightGradient buffer	Static array allocated once at model init

NOTE

Rule: every buffer that persists across function calls must be declared as a static (global or function-local-static) array. Buffers that only live within a single function call may be stack VLAs. The user is responsible for sizing the static arrays at compile time — the library never calls malloc or free.

```
/* Correct static allocation pattern for a 2x3 float32 tensor: */
static uint8_t dataBuffer[2 * 3 * sizeof(float)]; /* = 24 bytes */
static size_t dims[2] = {2, 3};
static size_t order[2] = {0, 1};
static float32QConfig_t qCfg;
static quantization_t quant;
static shape_t shape;
static tensor_t tensor;
/* This uses ZERO heap. All memory is in the BSS/data segment. */
/* Avoid: */
float32QConfig_t *qCfg = malloc(sizeof(float32QConfig_t)); /* NEVER DO THIS */
```

Chapter 33 — Debugging: DLEVEL and PRINT_ERROR in Depth

Debugging on an embedded system with no display is challenging. The ODT library provides a macro-based debug printing system that compiles out completely in production builds. This chapter explains every aspect of the system.

33.1 The Complete Common.h Debug System

```
/* Common.h — full content: */
#ifndef COMMON_H
#define COMMON_H
#include <stdio.h> /* for printf */
#include <stdlib.h> /* for exit() */
/* DLEVEL is set by compiler flags, e.g.: -DDEBUG_MODE_DEBUG */
#if defined DEBUG_MODE_DEBUG
#define DLEVEL 3
#elif defined DEBUG_MODE_INFO
#define DLEVEL 2
#elif defined DEBUG_MODE_WARNING
#define DLEVEL 1
#else
#define DLEVEL 0 /* production: no debug output */
#endif
#define PRINT_ERROR(str, ...) do { \
printf("ERROR: " str "\n", ##__VA_ARGS__); \
} while(false)
#if DLEVEL >= 1
#define PRINT_WARNING(str, ...) do { \
printf("WARNING: " str "\n", ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_WARNING(str, ...) do { } while(false)
#endif
#if DLEVEL >= 2
#define PRINT_INFO(str, ...) do { \
printf("INFO: " str "\n", ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_INFO(str, ...) do { } while(false)
#endif
#if DLEVEL >= 3
#define PRINT_DEBUG(str, ...) do { \
printf("DEBUG: " str "\n", ##__VA_ARGS__); \
} while(false)
#else
#define PRINT_DEBUG(str, ...) do { } while(false)
#endif
#endif /* COMMON_H */
```

9

#if defined DEBUG_MODE_DEBUG

Conditional compilation: if the symbol `DEBUG_MODE_DEBUG` is defined (via `-DDEBUG_MODE_DEBUG` passed to the compiler), set `DLEVEL=3`. `'#if defined X'` is equivalent to `'#ifdef X'`. The chain of `#elif` checks goes from most verbose to least verbose.

16	<code>#define DLEVEL 0 /* production: no debug output */</code>	If none of the debug symbols are defined, DLEVEL=0. This is the default for release/production builds. All PRINT_WARNING, PRINT_INFO, PRINT_DEBUG macros expand to empty do{} while blocks — the compiler removes them entirely (zero code size, zero runtime cost).
18	<code>#define PRINT_ERROR(str, ...) do { \</code>	PRINT_ERROR is ALWAYS compiled in (no DLEVEL guard). Errors are fatal and must always be reported, even in production. The \ at the end continues the macro definition to the next line.
20	<code>printf("ERROR: " str "\n", ##__VA_ARGS__); \</code>	String concatenation: 'ERROR: ' and str are two adjacent string literals. The C preprocessor concatenates adjacent string literals at compile time. ##__VA_ARGS__ expands to the variadic arguments (0 or more), with ## suppressing the comma if there are no arguments.
21	<code>} while(false)</code>	The do { } while(false) wrapper makes the macro safe to use after 'if' without braces: if (err) PRINT_ERROR("problem"); Without the wrapper, the macro expands to two statements — the 'if' only guards the first statement. With the wrapper, the entire block is one statement.
23	<code>#if DLEVEL >= 1</code>	Guards PRINT_WARNING for any build with DLEVEL 1, 2, or 3. The #if preprocessor directive evaluates DLEVEL as a compile-time integer expression.
26	<code>#define PRINT_WARNING(str, ...) do { } while(false)</code>	When DLEVEL < 1, PRINT_WARNING expands to an empty do-while. The compiler's dead-code elimination removes it. This pattern gives the same call-site syntax regardless of build config.

33.2 How to Control Debug Level at Build Time

```
/* Compiler flags to pass to GCC / arm-none-eabi-gcc: */
Production build (no output): (no flag – DLEVEL=0 by default)
Enable warnings: -DDEBUG_MODE_WARNING
Enable info + warnings: -DDEBUG_MODE_INFO
Enable all (debug + info + warn): -DDEBUG_MODE_DEBUG
/* In a Makefile: */
CFLAGS += -DDEBUG_MODE_DEBUG
/* In CMakeLists.txt: */
target_compile_definitions(my_target PRIVATE DEBUG_MODE_DEBUG)
/* On STM32CubeIDE: */
/* Project → Properties → C/C++ Build → Settings → */
/* Tool Settings → MCU GCC Compiler → Preprocessor → */
/* Add: DEBUG_MODE_DEBUG */
```

33.3 Redirecting printf to UART on STM32

```
/* On STM32, printf must be redirected to a UART peripheral. */
/* Add this to your main.c or a dedicated retarget.c file: */
#include <stdio.h>
#include "usart.h" /* STM32 HAL UART header */
int __io_putchar(int ch) {
    HAL_UART_Transmit(&huart3, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
    return ch;
}
/* Now all printf output goes to UART3. */
/* Connect UART3 to a USB-to-serial adapter and open a terminal */
/* (e.g. PuTTY, minicom, screen) at 115200 baud to see output. */
```


Chapter 34 — Line-by-Line: matmulSymInt32TensorsWithBias

This is the most computationally intensive function in the library — it performs the matrix multiply at the core of every Linear and Conv1d layer using 32-bit integer arithmetic. It is the function that will consume the most clock cycles during inference on the STM32.

```
void matmulSymInt32TensorsWithBias(
    tensor_t *aTensor, /* input: [batch × inFeatures] */
    tensor_t *bTensor, /* weights: [inFeatures × outFeatures] */
    tensor_t *bias, /* bias: [outFeatures] or NULL */
    tensor_t *output) { /* result: [batch × outFeatures] */
    symInt32QConfig_t *aCfg = aTensor->quantization->qConfig;
    symInt32QConfig_t *bCfg = bTensor->quantization->qConfig;
    float aScale = aCfg->scale;
    float bScale = bCfg->scale;
    size_t batch = aTensor->shape->dimensions[0];
    size_t inFeatures = aTensor->shape->dimensions[1];
    size_t outFeatures = bTensor->shape->dimensions[1];
    for (size_t b = 0; b < batch; b++) {
        for (size_t j = 0; j < outFeatures; j++) {
            int64_t acc = 0; /* 64-bit accumulator */
            for (size_t k = 0; k < inFeatures; k++) {
                int32_t aVal = readBytesAsInt32(
                    &aTensor->data[(b * inFeatures + k) * 4]);
                int32_t bVal = readBytesAsInt32(
                    &bTensor->data[(k * outFeatures + j) * 4]);
                acc += (int64_t)aVal * (int64_t)bVal;
            }
            /* Convert accumulator to real value, then to output mantissa */
            float realVal = (float)acc * aScale * bScale;
            if (bias != NULL) {
                int32_t biasMantissa = readBytesAsInt32(&bias->data[j * 4]);
                symInt32QConfig_t *biasCfg = bias->quantization->qConfig;
                realVal += (float)biasMantissa * biasCfg->scale;
            }
            /* Store as float (to be requantised later) */
            writeFloatToByteArray(realVal, &output->data[(b*outFeatures+j)*4]);
        }
    }
}
```

6	<code>symInt32QConfig_t *aCfg = aTensor->quantization->qConfig;</code>	Cast the generic void* qConfig to the concrete type. Safe here because the function is only called when both tensors are SYM_INT32.
10	<code>float aScale = aCfg->scale;</code>	Latch both scales before the triple-nested loop. Accessing them inside the inner loop would follow three pointer dereferences (tensor → quantization → qConfig → scale) on every iteration. Latching to locals lets the compiler keep them in registers.
19	<code>int64_t acc = 0; /* 64-bit accumulator */</code>	The accumulator is 64-bit (int64_t). Why? Each multiply is int32 × int32 — the result can be up to 2 ⁶² , which overflows int32_t (max 2 ³¹ -1). int64_t can hold up to 2 ⁶³ -1 — safe for thousands of int32×int32 products.

22	<code>int32_t aVal = readBytesAsInt32(</code>	Read input element A[b, k]: batch item b, feature k. Flat byte offset = $(b \times \text{inFeatures} + k) \times 4$ bytes per <code>int32_t</code> .
24	<code>int32_t bVal = readBytesAsInt32(</code>	Read weight B[k, j]: weight connecting input k to output j. Note: <code>bTensor</code> was transposed before this call in <code>linearForward</code> , so <code>B[k][j]</code> is stored at flat index $k \times \text{outFeatures} + j$.
26	<code>acc += (int64_t)aVal * (int64_t)bVal;</code>	Widening multiply: cast both operands to <code>int64_t</code> before multiplying. Without the cast, $aVal \times bVal$ is computed as <code>int32</code> $\times$ <code>int32</code> = <code>int32</code> which overflows. The cast promotes the multiplication to 64-bit arithmetic. This is the inner loop of the matrix multiply — it runs $\text{batch} \times \text{outF} \times \text{inF}$ times.
29	<code>float realVal = (float)acc * aScale * bScale;</code>	Convert the integer accumulation to a real value. The accumulated sum = $\sum_k A_mantissa[k] \times B_mantissa[k]$. The real value = $\sum_k (A_mantissa \times aScale) \times (B_mantissa \times bScale) = (\sum_k A_mantissa \times B_mantissa) \times aScale \times bScale = \text{acc} \times aScale \times bScale$. The scales factor out of the sum, which is why we can accumulate pure integers and scale only once at the end.
32	<code>realVal += (float)biasMantissa * biasCfg->scale;</code>	Add the bias as a real value. The bias has its own scale (set when the bias was initialised). Adding in real space is correct because both <code>realVal</code> and <code>bias</code> are in the same physical units (before normalisation).
37	<code>writeFloatToByteArray(realVal, &output->data[(b*outFeatures+j)*4]);</code>	Write the result as a <code>FLOAT32</code> even though input was <code>SYM_INT32</code> . <code>linearForward</code> will call <code>requantSymInt32Tensor</code> afterwards to convert back to <code>SYM_INT32</code> with an appropriate scale. Writing as float avoids choosing an ad-hoc scale here.

EXAMPLE

Full trace: 1×2 input $A = \begin{bmatrix} 3 & -2 \end{bmatrix}$, 2×2 weight $B = \begin{bmatrix} 1 & 4 \\ 2 & -1 \end{bmatrix}$, $aScale = 0.5$, $bScale = 0.25$, no bias. After transpose: $B^T = \begin{bmatrix} 1 & 2 \\ 4 & -1 \end{bmatrix}$ output[0, 0]: $\text{acc} = 3 \times 1 + (-2) \times 2 = 3 - 4 = -1$ $\text{realVal} = (-1) \times 0.5 \times 0.25 = -0.125$ output[0, 1]: $\text{acc} = 3 \times 4 + (-2) \times (-1) = 12 + 2 = 14$ $\text{realVal} = 14 \times 0.5 \times 0.25 = 1.75$ Output buffer: $[-0.125, 1.75]$ as float32 bytes. Then `requantSymInt32Tensor` converts these to `SYM_INT32` with $\text{newScale} = 1.75/qMax$.

Chapter 35 — Full Project File Tree with Purpose Annotations

This chapter lists every source file in the `OnDeviceTraining` project directory, explains its purpose, and shows how it fits into the 8-layer architecture.

```
OnDeviceTraining/
src/common/include/Common.h Layer 0: debug macros, always included
src/tensor/include/DTypes.h Layer 1: byte read/write primitives
src/tensor/include/Quantization.h Layer 2: quantization types & config
src/tensor/include/Tensor.h Layer 3: tensor_t struct & shape utilities
src/tensor/include/TensorConversion.h Layer 4: type conversion dispatch table
src/tensor/DTypes.c Quantization.c Tensor.c TensorConversion.c
src/arithmetic/include/Arithmetic.h Layer 5: generic element-wise engine
src/arithmetic/include/Add.h add operations (int32, float, mixed)
src/arithmetic/include/Sub.h subtract operations
src/arithmetic/include/Mul.h multiply operations
src/arithmetic/include/Div.h divide operations
src/arithmetic/include/Matmul.h matrix multiply (int32 & float)
src/arithmetic/include/MinMax.h find max/min/absMax in a tensor
src/arithmetic/include/Sum.h reduce-sum over an axis
src/arithmetic/include/Log.h element-wise natural log
src/arithmetic/include/Rounding.h round/clamp with rounding mode
src/arithmetic/Arithmetic.c Add.c Sub.c Mul.c Div.c
src/arithmetic/Matmul.c MinMax.c Sum.c Log.c Rounding.c
```

```
src/layer/include/Layer.h Layer 6: layer_t base struct, dispatch
src/layer/include/Linear.h fully-connected layer
src/layer/include/Conv1d.h 1D convolution layer
src/layer/include/Relu.h ReLU activation
src/layer/include/Softmax.h softmax (last layer of classifier)
src/layer/include/Dropout.h dropout regularisation
src/layer/include/LayerNorm.h layer normalisation
src/layer/include/AvgPool1d.h average pooling (1D)
src/layer/include/MaxPool1d.h max pooling (1D)
src/layer/include/AdaptiveAvgPool1d.h adaptive average pooling
src/layer/include/QuantizationLayer.h quantize/dequantize as a layer
src/layer/include/Flatten.h reshape [batch,C,L] to [batch,C*L]
src/layer/*.c (one .c file per layer header)
src/loss_functions/include/LossFunction.h Layer 7: loss registry
src/loss_functions/include/CrossEntropy.h cross-entropy fwd + bwd
src/loss_functions/include/MSE.h mean-squared-error fwd + bwd
src/optimizer/include/Optimizer.h optimizer_t base struct
src/optimizer/include/Sgd.h SGD and SGD+momentum
src/optimizer/Sgd.c
src/serial/include/Serialize.h write tensors/model to byte stream
src/serial/include/Deserialize.h read tensors/model from byte stream
src/serial/Serialize.c Deserialize.c
src/rng/include/RNG.h xorshift32 random number generator
src/rng/include/Bernoulli.h Bernoulli sampling (for Dropout)
src/rng/RNG.c Bernoulli.c
src/data_loader/include/DataLoader.h batch iterator with shuffle + transform
src/data_loader/include/NPYLoader.h load .npy files from flash/UART
src/data_loader/DataLoader.c NPYLoader.c
test/ (unit tests: run on host, not MCU)
test/test_tensor.c test_arithmetic.c test_layers.c ...
```

Chapter 36 — End-to-End Training Checklist

Use this checklist when setting up a new ODT training run on the STM32. Each step references the function(s) responsible.

Step	Action	Key Functions
1	Allocate static buffers for all tensors (input, weights, biases, gradients, activations)	uint8_t buf[N*4] declarations
2	Initialise quantization configs for each tensor	initSymInt32QConfig, initFloat32QConfig
3	Initialise quantization_t for each tensor	initSymInt32Quantization, initFloat32Quantization
4	Set up shape_t for each tensor	setOrderOfDimsForNewTensor, setShape
5	Link buffers, shapes, and quantizations into tensor_t	setTensorValues
6	Initialise each layer (allocates config, sets fwd/bwd pointers)	initLinear, initConv1d, initRelu, initSoftmax, ...
7	Initialise the optimizer	sgdInit (or sgdInitM for momentum)
8	Initialise the RNG with a seed	rngSetSeed
9	Initialise the DataLoader with getSample + transform callbacks	initDataLoader
10	For each epoch: shuffle the DataLoader	shuffleDataLoader
11	For each batch: zero gradients	sgdZeroGrad
12	Run forward pass through all layers	layer->forward (linearForward, reluForward, softmaxForward, ...)
13	Compute loss (forward only)	lossFunctions[CROSS_ENTROPY].forward
14	Compute initial gradient (loss backward)	lossFunctions[CROSS_ENTROPY].backward
15	Backpropagate through layers in REVERSE order	layer->backward (softmaxBackward, reluBackward, linearBackward, ...)
16	Update weights	sgdStep or sgdStepM
17	Repeat steps 11–16 until DataLoader exhausted (one epoch done)	dataLoaderHasNext, dataLoaderNext
18	Repeat steps 10–17 for all epochs	(outer loop)
19	Serialize trained weights to UART or flash	serializeModel
20	On host: deserialize and use for inference	deserializeModel

NOTE

Every step in the checklist corresponds to one or more functions documented in this reference. Use the Quick Reference table (Chapter 13) or the Function Index (Chapter 30) to look up the exact signature for any function you are not sure about.

This document has covered all 8 layers of the OnDeviceTraining library: from raw byte manipulation in DTypes.c, through the tensor abstraction, the element-wise arithmetic engine, the neural network layer implementations, training infrastructure, and the user-facing API. Every public function has been documented with its signature, purpose, parameters, return value, and worked examples. Critical functions have been walked through line by line. Use this document as a reference whenever you need to understand, extend, or debug any part of the ODT library.